

Statistical Gains from Looped Estimation under Parameter Budgets

Xinyu Tian Xiaotong Shen

School of Statistics, University of Minnesota

Abstract

Memory constraints in artificial intelligence motivate accurate function approximation with fewer parameters. We study looping, which repeatedly composes one update function with shared parameters; each output becomes the next input. A looped Transformer, for example, reuses one block, whereas its conventional untied counterpart uses separately parameterized blocks. We compare their parameter requirements for a given worst-case approximation accuracy, or equivalently, their approximation accuracy under a common budget limiting distinct trainable coefficients. We then ask whether this representational parsimony improves statistical accuracy. For general likelihood models, we establish an upper squared Hellinger risk bound for looped sieve maximum likelihood and a minimax lower bound for the jointly tuned untied family. Further loop iterations improve the approximation bound without adding parameters, while increasing computation and the fitted-class complexity bound. For targets of known Hölder smoothness, looped residual feedforward networks and post-layer-normalized Transformers attain the minimax polynomial rate up to logarithmic factors with a fixed number of bounded real parameters. At sufficiently large fixed budgets, looped worst-case risk vanishes while optimal worst-case untied risk remains bounded away from zero. The loop-to-untied risk ratio also tends to zero under specified growing-budget conditions. Regression, binary response, and energy-based generative models illustrate the theory.

Keywords: nonparametric estimation; parameter sharing; Hellinger risk; Transformer; sieve maximum likelihood.

1 Introduction

Memory constraints in artificial intelligence motivate learning with fewer trainable parameters. During training, parameters, optimizer states, and activations compete for memory (Rajbhandari et al. 2020); during inference, parameter transfers and communication can limit prediction throughput (Pope et al. 2023). Looped architectures offer one response to these resource demands by repeatedly applying a shared module, increasing computational depth without adding distinct trainable parameters. This approach is used in Universal Transformers (Dehghani et al. 2019) and recurrent-depth language models (Geiping et al. 2025), and

enables latent reasoning in ByteDance’s Ouro models (Zhu et al. 2025). More recently, Wang et al. (2026) study SMELT, a sparse mixture-of-experts Transformer, under closely matched parameter, computation, and key–value cache budgets. Looped computation has even been reported in OpenAI’s Astra model (Efrati et al. 2026; Raschka 2026). These developments motivate the question of whether repeated composition with shared parameters can improve approximation accuracy and reduce statistical risk under a common parameter budget.

To study these designs statistically, we view them as function representations. Each representation composes an input function, T update functions, and an output function. Each successive update is one iteration. A looped representation uses the same update function and parameters at every iteration; its conventional untied counterpart uses separately parameterized update functions. In neural networks, each update function corresponds to a block, so T measures effective depth in blocks. At fixed block width, increasing this depth reuses the loop’s fitted coefficients, whereas the untied network requires additional coefficients to be estimated.

To reflect the demand for parameter storage, we impose a parameter budget, an upper bound on the number of distinct trainable scalar coefficients. Statistically, this is a way to study representational parsimony. Over a common target class, we can ask how many coefficients are needed to attain a prescribed worst-case approximation accuracy or, equivalently, how accurately each representation can approximate under a given parameter budget. We adopt the latter view to compare looped and untied representations. Statistical risk depends on both this approximation accuracy and the estimation error incurred in fitting the coefficients to data.

Main finding. With a sufficiently large fixed budget, a looped estimator can attain vanishing worst-case risk. At the same budget, every estimator in the corresponding untied family has worst-case risk bounded away from zero. To see the approximation–estimation balance, fix either the residual feedforward network (FFN) or post-layer-normalized (post-LN) Transformer family, a likelihood example, and a Hölder target class of known smoothness s in dimension d . Write $a = 2s/d$. For this overview, fix the state or token embedding dimension and choose the largest admissible feedforward hidden width under budget B . Let $R_L(n, B, T)$ be the worst-case squared Hellinger risk of the resulting loop sieve maximum likelihood estimator and $R_U(n, B)$ the optimal worst-case risk among estimators valued in the budget- B untied family. Our main results give, under suitable conditions,

$$R_L(n, B, T) \lesssim \left\{ \frac{\log(eBT)}{BT} \right\}^a + \frac{B\{\log(en) + T\log(eB)\}}{n},$$

$$R_U(n, B) \gtrsim \{B^2 \log(eB)\}^{-a}.$$

The implicit constants depend on the fixed target-class, likelihood, and remaining architecture parameters, but not on n, B , or T . The bounds follow from Theorem 4.1 and Proposition 4.3.

For the untied networks considered here, increasing widths or adding blocks increases the parameter count. Improving approximation through a larger network therefore incurs the estimation cost of a larger parameterization. The loop upper bound expresses a different balance: at fixed B , its approximation (first term) and estimation (second term) both

vary with T , while the number of fitted coefficients stays fixed. Additional iterations reduce the approximation bound but increase the complexity bound for the fitted function class and hence the estimation term. Sieve-likelihood theory (Shen and Wong 1994; Wong and Shen 1995) converts this balance into a risk bound. For any sufficiently large fixed B , choosing $T_n = \lceil n^{1/(a+1)} \{\log(en)\}^{a/(a+1)} \rceil$ gives $R_L(n, B, T_n) \lesssim_B \{\frac{\log(en)}{n}\}^{2s/(2s+d)}$. The looped estimator therefore attains the classical Hölder minimax rate up to a logarithmic factor (Stone 1982). By contrast, increasing untied depth requires additional coefficients. Under a fixed budget, even the jointly tuned untied family retains positive worst-case approximation error and risk. This lower bound holds uniformly over all admissible choices of width and depth in the untied family under the same parameter budget, including wide, shallow networks and narrow, deep networks. Consequently, $R_L(n, B, T_n)/R_U(n, B) \rightarrow 0$ for every sufficiently large fixed budget admitting the loop, as stated precisely in Theorem 4.4.

The same bound displays the tradeoff between budget and iteration count. With the state or embedding dimension fixed and logarithmic factors suppressed, the approximation and estimation terms scale as $(BT)^{-a}$ and BT/n . A larger budget permits a wider shared block and comparable approximation with fewer iterations. Theorem 4.5 extends the comparison to sieves in which both state and hidden widths grow. Under its stated schedule, it attains the minimax polynomial rate with T_n of order $n^{d/(2s+d)}/B_n$, up to logarithmic factors.

The untied lower bound gives a contrasting parameter requirement. Attaining the minimax polynomial rate requires its budget to grow at least as $n^{d/(4s+2d)}$, up to logarithmic factors; this is a necessary growth order. For $B_n \asymp n^{\gamma_B}$ with $0 < \gamma_B < d/(4s + 2d)$, Theorem 4.5 shows that the loop-to-untied risk ratio still tends to zero. Looping thus attains comparable statistical accuracy with substantially fewer parameters, at the cost of additional iterations.

We obtain the required approximation rates by encoding local Taylor information in bounded real coefficients and repeatedly applying a shared update to decode and evaluate it. Combining these constructions with likelihood-specific risk bounds yields the comparisons for Gaussian and Laplace regression, binary response, and energy-based generative models.

Relation to existing work. Parameter reuse is present in Universal Transformers (Dehghani et al. 2019), deep equilibrium models (Bai et al. 2019), looped residual language models (Ng and Wang 2024), and recurrent-depth models (Geiping et al. 2025), in contrast to conventional separately parameterized Transformer blocks (Vaswani et al. 2017). Computational analyses study program execution (Giannou et al. 2023), iterative in-context learning (Yang et al. 2024; Gatmiry et al. 2024; Chen et al. 2025), time-invariant reasoning generators (Joshi et al. 2025), and programmable looped rectified linear unit (ReLU) networks (Liang et al. 2025). These studies motivate an explicit comparison of statistical risk under a common parameter constraint.

For smooth-function estimation, approximation and statistical theory typically balance increasing width or depth against the complexity of a growing neural sieve (Yarotsky 2017, 2018; Lu et al. 2021; Schmidt-Hieber 2020). Recurrent networks also attain Hölder approximation and near-minimax least-squares rates (Jiao et al. 2024), while depth-separation results identify representational gains under prescribed resource constraints (Eldan and Shamir 2016; Telgarsky 2016). Work on reuse itself gives quantitative approximation bounds for repeated composition of a fixed-size ReLU network (Zhang et al. 2023), fixed-dimensional recurrence

for univariate continuous functions (Abadie et al. 2026), and looped-Transformer approximation of continuous permutation-equivariant maps (Xu and Sato 2025). Existing complexity bounds quantify the effects of depth (Bartlett et al. 2019) and recurrence length (Chen et al. 2020) alongside parameter count. Model-selection theory accounts for the additional complexity of choosing among candidate models (Barron et al. 1999). The contribution of this article is to establish when the approximation advantage of parameter reuse yields a statistical risk advantage under a common parameter budget. We construct increasingly accurate approximations with a fixed number of bounded coefficients and control estimation over the entire fitted loop class. The resulting worst-case loop risk is asymptotically smaller than the best worst-case risk over the corresponding untied family, even when that family allows joint model selection under the same budget. To our knowledge, our fixed- and growing-budget theorems (Theorems 4.4 and 4.5) provide the first statistical risk separation between looped estimation and the corresponding jointly tuned untied family under a common parameter budget.

Organization. Section 2 formulates looped estimation under a parameter budget and develops its general likelihood-risk comparison with the untied benchmark. Section 3 verifies its ingredients for the four likelihood models and two neural realizations. Section 4 derives the fixed-budget separation and its growing-sieve extension. Section 5 discusses the statistical interpretation and remaining resource questions. Proofs and constructions are in the supplementary material; Supplementary Appendix F presents the numerical studies and experimental details.

2 Looped estimation under a parameter budget

This section defines looped and untied estimators under a common parameter budget and compares their likelihood risks. The budget counts distinct trainable coefficients, while the iteration count records how often the update function is composed. We first define the candidate function classes, then state a general sieve likelihood bound and apply it to compare looped and untied risks under the same budget.

Notation. Let $\mathcal{X}$ be the input space, $\mathcal{V}$ a class of functions on $\mathcal{X}$, and $\mathcal{F} \subseteq \mathcal{V}$ the fixed target family containing the unknown f_0 . An estimator is a measurable, data-dependent choice from a candidate function class. Write $\mathbb{N} = \{1, 2, \dots\}$, and let $\log$ denote the natural logarithm. For real x, y , write $x \vee y = \max\{x, y\}$ and $x \wedge y = \min\{x, y\}$. For functions, $\|\cdot\|_2$ denotes the L^2 norm under the specified measure; for vectors, it denotes the Euclidean norm.

Unless explicitly fixed, $C, c > 0$ denote constants that may change from line to line. They may depend on the fixed target family and the stated likelihood and computational-class constants, and are uniform in the sample size, varying parameter budgets and model tuning parameters, and target function. For positive quantities x, y , write $x \lesssim y$ if $x \leq Cy$ for such a constant, $x \gtrsim y$ if $y \lesssim x$, and $x \asymp y$ if both inequalities hold. Subscripts on these comparison symbols indicate the permitted dependence of the constants.

2.1 Looped and untied representations

A candidate function is constructed in three stages: an input function forms an intermediate representation, successive update functions transform it, and an output function produces a scalar value. A looped representation shares the update parameters, whereas its conventional untied counterpart uses separate parameters at each update (Figure 1).

The model tuning parameter λ specifies the forms and dimensions of the input, update, and output functions and their intermediate representation space $\mathcal{Z}_\lambda$, also called the state space. Neural networks are examples of this general framework. For a fixed feedforward block design, $\lambda = (q, r)$ specifies the dimension q of the vector passed between updates and the hidden-layer width r , with $\mathcal{Z}_\lambda = \mathbb{R}^q$. For example, with $q = 16$ and $r = 64$, each block maps a 16-dimensional input through 64 hidden neurons to a 16-dimensional output.

With these forms and dimensions fixed, let η , θ , and ω denote the coefficient vectors of the input, update, and output functions, with finite-dimensional parameter spaces $\Theta_\eta(\lambda)$, $\Theta_\theta(\lambda)$, and $\Theta_\omega(\lambda)$, respectively. The three functions are $E_\eta : \mathcal{X} \rightarrow \mathcal{Z}_\lambda$, $\Phi_\theta : \mathcal{Z}_\lambda \rightarrow \mathcal{Z}_\lambda$, and $O_\omega : \mathcal{Z}_\lambda \rightarrow \mathbb{R}$. The iteration count T is the number of updates applied to the intermediate representation and is specified separately from λ . Together, λ and T determine the candidate function classes; the coefficient vectors are fitted to data within the chosen class.

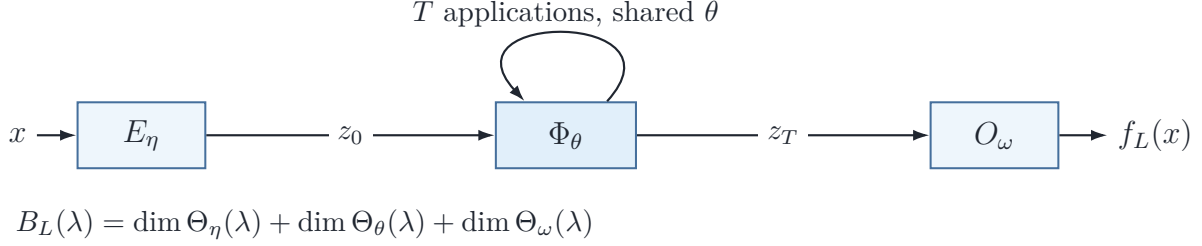
Definition 2.1 (Looped and untied representations). Fix a model tuning parameter λ and an iteration count $T \geq 1$. Both representations compose an input function, T update functions, and an output function. For $x \in \mathcal{X}$, define $f_L(x; \eta, \theta, \omega) = O_\omega(\Phi_\theta^{\circ T}(E_\eta(x)))$, $f_U(x; \eta, \theta_1, \dots, \theta_T, \omega) = O_\omega(\Phi_{\theta_T} \circ \dots \circ \Phi_{\theta_1}(E_\eta(x)))$. Here $\Phi_\theta^{\circ T}$ denotes T -fold composition of the same update function. The looped representation shares θ across these updates; the untied representation uses separate vectors $\theta_1, \dots, \theta_T$. Both use the same input and output function families.

The classes $\mathcal{F}_L(\lambda, T)$ and $\mathcal{F}_U(\lambda, T)$ comprise these functions as their coefficients range over $\Theta_L(\lambda) = \Theta_\eta(\lambda) \times \Theta_\theta(\lambda) \times \Theta_\omega(\lambda)$ and $\Theta_U(\lambda, T) = \Theta_\eta(\lambda) \times \prod_{t=1}^T \Theta_\theta(\lambda) \times \Theta_\omega(\lambda)$, respectively.

Let $\dim \Theta_\eta(\lambda)$, $\dim \Theta_\theta(\lambda)$, and $\dim \Theta_\omega(\lambda)$ denote the numbers of trainable scalar parameters in the input, update, and output functions, respectively. The looped representation fits one shared update vector, whereas the untied representation fits T separate update vectors. Separate parameter slots count individually even when their fitted values coincide. The total parameter counts are therefore $B_L(\lambda) = \dim \Theta_\eta(\lambda) + \dim \Theta_\theta(\lambda) + \dim \Theta_\omega(\lambda)$, $B_U(\lambda, T) = \dim \Theta_\eta(\lambda) + T \dim \Theta_\theta(\lambda) + \dim \Theta_\omega(\lambda)$. A parameter budget B bounds the total number of trainable scalar parameters: an admissible looped or untied representation satisfies $B_L(\lambda) \leq B$ or $B_U(\lambda, T) \leq B$, respectively, and need not exhaust the budget. For fixed λ , $B_L(\lambda)$ is independent of T because further iterations reuse the same update parameters; each new untied update adds $\dim \Theta_\theta(\lambda)$ parameters.

Definition 2.2 (Looped estimation under a parameter budget). For a budget B , *looped estimation under B* chooses model tuning parameters (λ, T) with $B_L(\lambda) \leq B$ and returns a measurable, data-dependent element of $\mathcal{F}_L(\lambda, T)$. A single update parameter vector is fitted and shared across all T applications. Its untied counterpart may select among model tuning parameters (λ, T) satisfying $B_U(\lambda, T) \leq B$. Thus the budget constrains the number of distinct trainable coefficients, while T specifies the number of composed update functions.

(a) Looped representation



(b) Untied representation

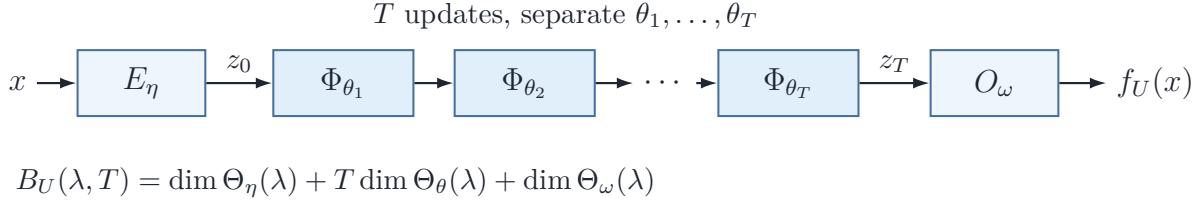


Figure 1: Looped and untied representations at a fixed model tuning parameter λ . Both use the input function E_η and output function O_ω . Here $z_0 = E_\eta(x)$ is the encoded input and z_T is the intermediate representation after T composed updates. In (a), the same update Φ_θ is applied T times; in (b), the updates $\Phi_{\theta_1}, \dots, \Phi_{\theta_T}$ have separate parameter vectors. The counts below each panel include the input, update, and output parameters. Thus $B_L(\lambda)$ is independent of T , whereas $B_U(\lambda, T)$ grows linearly with T .

Let Λ_0 be the set of candidate values of λ . Under a parameter budget B , the admissible values of λ for looped models and the admissible pairs (λ, T) for untied models are $\Lambda_L(B) = \{\lambda \in \Lambda_0 : B_L(\lambda) \leq B\}$, $\mathcal{I}_U(B) = \{(\lambda, T) \in \Lambda_0 \times \mathbb{N} : B_U(\lambda, T) \leq B\}$. Assume that $\mathcal{I}_U(B)$ is finite for each B . Because $B_L(\lambda)$ does not depend on T , membership in $\Lambda_L(B)$ places no restriction on the iteration count. Each additional untied iteration instead adds $\dim \Theta_\theta(\lambda)$ parameters and can carry the pair outside $\mathcal{I}_U(B)$. The budgeted untied family includes every admissible pair of model tuning parameters: $\mathcal{F}_U[B] = \bigcup_{(\lambda, T) \in \mathcal{I}_U(B)} \mathcal{F}_U(\lambda, T)$.

2.2 Likelihood risk and sieve maximum likelihood

Let $\{\mathbb{P}_v : v \in \mathcal{V}\}$ be a likelihood model with densities p_v relative to a common measure ν . We observe independent $Z_1, \dots, Z_n \sim \mathbb{P}_{f_0}$ for an unknown $f_0 \in \mathcal{F}$ and use squared Hellinger distance, $h^2(\mathbb{P}_f, \mathbb{P}_g) = \int (\sqrt{p_f} - \sqrt{p_g})^2 d\nu$, $0 \leq h^2 \leq 2$. We use the same notation for Hellinger distance between laws and between their densities.

For any deterministic candidate sieve $\mathcal{C}_n \subseteq \mathcal{V}$, possibly depending on n , write $\mathcal{P}_n = \{p_g : g \in \mathcal{C}_n\}$ for its induced density class.

Definition 2.3 (Sieve maximum likelihood estimator). Fix a deterministic sieve $\mathcal{C}_n$. For a deterministic optimization tolerance $\varepsilon_n \geq 0$ in the average log likelihood, the sieve maximum likelihood estimator (MLE) is any measurable $\hat{f}_n \in \mathcal{C}_n$ satisfying $\frac{1}{n} \sum_{i=1}^n \log p_{\hat{f}_n}(Z_i) \geq$

$\sup_{g \in \mathcal{C}_n} \frac{1}{n} \sum_{i=1}^n \log p_g(Z_i) - \varepsilon_n$. Existence of a measurable approximate maximizer is assumed.

We study the worst-case squared Hellinger risk of the sieve MLE over $f_0 \in \mathcal{F}$. Its risk bound separates approximation by the sieve from the complexity of fitting over it. For a deterministic sieve $\mathcal{C}_n$, define the worst-case squared Hellinger approximation error by $A(\mathcal{C}_n) = \sup_{f_0 \in \mathcal{F}} \inf_{g \in \mathcal{C}_n} h^2(\mathbb{P}_{f_0}, \mathbb{P}_g)$. A bracket $[p^-, p^+]$ contains the densities lying between its nonnegative lower and upper functions, with width $\|\sqrt{p^+} - \sqrt{p^-}\|_{L^2(\nu)}$. For a density class $\mathcal{P}$, the Hellinger bracketing number $N(\epsilon, \mathcal{P}, h)$ is the smallest number of brackets of width at most ϵ needed to contain the class. Its logarithm, $H(\epsilon, \mathcal{P}, h) = \log N(\epsilon, \mathcal{P}, h)$, is the bracketing entropy. Supplementary Section A.2 gives the precise pointwise conventions.

Likelihood estimation also requires an approximating candidate that is close in the one-sided likelihood quantity

$$\rho_\gamma(p_0, p_1) = \frac{1}{\gamma} \int p_0 \left\{ \left(\frac{p_0}{p_1} \right)^\gamma - 1 \right\} d\nu, \quad 0 < \gamma \leq 1. \quad (1)$$

For the selected sieve $\mathcal{C}_n$, set $\delta(f_0; \mathcal{C}_n) = \inf_{g \in \mathcal{C}_n} \rho_{1/2}(p_{f_0}, p_g)$. The next result combines complete-sieve bracketing with a comparison of likelihood and Hellinger approximation.

Corollary 2.4 (Sieve-MLE oracle inequality). *Let $C_1, C_2, C_3 > 0$ be the absolute constants in the sieve-likelihood bound in Supplementary Theorem A.2. Let $e_n > 0$ be a deterministic critical radius for $\mathcal{P}_n$ such that $n^{-1} \leq e_n^2 \leq 2$ and, for every $e_n \leq t \leq \sqrt{2}$,*

$$\int_{t^2/2^8}^{\sqrt{2}t} \sqrt{H(u/C_1, \mathcal{P}_n, h)} du \leq C_2 \sqrt{n} t^2. \quad (2)$$

Thus e_n depends on the selected sieve through the entropy of $\mathcal{P}_n$. If, for a fixed constant $C_4 > 0$ independent of n and f_0 ,

$$\sup_{f_0 \in \mathcal{F}} \delta(f_0; \mathcal{C}_n) \leq C_4 A(\mathcal{C}_n), \quad (3)$$

then there is a constant C_5 , depending only on C_3, C_4 and independent of n and f_0 , such that

$$\sup_{f_0 \in \mathcal{F}} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}, \mathbb{P}_{\hat{f}_n}) \leq C_5 \left\{ \underbrace{A(\mathcal{C}_n)}_{\text{sieve approximation}} + \underbrace{e_n^2}_{\text{fitted-sieve estimation}} + \underbrace{\varepsilon_n}_{\text{optimization}} \right\}. \quad (4)$$

The result follows from the one-sided truncated likelihood-ratio argument of Wong and Shen (1995); Supplementary Theorem A.2 gives the bound and derivation. Its two conditions play different roles. The bracketing integral controls fitting over the complete sieve, while the likelihood bridge certifies that an approximating member is close to the truth in the one-sided likelihood quantity as well as in Hellinger distance. An accurate estimator therefore needs a small approximation term, a controlled estimation term, and a suitably small optimization tolerance.

2.3 Looped and untied risk comparison

We compare the loop sieve MLE with estimators taking values in the untied union $\mathcal{F}_U[B]$, using the same likelihood model, target family, squared Hellinger risk, and parameter budget. Every function in the looped and untied classes is required to belong to $\mathcal{V}$.

Looped upper bound. For looped estimation at sample size n , choose a deterministic parameter budget $B = B_n$ and model tuning parameters $\lambda = \lambda_n \in \Lambda_L(B_n)$ and $T = T_n \geq 1$ before fitting. These choices may depend on n ; their n subscripts are suppressed below. The selected loop sieve is $\mathcal{F}_L(\lambda, T)$. Taking $\mathcal{C}_n = \mathcal{F}_L(\lambda, T)$ in Definition 2.3 defines the loop sieve MLE $\hat{f}_{L,n}$. Write $A_L(\lambda, T) = A(\mathcal{F}_L(\lambda, T))$ and let $e_{n,L}$ be a critical radius satisfying the hypotheses of Corollary 2.4 for the induced density class. If the likelihood bridge (3) holds for this sieve, then Corollary 2.4 gives

$$\sup_{f_0 \in \mathcal{F}} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}, \mathbb{P}_{\hat{f}_{L,n}}) \leq C_5 \{A_L(\lambda, T) + e_{n,L}^2 + \varepsilon_n\}. \quad (5)$$

A parameter budget limits the coefficients available for approximation. For fixed λ , increasing T leaves $B_L(\lambda)$ unchanged but changes the loop sieve and can affect both approximation and estimation. Thus the bound allows us to ask whether repeated composition improves approximation under a fixed budget while keeping estimation error controlled.

Untied lower bound. For a candidate class $\mathcal{C} \subseteq \mathcal{V}$, define its class-constrained worst-case risk by $\inf_{\hat{f}: \hat{f} \in \mathcal{C}} \sup_{f_0 \in \mathcal{F}} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}, \mathbb{P}_{\hat{f}})$. The infimum ranges over all measurable $\mathcal{C}$ -valued estimators. Taking $\mathcal{C} = \mathcal{F}_U[B]$ gives the untied benchmark. This benchmark allows data-dependent selection of λ and T and any measurable fitting rule valued in $\mathcal{F}_U[B]$.

We derive the lower bound by comparing the Hellinger packing number of the target family with the Hellinger covering number of the budgeted untied family. A packing consists of target laws separated by a specified Hellinger distance; a cover approximates every law in the untied family within a specified distance. If the target packing is too large for such a cover, at least one target remains separated from every member of the untied family. We state this comparison precisely below and combine it with the loop risk upper bound.

Assumption 2.5 (Hellinger packing of the target family). There exist an exponent $a > 0$ and constants $c_{\text{pack}}, \delta_0 > 0$ such that, for every $0 < \delta \leq \delta_0$, the target family contains $f_{1,\delta}, \dots, f_{M_\delta, \delta}$ satisfying $\min_{1 \leq j \neq k \leq M_\delta} h(\mathbb{P}_{f_{j,\delta}}, \mathbb{P}_{f_{k,\delta}}) \geq 8\delta$, $\log M_\delta \geq c_{\text{pack}} \delta^{-2/a}$.

For a class of probability laws $\mathcal{Q}$, let $\mathcal{N}(\epsilon, \mathcal{Q}, h)$ denote the smallest cardinality of an internal Hellinger cover of radius ϵ , with every center belonging to $\mathcal{Q}$. Write $\mathcal{P}_U[B] = \{p_g : g \in \mathcal{F}_U[B]\}$ for the laws induced by the budgeted untied union.

Theorem 2.6 (Untied minimax lower bound under a parameter budget). *Suppose there is a constant $C > 0$ such that, for sufficiently large B and $0 < \epsilon < 1$,*

$$\log \mathcal{N}(\epsilon, \mathcal{P}_U[B], h) \leq C \{B^\kappa \log^b(eB) + B \log(1/\epsilon)\}, \quad \kappa, b \geq 1. \quad (6)$$

Under Assumption 2.5, there is a constant $c_{\text{low}} > 0$, depending only on the fixed packing and

covering constants, such that every $n \geq 1$ and all sufficiently large B satisfy

$$\inf_{\hat{f}: \hat{f} \in \mathcal{F}_U[B]} \sup_{f_0 \in \mathcal{F}} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}, \mathbb{P}_{\hat{f}}) \geq c_{\text{low}} \{B^\kappa \log^b(eB)\}^{-a}. \quad (7)$$

At any fixed sufficiently large B , the right side of (7) is a positive constant valid for every sample size and every measurable fitting rule. It also covers data-dependent selection of (λ, T) within $\mathcal{F}_U[B]$. Theorem 2.6 follows from the packing-cover bound in Lemma A.1 with squared Hellinger risk.

Risk comparison.

Corollary 2.7 (Loop-versus-untied risk separation). *Let $B = B_n$, $\lambda = \lambda_n \in \Lambda_L(B_n)$, and $T = T_n \geq 1$ be deterministic sequences of parameter budgets and model tuning parameters satisfying the hypotheses of the loop risk bound (5) and Theorem 2.6. If $\{A_L(\lambda, T) + e_{n,L}^2 + \varepsilon_n\} \{B^\kappa \log^b(eB)\}^a \rightarrow 0$ as $n \rightarrow \infty$, then the ratio of the worst-case squared Hellinger risk of the loop sieve MLE to the untied benchmark over $\mathcal{F}_U[B]$ tends to zero.*

Corollary 2.7 gives a sufficient condition for the loop sieve MLE to attain a worst-case risk of smaller order than the untied benchmark under a common parameter budget. Section 3 verifies the approximation and complexity bounds for concrete likelihood and neural examples; Section 4 establishes separation under fixed and growing budgets.

3 Likelihood models and neural realizations

The general comparison separates the statistical model from the parameterized representation. A likelihood model determines how function approximation and covering translate into probability-law distances. A neural architecture determines what can be approximated and the complexity of its complete fitted class. This section verifies both sets of conditions for four likelihood models and two neural realizations.

3.1 Likelihood models and metric calibration

We consider three conditional prediction models and one energy-based generative model, each indexed by a bounded Hölder-smooth function. Squared Hellinger distance is comparable to squared $L^2(\mu)$ distance between the conditional model functions, or distance modulo additive constants for the energy model.

Definition 3.1 (Hölder class). Fix $d \geq 1$, $s > 0$ and $H > 0$. Write $k = \lfloor s \rfloor$ when s is not an integer and $k = s - 1$ otherwise, and put $\alpha = s - k \in (0, 1]$. Let $\mathcal{H}_d^s(H)$ consist of functions $f : [0, 1]^d \rightarrow [-H, H]$ that are C^k on $(0, 1)^d$, whose partial derivatives through order k are all bounded by H there, and whose order- k partial derivatives satisfy $|\partial^\nu f(x) - \partial^\nu f(y)| \leq H \|x - y\|_\infty^\alpha$ for $|\nu| = k$ and $x, y \in (0, 1)^d$. Here $\nu = (\nu_1, \dots, \nu_d)$ is a multi-index of nonnegative integers, $|\nu| = \sum_i \nu_i$, and ∂^ν denotes the corresponding mixed partial derivative. Also, $\|x - y\|_\infty = \max_i |x_i - y_i|$. Boundary values are bounded by H ; no boundary derivative condition is imposed.

Throughout this section, $\mathcal{X} = [0, 1]^d$, μ is uniform probability measure on $\mathcal{X}$, $\mathcal{F} = \mathcal{H}_d^s(H)$, and $a = 2s/d$. For these examples, let $\mathcal{V}$ be the measurable functions from $\mathcal{X}$ into $[-H, H]$; this likelihood index space extends beyond the Hölder target class $\mathcal{F}$.

The four likelihood models below use the same candidate space $\mathcal{V}$ and target class $\mathcal{F}$ in the sieve maximum likelihood framework of Section 2. In each model, the unknown function f_0 determines the law of the observations.

Conditional prediction models. Observe independent pairs (X_i, Y_i) with $X_i \sim \mu$ and, for $f_0 \in \mathcal{H}_d^s(H)$, consider

$$Y_i \mid X_i = x \sim N(f_0(x), \sigma^2), \quad \text{Laplace}(f_0(x), \tau), \quad \text{or} \quad \text{Bernoulli}\left((1 + e^{-f_0(x)})^{-1}\right).$$

The Gaussian standard deviation $\sigma > 0$ and Laplace scale $\tau > 0$ are fixed and known. Write $m = \text{G, La, Be}$ for these three models and $\mathbb{P}_f^m, p_f^m$ for the joint law and density indexed by f .

Energy-based generative model. Observe independent $X_1, \dots, X_n \sim \mathbb{P}_{f_0}^E$, where

$$Z_f = \int_{\mathcal{X}} e^{f(u)} d\mu(u), \quad p_f^E(x) = \frac{e^{f(x)}}{Z_f}, \quad f_0 \in \mathcal{H}_d^s(H). \quad (8)$$

Here f is a log-density potential, or negative energy (Song and Kingma 2021). Adding a constant to f leaves the density unchanged, so define $d_c(f, g)^2 = \inf_{c \in \mathbb{R}} \|f - g - c\|_{L^2(\mu)}^2$. Set $d_m(f, g) = \|f - g\|_{L^2(\mu)}$ for $m \in \{\text{G, La, Be}\}$ and $d_E(f, g) = d_c(f, g)$. For a function class $\mathcal{G}$, write $\mathcal{N}(\epsilon, \mathcal{G}, \|\cdot\|_\infty)$ for its internal uniform covering number.

The next proposition connects these models to the general likelihood theory. Its metric and likelihood bounds transfer function approximation to likelihood risk, while its bracket conversion and target packing provide the complexity conditions used in the upper and lower bounds.

Proposition 3.2 (Metric and likelihood bounds for the examples). *For $m \in \{\text{G, La, Be, E}\}$, let $\mathcal{P}^m(\mathcal{G}) = \{p_g^m : g \in \mathcal{G}\}$ for a function class $\mathcal{G}$, and let d_m be the pseudometric defined above. There are constants $0 < c_m < C_m < \infty$, depending only on H and on the fixed σ or τ , such that the following statements hold uniformly over measurable $f, g : \mathcal{X} \rightarrow [-H, H]$.*

- (i) Metric equivalence. $c_m d_m(f, g)^2 \leq h^2(\mathbb{P}_f^m, \mathbb{P}_g^m) \leq C_m d_m(f, g)^2$, $d_m(f, g) \leq \|f - g\|_{L^2(\mu)}$.
- (ii) One-sided likelihood bridge. $\rho_{1/2}(p_f^m, p_g^m) \leq C_m h^2(\mathbb{P}_f^m, \mathbb{P}_g^m)$.
- (iii) Bracket and cover conversion. *For every $\mathcal{G} \subseteq \{g : \mathcal{X} \rightarrow [-H, H]\}$ and $0 < u < 1$, $H(u, \mathcal{P}^m(\mathcal{G}), h) \leq \log \mathcal{N}(u/C_m, \mathcal{G}, \|\cdot\|_\infty)$. An internal uniform u -cover of $\mathcal{G}$ also induces an internal Hellinger $C_m u$ -cover of $\mathcal{P}^m(\mathcal{G})$.*
- (iv) Target packing. *The Hölder target family satisfies Assumption 2.5 in every model with exponent $a = 2s/d$.*

The proof is in Supplementary Section B.2, with the likelihood calculations in Supplementary Appendix C. In particular, $d_c \leq \|\cdot\|_{L^2(\mu)}$ means that the same ordinary $L^2(\mu)$ approximation bound in Section 3.3 suffices for all four models. Section 3.4 applies the bracket and cover

conversion to the neural classes defined next.

3.2 Looped FFN and Transformer

We specialize Definition 2.1 to residual FFNs and post-LN Transformers, indexed by $j = \text{F}$ and $j = \text{Tr}$, respectively. For each architecture, the model tuning parameter is $\lambda = (q, r)$. Here q is the intermediate representation dimension for the FFN and the dimension of each token for the Transformer; r is the hidden width of the update block’s feedforward subnetwork. For both architectures, fix $M_{\text{par}} > 0$ and restrict every trainable scalar to $[-M_{\text{par}}, M_{\text{par}}]$. Write $\Pi_H(u) = \min\{H, \max\{-H, u\}\}$ for the fixed output clipping function, which adds no trainable parameters.

Looped FFN. The input function E_η is an affine function from $\mathbb{R}^d$ to $\mathbb{R}^q$. For a state $z \in \mathbb{R}^q$, the shared residual block is

$$\Phi_\theta(z) = z + W_2 \text{ReLU}(W_1 z + b_1) + b_2, \quad W_1 \in \mathbb{R}^{r \times q}, \quad b_1 \in \mathbb{R}^r, \quad W_2 \in \mathbb{R}^{q \times r}, \quad b_2 \in \mathbb{R}^q.$$

Here $\text{ReLU}(u) = \max\{u, 0\}$ acts coordinatewise. The output function O_ω is an affine scalar function on $\mathbb{R}^q$ followed by Π_H . Applying the same block T times gives the class $\mathcal{F}_{L,\text{F}}(q, r, T)$. All input-function, block, and output-function coefficients range over the prescribed parameter box, with their shapes fixed before the class is formed. Allowing a separate update vector at each application defines $\mathcal{F}_{U,\text{F}}(q, r, T)$.

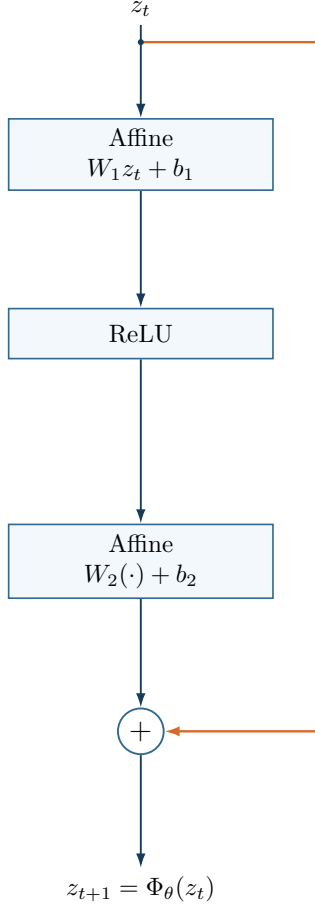
Looped Transformer. Fix a partition $I_1, \dots, I_M$ of $\{1, \dots, d\}$ into nonempty sets and put $N = M + 3$. The state $S \in \mathbb{R}^{N \times q}$ consists of M data tokens, one controller row, and two reference rows. The input function assigns data token $j \leq M$ the row $S_{0,j} = A_j x_{I_j} + a_j$, where $A_j \in \mathbb{R}^{q \times |I_j|}$ and $a_j \in \mathbb{R}^q$. The controller and reference rows are input-independent free constant vectors in $\mathbb{R}^q$.

Writing Attn for self-attention, FFN for the row-wise feedforward subnetwork, and LN_1, LN_2 for layer normalization, the shared post-LN block is $Y = \text{LN}_1\{S + \text{Attn}(S)\}$, $\Phi_\theta(S) = \text{LN}_2\{Y + \text{FFN}(Y)\}$. Attention has one q -dimensional softmax head with $1/\sqrt{q}$ scaling, no mask, and four affine projections from $\mathbb{R}^q$ to $\mathbb{R}^q$. The FFN has one ReLU hidden layer of width r . Each layer normalization has trainable coordinatewise gain and bias and a fixed positive stabilizer ε_{LN} . The output function O_ω is an affine scalar function of $\text{vec}(S) \in \mathbb{R}^{Nq}$ followed by Π_H , where vec stacks the matrix entries into a vector.

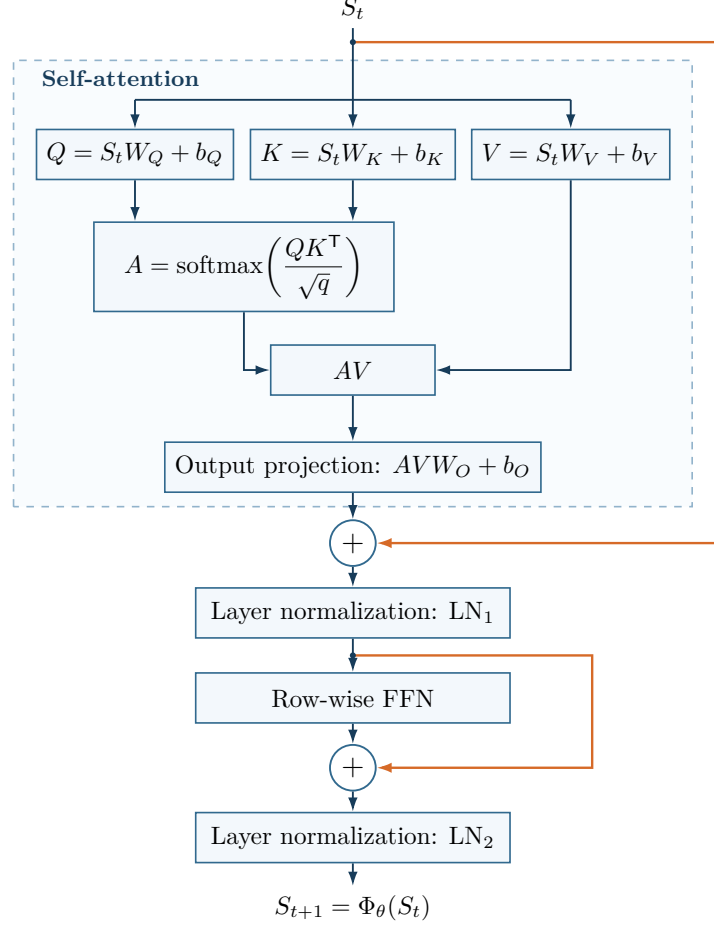
Applying this block T times with shared parameters defines $\mathcal{F}_{L,\text{Tr}}(q, r, T)$; using separate update vectors defines $\mathcal{F}_{U,\text{Tr}}(q, r, T)$. Both classes use the same token protocol, parameter box, and endpoint shapes. Reference values and zero coordinates used by an approximation witness remain free coordinates in the complete parameterized classes. Figure 2 displays the two update blocks within the common structure of Figure 1.

Estimation and parameter counts. For $j \in \{\text{F}, \text{Tr}\}$, Table 1 gives the total numbers of trainable scalar coefficients, including the input and output functions. The formulas count every matrix entry, bias, layer-normalization gain, and layer-normalization shift in the definitions above. Supplementary Section F.4.2 gives the corresponding dense counts for the residual FFNs used in the numerical studies.

(a) Residual FFN block



(b) Post-LN Transformer block



The same update block is applied T times with shared parameters.

Figure 2: Neural realizations of one shared update function. The residual FFN applies an affine–ReLU–affine function with a residual connection. The Transformer applies self-attention and a row-wise FFN, with layer normalization after each residual update. Orange arrows show residual connections. In both architectures, the displayed update is applied T times with the same parameters, which are counted once toward the parameter budget.

Table 1: Distinct trainable parameter counts. The looped count $B_L^j(q, r)$ is independent of T ; the untied count $B_U^j(q, r, T)$ grows linearly with T . The Transformer uses $N = M + 3$ tokens.

Architecture	$B_L^j(q, r)$	$B_U^j(q, r, T)$
Residual ReLU FFN	$q(d+3) + 1 + (2q+1)r$	$q(d+2) + 1 + T\{(2q+1)r + q\}$
Post-LN Transformer	$4q^2 + q(d+2N+9) + 1 + (2q+1)r$	$q(d+2N) + 1 + T\{4q^2 + (2q+1)r + 9q\}$

Under a parameter budget B , define

$$\mathcal{I}_{U,j}(B) = \{(q, r, T) : B_U^j(q, r, T) \leq B\}, \quad \mathcal{F}_{U,j}[B] = \bigcup_{(q,r,T) \in \mathcal{I}_{U,j}(B)} \mathcal{F}_{U,j}(q, r, T).$$

Here j identifies the architecture; the index m identifies the likelihood model from Section 3.1. The corresponding density classes are $\mathcal{P}_{L,j}^m(q, r, T) = \{p_g^m : g \in \mathcal{F}_{L,j}(q, r, T)\}$ and $\mathcal{P}_{U,j}^m[B] = \{p_g^m : g \in \mathcal{F}_{U,j}[B]\}$. For deterministic model tuning parameters (q, r, T) satisfying $B_L^j(q, r) \leq B$, let $\hat{f}_{L,n}^{m,j}$ be the sieve MLE of Definition 2.3 over $\mathcal{F}_{L,j}(q, r, T)$, with average-log-likelihood tolerance ε_n . The untied competitor may be any measurable estimator valued in $\mathcal{F}_{U,j}[B]$, allowing data-dependent selection of (q, r, T) and any fitting rule.

3.3 Approximation error

Approximation error measures how closely a function class can represent the target before data are used to fit its coefficients. For each architecture j and likelihood model m , define the worst-case squared Hellinger approximation error $A_{L,j}^m(q, r, T) = \sup_{f_0 \in \mathcal{H}_d^s(H)} \inf_{g \in \mathcal{F}_{L,j}(q, r, T)} h^2(\mathbb{P}_{f_0}^m, \mathbb{P}_g^m)$. We first bound ordinary $L^2(\mu)$ approximation of the Hölder index and then use Proposition 3.2 to transfer the result to all four likelihood models. The following theorem expresses the bound in terms of hidden width r and iteration count T , with $a = 2s/d$. Besides the leading approximation term, each bound includes a term arising from initialization of the approximation construction. We call the condition under which this term may be omitted the initialization condition.

Theorem 3.3 (Looped neural approximation in Hellinger distance). *Fix d, s, H and $j \in \{\text{F}, \text{Tr}\}$. For $j = \text{Tr}$, also fix the token partition and $\varepsilon_{\text{LN}} > 0$. There exist constants $C, c > 0$ and architecture-specific thresholds q_0^j, r_0^j, M_0^j , depending only on these fixed quantities, such that, for fixed $M_{\text{par}} \geq M_0^j$, integers $q \geq q_0^j, r \geq r_0^j$, and sufficiently large T , every likelihood model m of Section 3.1 satisfies*

$$A_{L,j}^m(q, r, T) \leq C_m \left\{ \left[\frac{\log(erT)}{rT} \right]^a + e^{-cT^{1/h_0^j}} \right\}, \quad (9)$$

where $a = 2s/d$, $h_0^{\text{F}} = 1$, $h_0^{\text{Tr}} = 2$, and $C_m > 0$ depends additionally on the fixed parameters of the likelihood model. The exponential term may be omitted when $T \geq C \log(erT)$ for the residual FFN and when $T \geq C \log^2(erT)$ for the Transformer.

For fixed admissible q, r , $B_L^j(q, r)$ is independent of T . Once the initialization condition above holds, the bound simplifies to $A_{L,j}^m(q, r, T) \lesssim_{q,r} \{\log(eT)/T\}^a$. Thus any budget $B \geq B_L^j(q, r)$ admits all iteration counts, with approximation error tending to zero as $T \rightarrow \infty$.

How looping improves approximation. Theorem 3.3 follows from the explicit FFN and Transformer constructions in Theorems D.1 and E.1, respectively. Their central idea is to approximate the target by local Taylor polynomials on a spatial grid and encode the coefficients in bounded real parameters. Repeated use of the same update function decodes and evaluates the encoded information; Supplementary Section D.1 details this procedure, and Supplementary Appendix E implements it in the Transformer. Above the initialization threshold, the proof of Supplementary Lemma B.1 shows that T loop iterations with hidden width r permit a grid with order $rT/\log(erT)$ cells. The resulting finer local approximation gives the squared-error bound $\{\log(erT)/(rT)\}^a$, with the number of trainable coefficients independent of T .

3.4 Complexity of the looped and untied classes

Approximation requires a suitable candidate for each target; estimation requires control of the complete class over which fitting takes place. Supplementary Lemma B.2 bounds uniform covering numbers of the neural index classes; Proposition 3.2 transfers these bounds to density-space complexity. For the loop sieve, Hellinger bracketing entropy controls estimation error. For the budgeted untied union defined in Section 3.2, a Hellinger covering bound combines with target packing to yield a lower bound that remains valid after data-

dependent architecture selection. For any class $\mathcal{C}$ equipped with a pseudometric d , $\mathcal{N}(\epsilon, \mathcal{C}, d)$ denotes its internal covering number at radius ϵ .

Fix a numerical constant $C_6 \geq 2$ and define

$$G(q, r, T) = \log(T + 1) + (T + 1) \log(C_6(q + 1)(r + 1)). \quad (10)$$

Up to a constant, $G(q, r, T)$ bounds the logarithm of the output sensitivity to coefficient changes after T loop iterations. Greater sensitivity requires a finer coefficient grid, so the covering bound depends on the iteration count even with a fixed number of coefficients.

Proposition 3.4 (Hellinger complexity bounds). *Fix $M_{\text{par}} > 0$, $j \in \{\text{F}, \text{Tr}\}$, and a likelihood model m of Section 3.1. There exists a constant $C > 1$, depending only on the fixed problem and implementation parameters, such that, for positive integers q, r, T and $0 < u < 1$,*

$$H(u, \mathcal{P}_{L,j}^m(q, r, T), h) \leq CB_L^j(q, r) \{\log(C/u) + G(q, r, T)\}. \quad (11)$$

For all sufficiently large B and $0 < \epsilon < 1$,

$$\log \mathcal{N}(\epsilon, \mathcal{P}_{U,j}^m[B], h) \leq C \{B^2 \log(eB) + B \log(C/\epsilon)\}. \quad (12)$$

For fixed admissible q, r , $B_L^j(q, r) = O(1)$ and $G(q, r, T) = O(T)$. Thus the looped-class entropy bound in (11) has the simpler form $H(u, \mathcal{P}_{L,j}^m(q, r, T), h) \lesssim_{q,r} T + \log(C/u)$ for $0 < u < 1$.

The proof is given in Supplementary Section B.2.

Repeated computation therefore affects both sides of the loop's statistical balance. At fixed admissible widths, its leading approximation bound decreases as $\{\log(eT)/T\}^a$, whereas the entropy bound controlling estimation grows linearly in T , up to the resolution term $\log(C/u)$. The next section selects model tuning parameters to balance these effects.

4 Risk under fixed and growing parameter budgets

We apply the approximation and complexity bounds of Section 3 to compare looped and untied estimation under a common parameter budget. With a sufficiently large fixed budget, a suitable choice of iteration count allows the loop sieve MLE to attain the Hölder minimax rate up to logarithmic factors, while the optimal worst-case risk over the untied family remains bounded away from zero. We also identify growing-budget regimes in which the loop-to-untied risk ratio tends to zero as the neural dimensions and iteration count increase together. The analysis begins with a loop risk upper bound and lower bounds for unrestricted estimation and the budgeted untied family, followed by the fixed- and growing-budget comparisons.

4.1 Risk bounds: estimation and representation

For a loop sieve with model tuning parameters (q, r, T) , let $\hat{f}_{L,n}^{m,j}$ denote its sieve MLE. Define its worst-case squared Hellinger risk and the best achievable worst-case squared

Hellinger risk over the budgeted untied family, respectively, by

$$\begin{aligned} R_{L,j,n}^m &= \sup_{f_0 \in \mathcal{H}_d^s(H)} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}^m, \mathbb{P}_{\hat{f}_{L,n}^m}^m), \\ R_{U,j,n}^m(B) &= \inf_{\hat{f}: \hat{f} \in \mathcal{F}_{U,j}[B]} \sup_{f_0 \in \mathcal{H}_d^s(H)} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}^m, \mathbb{P}_{\hat{f}}^m). \end{aligned} \quad (13)$$

All measurable fitting and model-tuning rules valued in the budgeted union are allowed.

Theorem 4.1 (Sieve-MLE Hellinger risk under a parameter budget). *Fix a likelihood model $m \in \{\text{G, La, Be, E}\}$ and an architecture $j \in \{\text{F, Tr}\}$. Let $n \geq 2$ and $a = 2s/d$.*

General width-dependent bound. *For any admissible q, r, M_{par}, T from Theorem 3.3, let $\hat{f}_{L,n}^{m,j}$ be a measurable sieve MLE over $\mathcal{F}_{L,j}(q, r, T)$ with average-log-likelihood tolerance ε_n . There exist constants $C, c > 0$, depending only on the fixed model and implementation parameters, such that, with $e_{n,j}^2 = 1 \wedge [CB_L^j(q, r)\{\log(en) + G(q, r, T)\}/n]$,*

$$R_{L,j,n}^m \leq C \left\{ \underbrace{\left[\frac{\log(erT)}{rT} \right]^a}_{\text{approximation}} + e^{-cT^{1/h_0^j}} + \underbrace{e_{n,j}^2 + \varepsilon_n}_{\text{fitting and optimization}} \right\}. \quad (14)$$

Budget-iteration bound. *In particular, fix admissible q, M_{par} from Theorem 3.3 and, for sufficiently large B , choose the largest affordable hidden width $r_B = \max\{r \in \mathbb{N} : B_L^j(q, r) \leq B\}$. Let $\hat{f}_{L,n}^{m,j}$ be a measurable sieve MLE over $\mathcal{F}_{L,j}(q, r_B, T)$ with average-log-likelihood tolerance $\varepsilon_n \leq n^{-1}$. If T is sufficiently large for Theorem 3.3 to apply at (q, r_B, M_{par}) and satisfies its initialization condition, then*

$$R_{L,j,n}^m \leq C' \left\{ \underbrace{\left[\frac{\log(eBT)}{BT} \right]^a}_{\text{approximation}} + \underbrace{\frac{B\{\log(en) + T \log(eB)\}}{n}}_{\text{fitting and optimization}} \right\}. \quad (15)$$

Here $C' > 0$ may depend on the fixed q and remaining problem and architecture parameters, but is independent of n, B, T , and f_0 .

For fixed q , Table 1 gives $r_B \asymp B$, which converts the general bound into (15). The latter holds for both fixed and sample-size-dependent budgets, provided its conditions are satisfied. Suppressing logarithmic factors, it balances approximation of order $(BT)^{-a}$ against estimation of order BT/n ; the balance is attained when BT has order $n^{1/(a+1)}$. At a fixed budget, this balance is achieved by increasing T without adding parameters. A larger shared block can achieve a prescribed approximation accuracy with fewer iterations. At fixed T , increasing B also increases the estimation bound. Repeated composition therefore balances approximation and estimation within a fixed parameter dimension, whereas additional untied depth requires additional coefficients. Theorems 4.4 and 4.5 specialize Theorem 4.1 to fixed and growing budgets, respectively, and combine its upper bound with the untied lower bound to establish risk separation. The growing-width analysis uses the general bound (14).

The proof combines the approximation in Theorem 3.3, the full-class entropy in Proposition 3.4, and the likelihood bridge in Proposition 3.2 through Corollary 2.4. To judge whether the resulting rate can be improved, we first give a minimax lower bound over all measurable distribution estimators. This benchmark quantifies the sampling limitation common to the four likelihood models.

Theorem 4.2 (Unrestricted minimax benchmark for the likelihood examples). *For every $m \in \{G, La, Be, E\}$, there is a constant $c_{\min} > 0$, depending only on the fixed likelihood-model and Hölder-class parameters, such that, for all sufficiently large n ,*

$$\inf_{\hat{P}} \sup_{f_0 \in \mathcal{H}_d^s(H)} \mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}^m, \hat{P}) \geq c_{\min} n^{-a/(a+1)}, \quad (16)$$

where the infimum is over all measurable distribution estimators.

Theorem 4.2 applies independently of the parameterized representation. Restricting the estimator to $\mathcal{F}_{U,j}[B]$ imposes an additional limitation: the next proposition gives a lower bound determined by the parameter budget that holds uniformly in sample size.

Proposition 4.3 (Untied lower bound for the likelihood examples). *Fix $m \in \{G, La, Be, E\}$ and $j \in \{F, Tr\}$. With $R_{U,j,n}^m(B)$ defined in (13), there is a constant $c_U > 0$, depending only on the fixed model, architecture, and Hölder-class parameters, such that, uniformly over every $n \geq 1$ and all sufficiently large B ,*

$$R_{U,j,n}^m(B) \geq c_U \{B^2 \log(eB)\}^{-a}. \quad (17)$$

Taken together, Theorem 4.2 and Proposition 4.3 give, for sufficiently large n and B , $R_{U,j,n}^m(B) \gtrsim n^{-a/(a+1)} \vee \{B^2 \log(eB)\}^{-a}$, representing sampling and representation limitations, respectively. The first term is a property of the statistical experiment. The second comes from restricting the estimator's output to the budgeted untied family and persists even with arbitrarily many observations. The proof of Proposition 4.3 combines the budgeted cover (12), the target packing in Proposition 3.2, and Theorem 2.6. At a fixed budget, the representation term is positive for every sample size. The next result constructs a loop estimator whose worst-case risk tends to zero under the same budget.

4.2 Main result: separation at a fixed parameter budget

We now fix the neural dimensions and choose T_n to balance the approximation and estimation terms in Theorem 4.1. Combining the resulting loop upper bound with Proposition 4.3 gives a risk separation under a common fixed parameter budget. Theorem 4.2 provides the benchmark for assessing the rate of the looped estimator.

Theorem 4.4 (Fixed-budget risk separation). *Fix a likelihood model $m \in \{G, La, Be, E\}$, one architecture $j \in \{F, Tr\}$, and admissible constants $q \geq q_0^j$, $r \geq r_0^j$, $M_{\text{par}} \geq M_0^j$. Let $\hat{f}_{L,n}^{m,j}$ be the sieve MLE over the corresponding loop class and take $\varepsilon_n \leq n^{-1}$. Choose*

$$T_n = \left\lceil n^{1/(a+1)} \{\log(en)\}^{a/(a+1)} \right\rceil. \quad (18)$$

Then the parameter count $B_L^j(q, r)$ is constant in n , and there is a constant $C_7 > 0$, possibly

depending on the fixed admissible q, r, M_{par} but independent of n and the target, such that

$$R_{L,j,n}^m \leq C_7 n^{-a/(a+1)} \{\log(en)\}^{a/(a+1)}. \quad (19)$$

Let $B \geq B_L^j(q, r)$ be any sufficiently large fixed parameter budget. Every estimator valued in the corresponding untied family obeys the positive lower bound $R_{U,j,n}^m(B) \geq c_U \{B^2 \log(eB)\}^{-a}$ supplied by Proposition 4.3, uniformly in n . Combining this with (19) gives

$$\frac{R_{L,j,n}^m}{R_{U,j,n}^m(B)} \longrightarrow 0. \quad (20)$$

The iteration schedule T_n in (18) balances the estimation term T_n/n and the approximation term $(\log T_n/T_n)^a$ at fixed (q, r) . Increasing T_n changes the loop sieve while leaving its parameter count fixed, yielding the minimax polynomial rate of Theorem 4.2 up to a logarithmic factor. Under the same fixed budget, the untied risk remains bounded away from zero, even with data-dependent tuning of width and depth. Thus, vanishing worst-case risk requires a growing untied budget, whereas the loop needs only more iterations.

4.3 Extension: expanding sieves under a growing budget

When $B = B_n$ grows, the untied lower bound in Proposition 4.3 also decreases, so separation requires a comparison of the two rates. The fixed-dimensional loop of Theorem 4.4 remains admissible and already gives separation in the budget range considered below. We now allow both widths to grow and choose their schedules, together with T_n , to balance the general risk bound (14) in Theorem 4.1. The resulting estimator retains the minimax polynomial rate while fitting over an expanding loop class, and its risk remains of smaller order than the optimal risk over the growing untied family.

Theorem 4.5 (Growing budget: jointly tuned rate and risk separation). *Fix a likelihood model $m \in \{G, \text{La}, \text{Be}, \text{E}\}$ and an architecture $j \in \{\text{F}, \text{Tr}\}$ of Section 3.2, with a fixed token partition when $j = \text{Tr}$. Fix $M_{\text{par}} \geq M_0^j$. Set $B = B_n \asymp n^{\gamma_B}$ with*

$$0 < \gamma_B < \frac{1}{2(a+1)} = \frac{d}{4s + 2d}. \quad (21)$$

Take $q_n \asymp \log(en)$ and $r_n = \max\{r \in \mathbb{N} : B_L^j(q_n, r) \leq B_n\}$. Define $\ell_n = \log(C_6(q_n + 1)(r_n + 1))$, $W_n = \{\frac{n\{\log(en)\}^a}{q_n \ell_n}\}^{1/(a+1)}$, $T_n = \lceil W_n/r_n \rceil$. The resulting loop sieve has at most B_n parameters and $q_n, r_n, T_n \rightarrow \infty$. Its sieve MLE with $\varepsilon_n \leq n^{-1}$ satisfies, for all sufficiently large n ,

$$R_{L,j,n}^m \leq C_8 n^{-a/(a+1)} \{\log(en)\}^{3a/(a+1)}, \quad (22)$$

and

$$\frac{R_{L,j,n}^m}{R_{U,j,n}^m(B_n)} \longrightarrow 0. \quad (23)$$

The constant $C_8 > 0$ depends only on the fixed model, architecture, Hölder-class parameters, and constants in the schedules.

Under these schedules, $B_n T_n$ has order $n^{1/(a+1)}$ up to logarithmic factors. The approximation and estimation terms in (14) are both bounded by a constant multiple of the right side of (22). The polynomial exponent matches (19); the larger logarithmic factor reflects the growth of q_n and ℓ_n in the bound. In the budget range (21), the loop upper bound is of smaller order than the untied lower bound in Proposition 4.3, yielding (23). Here the fitted loop sieve’s parameter count grows with n .

5 Discussion

The main statistical finding is that repeated composition of a shared function can improve approximation enough to yield vanishing risk without increasing parameter count. Theorem 4.4 establishes this conclusion for Hölder targets in four likelihood models. The loop approximation guarantee improves as T_n grows, while its complexity bound also increases. Balancing these effects yields the minimax rate up to a logarithmic factor. Under the same fixed cap, the untied family retains positive worst-case representation error.

Fixed and expanding sieves. The fixed-dimensional loop already gives the central separation. Theorem 4.5 shows that the conclusion also holds when state width, hidden width, and iteration count grow together and likelihood fitting ranges over the enlarged coefficient class. At polynomial order, hidden width satisfies $r \asymp B/q$; the approximation bound has order $(BT)^{-2s/d}$ up to logarithmic factors when q is fixed or logarithmic. The fitted-class entropy determines the statistical cost of this expansion. Sharper entropy and approximation bounds may reduce the logarithmic factors.

Numerical findings. Supplementary Appendix F compares single-block, looped, and untied residual FFNs under common parameter budgets in regression simulations, energy-based density estimation, and real-data prediction. Looping improves performance in several settings, with gains varying across targets, budgets, and tasks. The sample-size experiments keep the iteration count fixed; the iteration experiments combine additional iterations with continued training.

Parameters, computation, and precision. The budget counts distinct trainable scalar coefficients. Reusing the same update function adds computation without adding coefficients. In the separation regime, $T_n/B_n \rightarrow \infty$, whereas untied depth is at most $O(B_n)$; a dense residual FFN loop uses order $B_n T_n$ parameter-associated operations per forward pass, compared with order B_n for an untied stack. Coefficient magnitudes remain bounded, while the approximation constructions encode increasingly precise Taylor-table codes and the FFN construction uses intermediate states whose magnitudes grow. These requirements motivate jointly constraining parameter count, numerical precision, optimization, and computation to characterize their tradeoff with statistical risk.

Optimization. The statistical guarantees apply to an approximate maximizer of the empirical likelihood, with tolerance ε_n . Reaching this tolerance is a separate computational problem. Sharing couples coefficient changes across all repeated updates, whereas untied

updates can adjust their coefficients separately. Comparing the computational effort required to reach this tolerance remains open.

Scope and future directions. The concrete comparisons cover the dense residual ReLU and post-LN Transformer families specified in Section 3.2, with bounded coefficients, affine endpoints, and common block dimensions within each stack. The Transformer has one softmax head, the specified position-dependent input functions and special tokens, and a fixed positive normalization stabilizer. Different normalization or masking conventions, variable-width stacks, growing token counts, and compressed parameter descriptions require their own approximation and complexity analysis. For another likelihood or evaluation risk, the general framework requires the corresponding metric calibration and estimation bound. The energy-based example accommodates normalization and an index defined modulo constants.

The schedules use known smoothness s ; adaptation to unknown smoothness remains open. The sharp untied budget threshold is another open question. For polynomial budgets, the proved lower bound gives the necessary exponent condition $\gamma_B \geq d/(4s + 2d)$ for attaining the classical rate. Copying the shared block into separate untied blocks gives a larger sufficient budget. Sharper untied approximation and lower bounds are needed to close this gap.

Supplementary material

Supplementary Appendices A–F, included below, contain general risk tools, proofs of the main results, auxiliary likelihood and complexity bounds, the residual FFN and Transformer approximation constructions, and numerical studies with experimental details.

Funding

This work was supported in part by NSF Grant DMS-1952539 and NIH Grants R01AG069895, R01AG065636, R01AG074858, and U01AG073079.

References

- Abadie, V., Hutter, C. and Bölskei, H. (2026), ‘Recurrent neural networks approximate continuous functions’, arXiv preprint arXiv:2606.20325.
- Bai, S., Kolter, J. Z. and Koltun, V. (2019), Deep equilibrium models, *in* ‘Advances in Neural Information Processing Systems’, Vol. 32.
- Barron, A. R., Birgé, L. and Massart, P. (1999), ‘Risk bounds for model selection via penalization’, *Probability Theory and Related Fields* **113**(3), 301–413.
- Bartlett, P. L., Harvey, N., Liaw, C. and Mehrabian, A. (2019), ‘Nearly-tight VC-dimension and pseudodimension bounds for piecewise linear neural networks’, *Journal of Machine Learning Research* **20**(63), 1–17.

- Chen, B., Li, X., Liang, Y., Shi, Z. and Song, Z. (2025), Bypassing the exponential dependency: Looped transformers efficiently learn in-context by multi-step gradient descent, *in* ‘Proceedings of the 28th International Conference on Artificial Intelligence and Statistics’, Vol. 258 of *Proceedings of Machine Learning Research*, pp. 4447–4455.
- Chen, M., Li, X. and Zhao, T. (2020), On generalization bounds of a family of recurrent neural networks, *in* ‘Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics’, Vol. 108 of *Proceedings of Machine Learning Research*, pp. 1233–1243.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J. and Kaiser, L. (2019), Universal transformers, *in* ‘International Conference on Learning Representations’.
- Efrati, A., Palazzolo, S. and Drew, R. (2026), ‘OpenAI technique in “Astra” model sparks security concerns’, *The Information*.
- Eldan, R. and Shamir, O. (2016), The power of depth for feedforward neural networks, *in* ‘Proceedings of the 29th Annual Conference on Learning Theory’, Vol. 49 of *Proceedings of Machine Learning Research*, pp. 907–940.
- Gatmiry, K., Saunshi, N., Reddi, S. J., Jegelka, S. and Kumar, S. (2024), Can looped transformers learn to implement multi-step gradient descent for in-context learning?, *in* ‘Proceedings of the 41st International Conference on Machine Learning’, Vol. 235 of *Proceedings of Machine Learning Research*, pp. 15130–15152.
- Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A. and Goldstein, T. (2025), Scaling up test-time compute with latent reasoning: A recurrent depth approach, *in* ‘Advances in Neural Information Processing Systems’, Vol. 38.
- Giannou, A., Rajput, S., Sohn, J.-Y., Lee, K., Lee, J. D. and Papailiopoulos, D. (2023), Looped transformers as programmable computers, *in* ‘Proceedings of the 40th International Conference on Machine Learning’, Vol. 202 of *Proceedings of Machine Learning Research*, pp. 11398–11442.
- Jiao, Y., Wang, Y. and Yan, B. (2024), ‘Approximation bounds for recurrent neural networks with application to regression’, arXiv preprint arXiv:2409.05577. Revised 2025.
- Joshi, N., Vardi, G., Block, A., Goel, S., Li, Z., Misiakiewicz, T. and Srebro, N. (2025), A theory of learning with autoregressive chain of thought, *in* ‘Proceedings of Thirty Eighth Conference on Learning Theory’, Vol. 291 of *Proceedings of Machine Learning Research*, pp. 3161–3212.
- Liang, Y., Sha, Z., Shi, Z., Song, Z. and Zhou, Y. (2025), Looped ReLU MLPs may be all you need as practical programmable computers, *in* ‘Proceedings of the 28th International Conference on Artificial Intelligence and Statistics’, Vol. 258 of *Proceedings of Machine Learning Research*, pp. 2647–2655.
- Lu, J., Shen, Z., Yang, H. and Zhang, S. (2021), ‘Deep network approximation for smooth functions’, *SIAM Journal on Mathematical Analysis* **53**(5), 5465–5506.

- Ng, K.-S. and Wang, Q. (2024), ‘Loop neural networks for parameter sharing’, arXiv preprint arXiv:2409.14199.
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S. and Dean, J. (2023), ‘Efficiently scaling Transformer inference’, *Proceedings of Machine Learning and Systems* **5**, 606–624.
- Rajbhandari, S., Rasley, J., Ruwase, O. and He, Y. (2020), ZeRO: Memory optimizations toward training trillion parameter models, *in* ‘Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis’, pp. 1–16.
- Raschka, S. (2026), ‘GPT-6 Astra, looped transformers, and hidden reasoning’, *Ahead of AI*.
- Schmidt-Hieber, J. (2020), ‘Nonparametric regression using deep neural networks with ReLU activation function’, *Annals of Statistics* **48**(4), 1875–1897.
- Shen, X. and Wong, W. H. (1994), ‘Convergence rate of sieve estimates’, *Annals of Statistics* **22**(2), 580–615.
- Song, Y. and Kingma, D. P. (2021), ‘How to train your energy-based models’, arXiv preprint arXiv:2101.03288.
- Stone, C. J. (1982), ‘Optimal global rates of convergence for nonparametric regression’, *Annals of Statistics* **10**(4), 1040–1053.
- Telgarsky, M. (2016), Benefits of depth in neural networks, *in* ‘Proceedings of the 29th Annual Conference on Learning Theory’, Vol. 49 of *Proceedings of Machine Learning Research*, pp. 1517–1539.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017), Attention is all you need, *in* ‘Advances in Neural Information Processing Systems’, Vol. 30.
- Wang, S., Zhang, G., Luo, K., Wu, Y., Liu, S., Liu, J., Huang, W., Yan, S. and Li, J. (2026), ‘SMELT: Scaling laws for compute-matched MoE looped Transformers’, arXiv preprint arXiv:2609.01343.
- Wong, W. H. and Shen, X. (1995), ‘Probability inequalities for likelihood ratios and convergence rates of sieve MLEs’, *Annals of Statistics* **23**(2), 339–362.
- Xu, K. and Sato, I. (2025), On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding, *in* ‘Proceedings of the 42nd International Conference on Machine Learning’, Vol. 267 of *Proceedings of Machine Learning Research*, pp. 69613–69646.
- Yang, L., Lee, K., Nowak, R. D. and Papailiopoulos, D. (2024), Looped transformers are better at learning learning algorithms, *in* ‘International Conference on Learning Representations’.

- Yarotsky, D. (2017), ‘Error bounds for approximations with deep ReLU networks’, *Neural Networks* **94**, 103–114.
- Yarotsky, D. (2018), Optimal approximation of continuous functions by very deep ReLU networks, in ‘Proceedings of the 31st Conference On Learning Theory’, Vol. 75 of *Proceedings of Machine Learning Research*, PMLR, pp. 639–649.
- Zhang, S., Lu, J. and Zhao, H. (2023), On enhancing expressive power via compositions of single fixed-size ReLU network, in ‘Proceedings of the 40th International Conference on Machine Learning’, Vol. 202 of *Proceedings of Machine Learning Research*, PMLR, pp. 41452–41487.
- Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., Li, L., Shi, J., Ma, K., Li, S., Kergan, T., Smith, A., Qu, X., Hui, M., Wu, B., Min, Q., Huang, H., Zhou, X., Ye, W., Liu, J., Yang, J., Shi, Y., Lin, C., Zhao, E., Cai, T., Zhang, G., Huang, W., Bengio, Y. and Eshraghian, J. (2025), ‘Scaling latent reasoning via looped language models’, arXiv preprint arXiv:2510.25741.

Supplementary Appendices

This supplement develops the ingredients used to prove the main results. Appendix A gives the general sieve-likelihood upper bound and packing-cover lower bound. Appendix C establishes the likelihood comparisons, target packing, and parameter sensitivity bounds needed to apply these tools. Appendices D and E construct the residual FFN and Transformer approximations at a fixed grid resolution. Appendix B then chooses the resolution from the iteration budget, transfers index approximation and covering bounds to Hellinger distance, and derives the main risk bounds and their fixed- and growing-budget consequences. Appendix F presents the numerical results; Section F.4 collects their experimental details.

A General risk and likelihood tools

We use the statistical experiment and notation of Section 2. The following tools provide a class-constrained lower bound for calibrated risks and a likelihood upper bound for a fitted sieve.

A.1 A packing-cover lower-bound lemma

For a pseudometric d on the target and candidate indices, let $\mathcal{N}(\delta, \mathcal{C}, d)$ be the internal covering number, as in Section 2.3, and let $\mathcal{M}(\delta, \mathcal{F}, d)$ be the supremum of the cardinalities of finite subsets of $\mathcal{F}$ with pairwise distances at least δ .

Lemma A.1 (Packing-cover lower bound). *Let $\mathcal{C} \subseteq \mathcal{V}$ be a nonempty candidate class and $R(f, g) \geq 0$ an evaluation risk whose value at each estimator under consideration is measurable. Suppose that, for constants $c_d, p > 0$,*

$$R(f, g) \geq c_d d(f, g)^p, \quad f \in \mathcal{F}, \quad g \in \mathcal{C}.$$

If, for some $\delta > 0$,

$$\mathcal{M}(8\delta, \mathcal{F}, d) > \mathcal{N}(\delta, \mathcal{C}, d), \quad \mathcal{N}(\delta, \mathcal{C}, d) < \infty,$$

then there is an $f_ \in \mathcal{F}$ such that $d(f_*, g) \geq \delta$ for every $g \in \mathcal{C}$. Consequently, for every sample size $n \geq 1$,*

$$\inf_{\hat{g}: \hat{g} \in \mathcal{C}} \sup_{f \in \mathcal{F}} \mathbb{E}_f R(f, \hat{g}) \geq c_d \delta^p, \tag{S1}$$

where the infimum ranges over all measurable $\mathcal{C}$ -valued estimators.

Proof. Choose an 8δ -packing $f_1, \dots, f_M$ and an internal δ -cover $g_1, \dots, g_N$ of $\mathcal{C}$ with $M > N$. If each f_j were within distance less than δ of some member of $\mathcal{C}$, it would be within distance less than 2δ of a cover center. Two targets assigned to the same center would have distance less than 4δ , contrary to their separation. The assignment would therefore be injective, contradicting $M > N$. Hence some f_* is at distance at least δ from every candidate. For this target, $R(f_*, \hat{g}) \geq c_d \delta^p$ for every sample. Taking expectation, the supremum over targets, and the infimum over estimators proves the bound. $\square$

The lemma permits arbitrary data-dependent fitting and selection within $\mathcal{C}$. Theorem 2.6 uses squared Hellinger risk and $d = h$.

A.2 A sieve-likelihood risk bound

Use the density model and Hellinger distance of Section 2, and the one-sided likelihood quantity ρ_γ in (1). Fix measurable density versions on one common set $\mathcal{Z}_\nu$ whose complement has ν -measure zero. All pointwise comparisons below use this set. Density ratios use $0/0 = 0$ and $a/0 = +\infty$ for $a > 0$, and $\log 0 = -\infty$.

A bracket $[p^-, p^+]$ contains all densities p satisfying $p^- \leq p \leq p^+$ on $\mathcal{Z}_\nu$, where the endpoints are nonnegative measurable functions with $\int p^+ d\nu < \infty$. Its width is $\|\sqrt{p^+} - \sqrt{p^-}\|_{L^2(\nu)}$; the endpoints need not be densities. As in Section 2.2, $N(u, \mathcal{P}, h)$ counts brackets of width at most u and $H(u, \mathcal{P}, h) = \log N(u, \mathcal{P}, h)$. In particular, a pointwise envelope $|\sqrt{p} - \sqrt{p_j}| \leq uL$, with nonnegative $L \in L^2(\nu)$, gives the bracket

$$\left[\{(\sqrt{p_j} - uL)_+\}^2, (\sqrt{p_j} + uL)^2 \right]$$

of width at most $2u\|L\|_2$, where $t_+ = \max\{t, 0\}$. Choosing a member of each nonempty bracket gives $\mathcal{N}(u, \mathcal{P}, h) \leq N(u, \mathcal{P}, h)$; an ordinary metric cover alone does not supply the pointwise envelopes. Appendix C constructs the required brackets for the likelihood examples.

For a deterministic sieve $\mathcal{S}_n \subseteq \mathcal{V}$, write $\mathcal{P}_n = \{p_g : g \in \mathcal{S}_n\}$ and let $\hat{f}_n$ be a measurable approximate maximizer with average-log-likelihood tolerance ε_n , as in Definition 2.3 with $\mathcal{C}_n$ replaced by $\mathcal{S}_n$. The next result combines Theorem 1 of Wong and Shen (1995) with a comparison to one approximating sieve density.

Theorem A.2 (Sieve-MLE Hellinger bound). *For the sieve MLE and density class specified above, set*

$$\delta_n(f_0) = \inf_{q \in \mathcal{P}_n} \rho_{1/2}(p_{f_0}, q).$$

Let $C_1, C_2 > 0$ be the absolute constants in Theorem 1 of Wong and Shen (1995); they are independent of the sieve, sample size, and target. Suppose that $e_n > 0$ is deterministic, $n^{-1} \leq e_n^2 \leq 2$, and, for every $e_n \leq t \leq \sqrt{2}$,

$$\int_{t^2/2^8}^{\sqrt{2}t} \sqrt{H(u/C_1, \mathcal{P}_n, h)} du \leq C_2 \sqrt{n} t^2. \quad (\text{S2})$$

The resolution e_n may depend on the selected sieve $\mathcal{P}_n$. Then an absolute constant C_3 satisfies

$$\mathbb{E}_{f_0} h^2(\mathbb{P}_{f_0}, \mathbb{P}_{\hat{f}_n}) \leq C_3 \{\delta_n(f_0) + e_n^2 + \varepsilon_n\}. \quad (\text{S3})$$

The bound holds uniformly over any target set on which the right-hand side is uniformly bounded.

Proof. Fix f_0 and put $p_0 = p_{f_0}$ and $L_n(q) = \prod_{i=1}^n q(Z_i)/p_0(Z_i)$. Theorem 1 of Wong and

Shen (1995), under (S2), gives absolute constants $c_1, c_2 > 0$ such that

$$\mathbb{P}_{f_0}^* \left\{ \sup_{q \in \mathcal{P}_n: h(p_0, q) \geq t} L_n(q) \geq e^{-c_1 n t^2} \right\} \leq 4e^{-c_2 n t^2}, \quad e_n \leq t \leq \sqrt{2}. \quad (\text{S4})$$

Outer probability permits a supremum over a general density class; the selected estimator is measurable by hypothesis.

If $\delta_n(f_0) = \infty$, the conclusion is immediate. Otherwise, fix $\eta > 0$ and choose a deterministic $q_* \in \mathcal{P}_n$ such that $\rho_{1/2}(p_0, q_*) \leq \delta_n(f_0) + \eta$. Independence and the definition of $\rho_{1/2}$ imply

$$\mathbb{E}_{f_0} L_n(q_*)^{-1/2} = \left\{ 1 + \frac{1}{2} \rho_{1/2}(p_0, q_*) \right\}^n \leq \exp\{n(\delta_n(f_0) + \eta)/2\}.$$

Markov's inequality therefore gives

$$\mathbb{P}_{f_0} \{L_n(q_*) \leq e^{-c_1 n t^2 + n \varepsilon_n}\} \leq \exp\left\{-\frac{n}{2}[c_1 t^2 - \varepsilon_n - \delta_n(f_0) - \eta]\right\}. \quad (\text{S5})$$

The approximate maximum-likelihood property gives $L_n(p_{\hat{f}_n}) \geq e^{-n \varepsilon_n} L_n(q_*)$. If $h(p_0, p_{\hat{f}_n}) > t$, either the event in (S4) or the event in (S5) must occur. For a sufficiently large absolute K , set

$$t^2 = K\{\delta_n(f_0) + \eta + e_n^2 + \varepsilon_n + x/n\}, \quad x \geq 0.$$

When $t^2 < 2$, the two probability bounds give a tail at most Ce^{-cx} ; when $t^2 \geq 2$, the event $\{h^2(p_0, p_{\hat{f}_n}) > t^2\}$ is empty. Integrating this bound and using $n^{-1} \leq e_n^2$ proves (S3) with an additional $C\eta$. Letting $\eta \downarrow 0$ completes the proof. $\square$

B Proofs of the main results

This appendix gives the proofs of the results in Sections 2–4, in their order of appearance. Appendix A supplies the general risk tools, Appendix C collects auxiliary likelihood and complexity bounds, and Appendices D and E give the neural constructions.

B.1 Proofs for Section 2

Proof of Corollary 2.4. Apply Theorem A.2 with $\mathcal{S}_n = \mathcal{C}_n$. Then $\delta_n(f_0) = \delta(f_0; \mathcal{C}_n)$, and the bridge condition (3) gives $\sup_{f_0 \in \mathcal{F}} \delta_n(f_0) \leq C_4 A(\mathcal{C}_n)$. Taking the supremum in (S3) proves (4) with $C_5 = C_3 \max\{1, C_4\}$. $\square$

Proof of Theorem 2.6. Set $W_B = B^\kappa \log^b(eB)$ and $\delta_B = c_0 W_B^{-a/2}$, where $0 < c_0 < 1$ will be chosen below. Use the Hellinger pseudometric on indices, $d(f, g) = h(\mathbb{P}_f, \mathbb{P}_g)$, so

$$\mathcal{N}(\delta, \mathcal{F}_U[B], d) = \mathcal{N}(\delta, \mathcal{P}_U[B], h).$$

For all sufficiently large B , $\delta_B < 1$, and (6) with $\kappa, b \geq 1$ gives

$$\log \mathcal{N}(\delta_B, \mathcal{F}_U[B], d) \leq C' \{1 + \log(1/c_0)\} W_B.$$

Assumption 2.5 supplies a target packing with logarithmic cardinality at least $c_{\text{pack}} c_0^{-2/a} W_B$ whenever $\delta_B \leq \delta_0$. Choose c_0 sufficiently small that

$$c_{\text{pack}} c_0^{-2/a} > C' \{1 + \log(1/c_0)\}.$$

The packing is then larger than the candidate cover. Applying Lemma A.1 with $R(f, g) = h^2(\mathbb{P}_f, \mathbb{P}_g)$, $p = 2$, $c_d = 1$, and $\mathcal{C} = \mathcal{F}_U[B]$ gives the lower bound $\delta_B^2 = c_0^2 W_B^{-a}$, proving (7). $\square$

Proof of Corollary 2.7. Divide the loop upper bound (5) by the untied lower bound (7). $\square$

B.2 Proofs for Section 3

Proof of Proposition 3.2. Conditional models. The calculations in Sections C.1–C.3 establish the two-sided metric equivalence in part (i). Lemma C.1 establishes part (ii). The root-density envelope converts uniform index covers into Hellinger brackets and internal Hellinger covers, proving part (iii). Finally, Lemma C.3 and the lower metric equivalence give part (iv), after a fixed rescaling of the packing radius, with $a = 2s/d$.

Energy-based generative model. Lemma C.2 proves parts (i)–(iii), using $d_E = d_c$ and $d_c(f, g) \leq \|f - g\|_{L^2(\mu)}$. For part (iv), use the zero-mean packing in Lemma C.3. Its elements satisfy $d_c(f_{j,\delta}, f_{k,\delta}) = \|f_{j,\delta} - f_{k,\delta}\|_{L^2(\mu)}$. The lower bound in (S9) transfers this packing to Hellinger distance after a fixed rescaling of the radius, with exponent $a = 2s/d$. $\square$

Existence of the neural sieve MLE. For fixed (q, r, T) , the closed, bounded coefficient box is compact, and the network output is continuous in its coefficients. The stabilized layer normalizations preserve this continuity. In each of the four models, the sample log likelihood is jointly measurable in the observations and coefficients and is continuous in the coefficients. For the energy model, continuity of the normalizing constant follows from bounded convergence, since $|g| \leq H$. The measurable maximum theorem therefore supplies a measurable exact maximizer on the coefficient box, and hence an estimator satisfying Definition 2.3 for every $\varepsilon_n \geq 0$.

Lemma B.1 (Index approximation for looped neural classes). *Fix d, s, H and $j \in \{\text{F}, \text{Tr}\}$. For $j = \text{Tr}$, also fix the token partition and $\varepsilon_{\text{LN}} > 0$. There exist constants $C, c > 0$ and architecture-specific thresholds q_0^j, r_0^j, M_0^j , depending only on these fixed quantities, such that, for fixed $M_{\text{par}} \geq M_0^j$, integers $q \geq q_0^j$, $r \geq r_0^j$, and sufficiently large T , with $a = 2s/d$, the following bound holds.*

$$\sup_{f_0 \in \mathcal{H}_d^s(H)} \inf_{g \in \mathcal{F}_{L,j}(q,r,T)} \|f_0 - g\|_{L^2(\mu)}^2 \leq C \left[\left(\frac{\log(erT)}{rT} \right)^a + \exp(-cT^{1/h_0^j}) \right], \quad (\text{S6})$$

where $h_0^{\text{F}} = 1$ and $h_0^{\text{Tr}} = 2$. The exponential term may be omitted when $T \geq C \log(erT)$ for the residual FFN and when $T \geq C \log^2(erT)$ for the Transformer.

For fixed admissible q, r , the initialization condition gives the simplified bound $\sup_{f_0 \in \mathcal{H}_d^s(H)} \inf_{g \in \mathcal{F}_{L,j}(q,r,T)} \|f_0 - g\|_{L^2(\mu)}^2 \lesssim \{\log(eT)/T\}^a$.

Proof of Lemma B.1. For $j \in \{\text{F}, \text{Tr}\}$, Theorems D.1 and E.1 give squared index error at most CK^{-2s} at every dyadic resolution K whenever

$$T \geq C\{\log^{h_0^j}(eK) + (K^d/r) \log(eK)\}, \quad h_0^{\text{F}} = 1, \quad h_0^{\text{Tr}} = 2.$$

Write $L = \log(erT)$. For sufficiently small constants $c_1, c_2 > 0$, set

$$J_0 = \min\{c_1 rT/L, \exp(c_2 T^{1/h_0^j})\}, \quad K = 2^{\lfloor d^{-1} \log_2 J_0 \rfloor}.$$

For all sufficiently large T , uniformly over the admissible widths, $K \geq 2$ and $K^d \asymp J_0$. Moreover, $\log(eK) \leq CL$ and $\log(eK) \leq C + c_2 T^{1/h_0^j}/d$. The two iteration costs are therefore at most $C' c_2^{h_0^j} T$ and $C'' c_1 T$, after absorbing the fixed additive constant. Choosing c_1, c_2 sufficiently small makes their sum at most T . The construction at this resolution consequently gives

$$\sup_{f_0 \in \mathcal{H}_d^s(H)} \inf_{g \in \mathcal{F}_{L,j}(q,r,T)} \|f_0 - g\|_{L^2(\mu)}^2 \leq C\{[L/(rT)]^a + \exp(-cT^{1/h_0^j})\}, \quad a = 2s/d.$$

When $T \geq CL^{h_0^j}$, taking $J_0 = c_1 rT/L$ alone satisfies the same iteration bound and removes the exponential term. This proves (S6) and both initialization conditions.

For fixed admissible q, r , $\log(erT)/(rT) \lesssim \log(eT)/T$, which gives the final simplified bound. $\square$

Proof of Theorem 3.3. Proposition 3.2(i) bounds squared Hellinger distance by a constant times squared $L^2(\mu)$ distance in every likelihood model, using $d_c(f, g) \leq \|f - g\|_{L^2(\mu)}$ for the energy model. Applying this bound to the candidates from Lemma B.1 proves (9), with the same initialization conditions and fixed-width simplification. Table 1 shows that $B_L^j(q, r)$ is independent of T , giving the final budget statement. $\square$

Lemma B.2 (Uniform covering of the neural index classes). *Fix $M_{\text{par}} > 0$ and $j \in \{\text{F}, \text{Tr}\}$. There exists a constant $C > 1$, depending only on the fixed problem and implementation parameters, such that the following bounds hold.*

Let G be defined by (10). For positive integers q, r, T and $0 < \epsilon < 1$,

$$\log \mathcal{N}(\epsilon, \mathcal{F}_{L,j}(q, r, T), \|\cdot\|_\infty) \leq CB_L^j(q, r) \{\log(C/\epsilon) + G(q, r, T)\}. \quad (\text{S7})$$

For all sufficiently large B ,

$$\log \mathcal{N}(\epsilon, \mathcal{F}_{U,j}[B], \|\cdot\|_\infty) \leq C\{B^2 \log(eB) + B \log(C/\epsilon)\}. \quad (\text{S8})$$

Proof of Lemma B.2. Let Γ denote the parameter-sensitivity bound in Section C.6, (S11): for parameter vectors $\xi, \tilde{\xi}$, it satisfies $\|f_\xi - f_{\tilde{\xi}}\|_\infty \leq \Gamma \|\xi - \tilde{\xi}\|_\infty$. We use its recurrence and the parameter-grid bound (S12). We first verify the required sensitivity bounds on the full parameter boxes.

Residual FFN. We bound parameter sensitivity uniformly over the full coefficient box. Put

$L = C(q+1)^C(r+1)^C \geq 2$. The residual recurrence gives $\|z_t\|_\infty + 1 \leq L^{t+1}$. The state Lipschitz factor is at most L and the parameter derivative is at most L^{t+2} . Thus (S11) gives $\Gamma \leq (T+1)L^{C(T+1)}$, also for arbitrary untied sequences. Hence (10) is valid, after changing constants.

The full predictor box satisfies (S12). Since $\log \Gamma \leq CG(q, r, T)$, this proves the residual-FFN instance of (S7). The same sensitivity calculation applies to arbitrary untied sequences and is used for (S8).

Post-LN Transformer. Lemma C.4 verifies the sensitivity recurrence on the complete fitted box, including the affine initial state, all four affine attention projections, both layer normalizations, and the clipped affine output function. Its bound (S15) gives $\log \Gamma \leq CG(q, r, T)$. Equation (S12) therefore proves (S7). The lemma also applies to every untied block sequence and supplies the same sensitivity bound for (S8).

Budgeted union. For a budgeted untied class, the same calculations give $\log \Gamma \leq C(T+1)\log(eB)$ for every tuning triple in $\mathcal{I}_{U,j}(B)$. Since $B_U^j(q, r, T) \leq B$ and $T \leq B$, the parameter grid in (S12) has logarithmic cardinality at most

$$C\{B^2 \log(eB) + B \log(C/\epsilon)\}.$$

There are at most B^C admissible integer triples. Adding their logarithmic cardinality proves (S8). $\square$

Proof of Proposition 3.4. Apply Lemma B.2. Proposition 3.2(iii) converts the loop uniform cover into Hellinger brackets and converts the budgeted uniform cover into an internal Hellinger cover. A fixed rescaling of the radius is absorbed into $\log(C/u)$ or $\log(C/\epsilon)$, giving (11) and (12). For fixed admissible q, r , Table 1 and (10) give $B_L^j(q, r) = O(1)$ and $G(q, r, T) = O(T)$, proving the simplified entropy bound. $\square$

B.3 Proofs for Section 4

Proof of Theorem 4.1. Existence of a measurable exact sieve MLE was established in Section B.2. Theorem 3.3 supplies the Hellinger approximation bound (9). Proposition 3.4 gives (11), and Lemma C.5 verifies (2) with $e_{n,L} = e_{n,j}$ whenever $e_{n,j} < 1$. Proposition 3.2(ii) supplies the one-sided bridge (3). If $e_{n,j} = 1$, the conclusion follows after enlarging the constant because $h^2 \leq 2$; otherwise Corollary 2.4 gives the first bound in (14).

For (15), fix q, M_{par} and take the largest admissible width r_B under budget B . Table 1 gives $B_L^j(q, r) = \alpha_j(q) + (2q+1)r$, where $\alpha_F(q) = q(d+3) + 1$ and $\alpha_{\text{Tr}}(q) = 4q^2 + q(d+2N+9) + 1$. Thus $r_B = \lfloor (B - \alpha_j(q))/(2q+1) \rfloor \asymp B$ for sufficiently large B , and $r_B \geq r_0^j$. Consequently, $\log(er_B T)/(r_B T) \lesssim \log(eBT)/(BT)$ and $G(q, r_B, T) \lesssim T \log(eB)$ for $T \geq 1$. Using $B_L^j(q, r_B) \leq B$ in the first risk bound, applying the initialization refinement, and absorbing $\varepsilon_n \leq n^{-1}$ into $B \log(en)/n$ proves (15). All comparison constants are uniform in n, B, T , and f_0 .

For the fixed-budget specialization used in the Introduction, keep B fixed in (15). Since $\log(eBT) \lesssim_B \log(eT)$ and $\log(eB)$ is constant, the bound becomes $R_{L,j,n}^m \lesssim_B \{\log(eT)/T\}^a +$

$(T + \log(en))/n$. The iteration schedule (18) then gives the displayed fixed-budget rate in the Introduction. $\square$

Proof of Theorem 4.2. For $m \in \{G, \text{La}, \text{Be}\}$, use the packing in Lemma C.3 at scale $\delta_n = K_n^{-s}$, where $K_n \asymp n^{1/(2s+d)}$. Lemma C.1 and Jensen's inequality give

$$\text{KL}(\mathbb{P}_f^m \| \mathbb{P}_0^m) \leq \rho_{1/2}(p_f^m, p_0^m) \leq C \|f\|_{L^2(\mu)}^2.$$

The packing has logarithmic cardinality at least cK_n^d , and every member satisfies $\|f\|_{L^2(\mu)}^2 \leq CK_n^{-2s}$. Choosing the fixed multiplicative constant in K_n sufficiently large makes the product-law divergences a fixed small fraction of the logarithmic packing cardinality. Fano's inequality and the lower metric equivalence in Proposition 3.2(i) give the stated rate.

For $m = E$, use the zero-mean packing of the same lemma. Equation (S9) gives $\text{KL}(\mathbb{P}_f^E \| \mathbb{P}_0^E) \leq CK_n^{-2s}$ for every packing member. The same Fano argument applies. Assigning an arbitrary distribution estimator to its closest packing member converts an incorrect assignment into Hellinger error at least half the packing separation; estimates not dominated by the model's reference measure are handled with a common dominating measure. In every model the resulting squared Hellinger risk is at least $cK_n^{-2s} \asymp n^{-2s/(2s+d)} = n^{-a/(a+1)}$, proving (16). $\square$

Proof of Proposition 4.3. Equation (12) gives (6) with $\kappa = 2$ and $b = 1$ for the induced law class, while Proposition 3.2(iv) verifies the target-packing assumption. Theorem 2.6 yields (17), uniformly in n and over all sufficiently large B . $\square$

Proof of Theorem 4.4. Fix the family j and its admissible (q, r) . The parameter count is constant, and (10) gives $G(q, r, T_n) = O(T_n)$. Hence the fitted-sieve term $e_{n,j}^2$ is $O(T_n/n)$. With the choice (18),

$$\left\{ \frac{\log(erT_n)}{rT_n} \right\}^a + \frac{T_n}{n} \leq Cn^{-a/(a+1)} \{\log(en)\}^{a/(a+1)}.$$

The architecture-specific initialization remainder is negligible and $\varepsilon_n \leq n^{-1}$. The general bound (14) in Theorem 4.1 proves (19).

At the fixed cap B , Proposition 4.3 bounds the untied risk below by $c_U \{B^2 \log(eB)\}^{-a}$. Dividing (19) by this bound proves the limit in (20). $\square$

Proof of Theorem 4.5. By Table 1, $B_L^j(q, r)$ is affine and strictly increasing in r with slope $2q + 1$. Thus the maximizing choice of r_n satisfies the parameter budget. Since $q_n \asymp \log n$ and $B_n \asymp n^{\gamma_B}$, the counts give $r_n \asymp n^{\gamma_B} / \log n$ for all sufficiently large n . The condition $\gamma_B < 1/(a+1)$ implies that W_n/r_n diverges at a positive polynomial rate. Thus rounding T_n up changes its order by at most a constant, $r_n T_n \asymp W_n$, and both initialization thresholds hold.

Because $\ell_n \asymp \log n$, the approximation term in (14) is bounded by

$$C \left(\frac{\log n}{W_n} \right)^a.$$

Moreover, $B_L^j(q_n, r_n) \asymp B_n$ and $G(q_n, r_n, T_n) \asymp T_n \ell_n$, so the fitted-sieve term is

$$e_{n,j}^2 \leq C \frac{B_n T_n \ell_n}{n} \asymp C \frac{W_n q_n \ell_n}{n}.$$

Since $q_n \ell_n \asymp (\log n)^2$, the definition of W_n gives the order in (22) for both displays. This order tends to zero, so $e_{n,j} < 1$ eventually and the entropy lemma applies. The tolerance is smaller, completing the upper bound. Proposition 4.3 at $B = B_n$ gives $R_{U,j,n}^m(B_n) \geq c_U \{B_n^2 \log(e B_n)\}^{-a}$. Dividing the upper bound by the untied lower bound produces the power $-a/(a+1) + 2a\gamma_B < 0$, which dominates the logarithmic factors and proves (23). $\square$

C Auxiliary likelihood and complexity bounds

This appendix collects the likelihood calculations, Hölder target packing, and parameter-sensitivity estimates used in Appendix B. In the likelihood calculations, f and g take values in $[-H, H]$.

C.1 Gaussian regression

Suppose $Y = f(X) + \varepsilon$, where $X \sim \mu$ and $\varepsilon \sim N(0, \sigma^2)$ is independent of X , with known $\sigma^2 > 0$. Direct calculation gives

$$h^2(\mathbb{P}_f, \mathbb{P}_g) = 2 \int \left[1 - \exp \left\{ -\frac{(f-g)^2}{8\sigma^2} \right\} \right] d\mu.$$

Because $|f - g| \leq 2H$, positive constants depending only on H and σ bound this display above and below by $\|f - g\|_{L^2(\mu)}^2$. Maximum likelihood over an index-function class is empirical least squares.

C.2 Laplace regression

Suppose $X \sim \mu$ and $Y = f(X) + \varepsilon$, where ε is independent of X and has density $(2\tau)^{-1} \exp(-|u|/\tau)$ for fixed $\tau > 0$. For $v = |f(x) - g(x)|/\tau$, the conditional Hellinger affinity is $(1 + v/2)e^{-v/2}$, and hence

$$h^2(\mathbb{P}_f, \mathbb{P}_g) = 2 \int \{1 - (1 + v/2)e^{-v/2}\} d\mu.$$

On the bounded range $0 \leq v \leq 2H/\tau$, this quantity is uniformly equivalent to $\|f - g\|_{L^2(\mu)}^2$. Maximum likelihood is empirical absolute deviation.

C.3 Bernoulli binary-response model

Suppose $Y \mid X = x$ is Bernoulli with success probability $\text{sigmoid}\{f(x)\}$. On the bounded logit interval, the derivatives of both square-root probabilities are bounded. The derivative of the increasing coordinate is also bounded away from zero, so the mean value theorem

gives constants $c_{\text{Be}}, C_{\text{Be}} > 0$, depending only on H , such that

$$c_{\text{Be}} \|f - g\|_{L^2(\mu)}^2 \leq h^2(\mathbb{P}_f, \mathbb{P}_g) \leq C_{\text{Be}} \|f - g\|_{L^2(\mu)}^2.$$

Maximum likelihood is empirical logistic-loss minimization.

For all three families, the square-root conditional densities are absolutely continuous in the bounded index parameter and admit an L^2 envelope L for their almost-everywhere derivative. Here the norm is under the joint dominating measure $\mu \otimes \nu_Y$, where ν_Y is Lebesgue measure for the regression families and counting measure for Bernoulli. Thus

$$|\sqrt{p_t} - \sqrt{p_s}| \leq L|t - s|, \quad \|L\|_{L^2(\mu \otimes \nu_Y)} < \infty.$$

A uniform ϵ -net of index functions therefore yields density brackets with endpoints $(\sqrt{p_{g_j}} - \epsilon L)_+^2$ and $(\sqrt{p_{g_j}} + \epsilon L)^2$, of Hellinger width at most $2\epsilon \|L\|_{L^2(\mu \otimes \nu_Y)}$. Predictor covers therefore lift to Hellinger brackets of the target density family and to Hellinger covers of the architecture families with the same logarithmic order. The metric equivalences transfer the loop approximation and the disjoint-bump packing from $L^2(\mu)$ to Hellinger distance. The positive likelihood approximation condition is verified next.

Lemma C.1 (Positive likelihood approximation on the bounded index range). *For each of the Gaussian, Laplace, and Bernoulli conditional models, there is a constant C such that*

$$\rho_{1/2}(p_f, p_g) \leq C \|f - g\|_{L^2(\mu)}^2 \leq C h^2(\mathbb{P}_f, \mathbb{P}_g), \quad f, g : \mathcal{X} \rightarrow [-H, H].$$

Proof. Condition on $X = x$ and put $\Delta = f(x) - g(x)$. In the Gaussian model,

$$\int p_f^{3/2} p_g^{-1/2} d\nu_Y = \exp\{3\Delta^2/(8\sigma^2)\}.$$

For Laplace regression, with $v = |\Delta|/\tau$, the corresponding integral is

$$\{3e^{v/2} + e^{-3v/2}\}/4.$$

For Bernoulli regression, write $b(t) = \log(1 + e^t)$. The integral is

$$\exp\left\{b\left(\frac{3f - g}{2}\right) - \frac{3}{2}b(f) + \frac{1}{2}b(g)\right\}.$$

Each display equals one at $\Delta = 0$, has first derivative zero there, and has uniformly bounded second derivative on the bounded index range. Thus it is at most $1 + C\Delta^2$. Integrating in x and using $\rho_{1/2}(p_f, p_g) = 2\{\int p_f^{3/2} p_g^{-1/2} d(\mu \otimes \nu_Y) - 1\}$ proves the first inequality. The second is the metric equivalence proved in the three preceding subsections. $\square$

C.4 Energy-based generative models

Throughout this subsection, μ is uniform probability measure on $[0, 1]^d$ and f, g take values in $[-H, H]$. Densities are defined by (8). Then $e^{-2H} \leq p_f \leq e^{2H}$.

Lemma C.2 (Geometry and likelihood approximation of bounded energy-based generative

models). *There exist constants $C, c > 0$, depending only on H , such that the Kullback–Leibler divergence KL satisfies*

$$cd_c(f, g)^2 \leq h^2(\mathbb{P}_f, \mathbb{P}_g) \leq \text{KL}(\mathbb{P}_f \| \mathbb{P}_g) \leq Cd_c(f, g)^2, \quad (\text{S9})$$

and

$$\rho_{1/2}(p_f, p_g) \leq Ch^2(\mathbb{P}_f, \mathbb{P}_g). \quad (\text{S10})$$

A uniform u -cover of a bounded index class induces density brackets of Hellinger width at most $2 \sinh(u)$ with the same cardinality.

Proof. Put $\delta = g - f$ and $f_t = f + t\delta$. Differentiation of the log normalizer, justified by boundedness, gives

$$\frac{d^2}{dt^2} \log Z_{f_t} = \text{Var}_{\mathbb{P}_{f_t}}(\delta).$$

Consequently,

$$\text{KL}(\mathbb{P}_f \| \mathbb{P}_g) = \int_0^1 (1-t) \text{Var}_{\mathbb{P}_{f_t}}(\delta) dt \leq \frac{1}{2} e^{2H} \inf_{c \in \mathbb{R}} \|\delta - c\|_2^2.$$

For the lower bound, the logarithm is e^H -Lipschitz on $[e^{-H}, e^H]$, the range of the square-root densities. Thus

$$\begin{aligned} d_c(f, g) &\leq \|f - g - \log Z_f + \log Z_g\|_2 \\ &= \|\log p_f - \log p_g\|_2 \leq 2e^H \|\sqrt{p_f} - \sqrt{p_g}\|_2. \end{aligned}$$

Together with $h^2 \leq \text{KL}$, this proves (S9).

To prove (S10), put $t = p_f/p_g$. Then $e^{-4H} \leq t \leq e^{4H}$ and $\int p_g(t-1) d\mu = 0$, so

$$\rho_{1/2}(p_f, p_g) = 2 \int p_g \{t^{3/2} - 1 - \frac{3}{2}(t-1)\} d\mu.$$

The identity $2\{t^{3/2} - 1 - \frac{3}{2}(t-1)\} = (\sqrt{t} - 1)^2(2\sqrt{t} + 1)$ therefore gives

$$\rho_{1/2}(p_f, p_g) \leq (2e^{2H} + 1)h^2(\mathbb{P}_f, \mathbb{P}_g),$$

which proves (S10).

If $\|f - f_j\|_\infty \leq u$, then $|\log Z_f - \log Z_{f_j}| \leq u$ and

$$e^{-2u} p_{f_j} \leq p_f \leq e^{2u} p_{f_j}.$$

The square-root width of this bracket is $(e^u - e^{-u})\|\sqrt{p_{f_j}}\|_2 = 2 \sinh(u)$. $\square$

C.5 Hölder target packing

Lemma C.3 (Local packing of the Hölder class). *Fix $d \geq 1$, $s > 0$ and $H > 0$, and let μ be uniform probability measure on $[0, 1]^d$. There exist constants $c, C, \delta_0 > 0$, depending only on*

d, s, H , such that, for every $0 < \delta \leq \delta_0$, the class $\mathcal{H}_d^s(H)$ contains $f_{1,\delta}, \dots, f_{M_\delta,\delta}$ satisfying

$$\min_{j \neq k} \|f_{j,\delta} - f_{k,\delta}\|_{L^2(\mu)} \geq 8\delta, \quad \max_j \|f_{j,\delta}\|_{L^2(\mu)} \leq C\delta, \quad \log M_\delta \geq c\delta^{-d/s}.$$

The functions can additionally be chosen so that $\int f_{j,\delta} d\mu = 0$ for every j .

Proof. Choose a nonzero smooth function ψ supported strictly inside $(0, 1)^d$ with integral zero. Such a function is obtained by differentiating a suitable compactly supported smooth function in one coordinate. For an integer $m \geq 1$ and $\omega \in \{0, 1\}^{m^d}$, define

$$f_\omega(x) = c_* m^{-s} \sum_{j \in \{0, \dots, m-1\}^d} \omega_j \psi(mx - j).$$

For sufficiently small fixed $c_* > 0$, depending only on d, s, H and ψ , these functions belong to $\mathcal{H}_d^s(H)$. Derivatives of order $v \leq k$ are bounded by Cm^{v-s} , and the order- k derivatives have uniformly bounded α -Hölder seminorm. Separation of the bump supports gives the same bound across distinct cells. Every f_ω has integral zero, and disjointness of the supports gives

$$\|f_\omega - f_{\omega'}\|_{L^2(\mu)}^2 = c_*^2 m^{-2s-d} \|\psi\|_{L^2(\mathbb{R}^d)}^2 \sum_j |\omega_j - \omega'_j|.$$

A binary code with Hamming separation at least a fixed positive fraction of m^d has logarithmic cardinality at least cm^d for all sufficiently large m . The corresponding functions therefore have pairwise $L^2(\mu)$ distances at least $c_{\text{sep}} m^{-s}$, while every function has norm at most Cm^{-s} , for a fixed $c_{\text{sep}} > 0$.

For sufficiently small δ , take $m = \lfloor (c_{\text{sep}}/(8\delta))^{1/s} \rfloor$. Then $m \asymp \delta^{-1/s}$ and the pairwise distances are at least 8δ . The norm bound and logarithmic cardinality become $C\delta$ and $c\delta^{-d/s}$, respectively. Relabeling these functions proves the lemma, including the zero-mean assertion. $\square$

C.6 Parameter sensitivity and architecture covers

Let $\xi, \tilde{\xi}$ be two parameter vectors and put $\delta = \|\xi - \tilde{\xi}\|_\infty$. Write e_t for a uniform bound on the distance between their states after t iterations. Suppose that nonnegative constants, uniform on the full parameter box, satisfy

$$e_0 \leq c_E \delta, \quad e_{t+1} \leq a_t e_t + b_t \delta, \quad |f_\xi - f_{\tilde{\xi}}| \leq c_R e_T + d_R \delta.$$

For the untied class, assume the same bounds hold for every admissible sequence of block parameters. Iteration gives

$$\Gamma = \max \left\{ 1, d_R + c_R \left(c_E \prod_{t < T} a_t + \sum_{j < T} b_j \prod_{t=j+1}^{T-1} a_t \right) \right\}, \quad \|f_\xi - f_{\tilde{\xi}}\|_\infty \leq \Gamma \delta. \quad (\text{S11})$$

Let P denote the total parameter count and $\mathcal{G} = \{f_\xi : \xi \in [-M_{\text{par}}, M_{\text{par}}]^P\}$ the corresponding predictor class. Partition the P -dimensional parameter box into cells of ℓ^∞ diameter at most ϵ/Γ and choose one parameter vector from each nonempty cell. Their predictors form

an internal cover, so

$$\log \mathcal{N}(\epsilon, \mathcal{G}, \|\cdot\|_\infty) \leq P \log(1 + 4M_{\text{par}}\Gamma/\epsilon). \quad (\text{S12})$$

The same recurrence applies to shared and untied parameters, while their parameter counts differ. Applied to the loop classes and the budgeted unions, this calculation gives the two uniform covering bounds in Lemma B.2. Proposition 3.2(iii) transfers these predictor covers into Hellinger brackets and internal Hellinger covers.

The following lemma supplies explicit sensitivity bounds for the complete post-LN Transformer class.

Lemma C.4 (Uniform sensitivity of the complete post-LN class). *Consider the Transformer of Section 3.2, with $x \in [0, 1]^d$, positive integers q, r, N , parameter bound $M_{\text{par}} > 0$, and fixed $\varepsilon_{\text{LN}} > 0$. Write $m = M_{\text{par}}$, $\varepsilon = \varepsilon_{\text{LN}}$, and $R = \max\{1, m(d+1), m(\sqrt{q}+1)\}$. There exist $L_{\text{blk}}, K_{\text{blk}} > 0$, depending only on $d, q, r, M_{\text{par}}, \varepsilon_{\text{LN}}$, with the following properties. For matrices, $\|\cdot\|_\infty$ denotes the maximum absolute entry. For any two states $S, \tilde{S} \in \mathbb{R}^{N \times q}$ with $\|S\|_\infty, \|\tilde{S}\|_\infty \leq R$, and any two block parameter vectors $\theta, \tilde{\theta}$ in the complete prescribed box,*

$$\|\Phi_\theta(S) - \Phi_{\tilde{\theta}}(\tilde{S})\|_\infty \leq L_{\text{blk}}\|S - \tilde{S}\|_\infty + K_{\text{blk}}\|\theta - \tilde{\theta}\|_\infty. \quad (\text{S13})$$

Every initial state and every block output has norm at most R , so (S13) applies at every iteration. If $\delta = \|\xi - \tilde{\xi}\|_\infty$ is the distance between two complete predictor parameter vectors and $e_t = \sup_{x \in [0, 1]^d} \|S_t(x) - \tilde{S}_t(x)\|_\infty$, where S_t and $\tilde{S}_t$ are the states after t updates under ξ and $\tilde{\xi}$, respectively, then

$$e_0 \leq (d+1)\delta, \quad e_{t+1} \leq L_{\text{blk}}e_t + K_{\text{blk}}\delta, \quad \|f_\xi - f_{\tilde{\xi}}\|_\infty \leq Nqm e_T + (NqR + 1)\delta. \quad (\text{S14})$$

These inequalities hold both for shared block parameters and for every admissible sequence of untied block parameters. Consequently, for every fixed numerical $C_6 \geq 2$, the sensitivity constant in (S11) satisfies

$$\log \Gamma \leq C\{\log(T+1) + (T+1)\log(C_6(q+1)(r+1))\}, \quad T \geq 1, \quad (\text{S15})$$

where C depends only on $d, N, M_{\text{par}}, \varepsilon_{\text{LN}}$. The bounds hold with all reference-token coordinates treated as free fitted parameters.

Proof. Throughout the proof, δ bounds all coefficient differences being compared. We first bound layer normalization, then combine attention and FFN perturbations into a one-block bound. Including the input and output functions gives the recurrence; iterating it yields the bound on $\log \Gamma$.

Step 1: Layer normalization. Put $\ell = m\sqrt{q}/\varepsilon$ and $\nu = \sqrt{q} + 1$. Let $P_q = I_q - q^{-1}\mathbf{1}\mathbf{1}^\top$, and define

$$h(z) = \frac{P_q z}{s(z)}, \quad s(z) = \{q^{-1}\|P_q z\|_2^2 + \varepsilon\}^{1/2}.$$

The row-wise layer normalization is $\text{LN}_{\gamma, \beta}(z) = \gamma \odot h(z) + \beta$. Since $\|h(z)\|_2 \leq \sqrt{q}$, its output obeys

$$\|\text{LN}_{\gamma, \beta}(z)\|_\infty \leq m(\sqrt{q} + 1) \quad (\text{S16})$$

for every $z \in \mathbb{R}^q$ and every admissible gain and bias. The positive stabilizer gives the derivative

$$Dh(z) = s(z)^{-1}P_q - \frac{(P_q z)(P_q z)^\top}{q s(z)^3}.$$

On the span of $P_q z$, its eigenvalue is $\varepsilon/s(z)^3$; on the orthogonal directions within the centered subspace it is $1/s(z)$, and it vanishes on the constant direction. Since $s(z) \geq \sqrt{\varepsilon}$, every eigenvalue is at most $\varepsilon^{-1/2}$. When $P_q z = 0$, the derivative is $\varepsilon^{-1/2}P_q$. Thus $\|Dh(z)\|_{2 \rightarrow 2} \leq \varepsilon^{-1/2}$ globally, including $q = 1$. Integration along the segment between z and $\tilde{z}$, followed by $\|v\|_2 \leq \sqrt{q}\|v\|_\infty$, gives

$$\|h(z) - h(\tilde{z})\|_\infty \leq \sqrt{q/\varepsilon} \|z - \tilde{z}\|_\infty.$$

Separating the changes in the input, gain, and bias therefore gives

$$\|\text{LN}_{\gamma,\beta}(z) - \text{LN}_{\tilde{\gamma},\tilde{\beta}}(\tilde{z})\|_\infty \leq \ell \|z - \tilde{z}\|_\infty + \nu\delta, \quad (\text{S17})$$

whenever each gain and bias differs by at most δ . This estimate requires no bound on the arguments of layer normalization.

Step 2: Attention. Put $A = m(qR + 1)$ and $\zeta = 1 + 4\sqrt{q}A^2$, and define

$$a_{\text{att}} = (qm)^2\zeta, \quad b_{\text{att}} = qm\zeta(qR + 1) + qA + 1.$$

Set $u = \|S - \tilde{S}\|_\infty$ and let δ bound the differences of all block coefficients. Each of the affine query, key, and value projections has entries bounded in magnitude by A , and each difference is bounded by

$$D = qm u + (qR + 1)\delta.$$

Indeed, separate the change in the state from the change in the projection matrix and its bias. The scaled query-key logits $Z = QK^\top/\sqrt{q}$ consequently satisfy

$$\|Z - \tilde{Z}\|_\infty \leq 2\sqrt{q}AD.$$

For softmax σ on $\mathbb{R}^N$, its Jacobian obeys

$$\|D\sigma(z)v\|_1 = \sum_i \sigma_i(z) \left| v_i - \sum_j \sigma_j(z)v_j \right| \leq 2\|v\|_\infty.$$

Hence each row of the attention-weight matrix changes by at most $4\sqrt{q}AD$ in ℓ^1 norm. Writing $V_{\text{av}} = \sigma(Z)V$, with softmax applied separately to each row, the probability weights imply

$$\|V_{\text{av}}\|_\infty \leq A, \quad \|V_{\text{av}} - \tilde{V}_{\text{av}}\|_\infty \leq A(4\sqrt{q}AD) + D = \zeta D.$$

The fourth affine projection, applied to V_{av} , therefore gives

$$\begin{aligned} \|\text{Attn}_\theta(S) - \text{Attn}_{\tilde{\theta}}(\tilde{S})\|_\infty &\leq qm\zeta D + (qA + 1)\delta \\ &= a_{\text{att}}u + b_{\text{att}}\delta. \end{aligned}$$

All four projection matrices and all four projection biases have been included. The probability-row bound is why no factor N is needed in this attention estimate.

Step 3: FFN and the complete block. Put $a_{\text{ff}} = qrm^2$ and $b_{\text{ff}} = 2rA + 1$. Adding the first residual and using (S17) gives, for the first normalized states $Y, \tilde{Y}$,

$$v := \|Y - \tilde{Y}\|_{\infty} \leq \ell(1 + a_{\text{att}})u + (\ell b_{\text{att}} + \nu)\delta.$$

By (S16), both these states have norm at most R . For the row-wise FFN, the hidden affine values and their ReLU outputs have norm at most A . ReLU is 1-Lipschitz, so the change of a hidden output is at most $qmv + (qR + 1)\delta$. The second FFN affine map then yields

$$\begin{aligned} \|\text{FFN}_{\theta}(Y) - \text{FFN}_{\tilde{\theta}}(\tilde{Y})\|_{\infty} &\leq rm\{qmv + (qR + 1)\delta\} + (rA + 1)\delta \\ &= a_{\text{ff}}v + b_{\text{ff}}\delta. \end{aligned}$$

Adding the second residual and applying the second layer-normalization bound gives

$$\|\Phi_{\theta}(S) - \Phi_{\tilde{\theta}}(\tilde{S})\|_{\infty} \leq \ell(1 + a_{\text{ff}})v + (\ell b_{\text{ff}} + \nu)\delta,$$

which proves (S13) with

$$\begin{aligned} L_{\text{blk}} &= \ell^2(1 + a_{\text{ff}})(1 + a_{\text{att}}), \\ K_{\text{blk}} &= \ell(1 + a_{\text{ff}})(\ell b_{\text{att}} + \nu) + \ell b_{\text{ff}} + \nu. \end{aligned}$$

Step 4: Input, output, and iteration. We include the endpoints and the first application of the block. Every coordinate of data token j at input has magnitude at most $m(|I_j| + 1) \leq m(d + 1)$, because $x \in [0, 1]^d$. Each controller and reference coordinate has magnitude at most m . Thus the initial state has norm at most R , even though it has not been layer-normalized. The same calculation with coefficient differences gives $e_0 \leq (d + 1)\delta$, including the free controller and reference rows. Every subsequent block output satisfies (S16). The one-block bound therefore applies at every time, even when the block parameter vector changes between applications within the prescribed box. Since every individual block's parameter difference is at most the max-norm difference of the full untied vector, the same recurrence holds for shared and untied parameterizations.

Before clipping, the readout is an affine function of Nq coordinates. Separating its state and coefficient changes bounds its difference by $Nqm e_T + (NqR + 1)\delta$. The fixed clipping map Π_H is 1-Lipschitz, so this also bounds the predictor difference. This proves (S14).

Step 5: The logarithmic sensitivity bound. To prove (S15), we bound all constants by a single polynomial majorant. Put

$$c = \max\{2, d + 1, N, m, \varepsilon^{-1/2}\}, \quad P = 32c^{16}(q + 1)^8(r + 1).$$

Writing $\bar{q} = q + 1$ and $\bar{r} = r + 1$ only in the next bounds, the definitions give

$$\begin{aligned} R &\leq c^2 \bar{q}^{1/2}, \quad A \leq c^3 \bar{q}^{3/2}, \quad \zeta \leq 5c^6 \bar{q}^{7/2}, \quad \ell \leq c^2 \bar{q}^{1/2}, \quad \nu \leq 2\bar{q}^{1/2}, \\ 1 + a_{\text{att}} &\leq 6c^8 \bar{q}^{11/2}, \quad b_{\text{att}} \leq 7c^9 \bar{q}^6, \quad 1 + a_{\text{ff}} \leq 2c^2 \bar{q} \bar{r}, \quad b_{\text{ff}} \leq 3c^3 \bar{q}^{3/2} \bar{r}. \end{aligned}$$

In particular, $L_{\text{blk}} \leq 12c^{14}\bar{q}^{15/2}\bar{r}$ and $K_{\text{blk}} \leq 23c^{15}\bar{q}^8\bar{r}$, so

$$\max\{2, L_{\text{blk}}, K_{\text{blk}}, d+1, Nqm, NqR+1\} \leq P.$$

Iterating the recurrence gives

$$e_T \leq \delta \left\{ (d+1)L_{\text{blk}}^T + K_{\text{blk}} \sum_{j=0}^{T-1} L_{\text{blk}}^j \right\} \leq \delta(T+1)P^{T+1}.$$

Thus the constant from (S11) obeys

$$\Gamma \leq (T+2)P^{T+2} \leq (T+1)P^{2(T+1)}, \quad T \geq 1.$$

Finally, for any fixed numerical $C_6 \geq 2$,

$$\log P \leq \left\{ 8 + \frac{\log(32c^{16})}{\log 2} \right\} \log(C_6(q+1)(r+1)).$$

Taking logarithms proves (S15); in particular, the fixed numerical C_6 in (10) need not be changed. $\square$

Lemma C.5 (Bracketing resolution for a complete parameter class). *Suppose $P \geq 1$, $G \geq 0$, and, for some constant $C > 1$, $H(u, \mathcal{P}, h) \leq CP\{\log(C/u) + G\}$ for $0 < u < 1$. For a sufficiently large constant K , put $e_n^2 = KP\{\log(en) + G\}/n$. If $e_n < 1$, then (S2) holds for $e_n \leq t \leq \sqrt{2}$.*

Proof. Monotonicity of bracketing entropy extends its bound to bounded positive u by increasing the constant. On the integration interval, its square root is at most $C\sqrt{P\{\log(C/t^2) + G\}}$. The interval has length at most $\sqrt{2}t$. Since $t \geq e_n \geq n^{-1/2}$, its integral is bounded by

$$Ct\sqrt{P\{\log(en) + G\}} \leq CK^{-1/2}\sqrt{n}t^2.$$

Choosing K sufficiently large proves the assertion for the fixed Wong–Shen constants. $\square$

Lemma C.6 (Untied budget-union entropy). *Suppose there is a constant $C > 1$ such that every architecture in the untied budget union satisfies $B_U \leq B$, $T \leq B$, the induced-law map satisfies $h(\mathbb{P}_f, \mathbb{P}_g) \leq C\|f - g\|_\infty$, and*

$$\log \Gamma \leq C(T+1)^\beta \log(eB), \quad \beta \geq 1,$$

uniformly over its full parameter box. If $|\mathcal{I}_U(B)| \leq B^C$, then (6) holds with $\kappa = \beta + 1$ and $b = 1$.

Proof. For each architecture, the metric transfer, (S12), and $T \leq B$ give

$$\log \mathcal{N}(\epsilon, \mathcal{P}_U, h) \leq C\{B^{\beta+1} \log(eB) + B \log(1/\epsilon)\}.$$

Taking the union adds at most $\log |\mathcal{I}_U(B)| \leq C \log(eB)$, which is absorbed by the first term. This proves the claim. $\square$

In the FFN and post-LN examples, the dense counts imply $q, r \leq B$. Because each untied update contains at least one fitted scalar, $T \leq B$. Thus there are at most B^3 integer triples. Their sensitivity bounds have $\beta = 1$, so the lemma gives $\kappa = 2$ separately for the complete FFN and Transformer unions.

D Residual FFN approximation construction

We construct a shared residual block that approximates a Hölder target at a prescribed dyadic resolution. The following result is the finite-resolution input to Lemma B.1.

Theorem D.1 (Finite-resolution construction for looped residual FFNs). *Fix $d \geq 1$, $s > 0$ and $H > 0$, and let μ be uniform probability measure on $[0, 1]^d$. There exist a constant $C > 0$ and thresholds $q_0^F, r_0^F, M_0^F > 0$, depending only on d, s, H , with the following property. For every $M_{\text{par}} \geq M_0^F$, integers $q \geq q_0^F$, $r \geq r_0^F$, dyadic resolution $K = 2^m$ with $m \geq 1$, and $f_0 \in \mathcal{H}_d^s(H)$, there is one admissible input-function, update-block, and output-function parameter choice for the residual FFN of Section 3.2 whose looped candidates $g_T \in \mathcal{F}_{L,F}(q, r, T)$ satisfy*

$$\|g_T - f_0\|_{L^2(\mu)}^2 \leq CK^{-2s} \quad (\text{S18})$$

for every integer

$$T \geq C\{\log(eK) + (K^d/r) \log(eK)\}. \quad (\text{S19})$$

The parameter choice may depend on f_0, K, q, r , but is the same for all such T . Every scalar coefficient belongs to $[-M_{\text{par}}, M_{\text{par}}]$, and all endpoint and block coordinates are counted in $B_L^F(q, r)$, which is independent of T .

Proof roadmap. We approximate f_0 on a grid of K^d cells by local Taylor polynomials and encode their quantized coefficients as scalar records. Section D.1 constructs one fixed-dimensional shared evaluator: it decodes a record, evaluates its polynomial, and uses a fine-cell gate to retain the contribution from the cell containing x . Each record costs $O(\log(eK))$ iterations. Section D.2 adds a spatial selector that loads only the group of records containing x . Choosing $N_{\text{grp}} \asymp \min\{r, K^d\}$ groups makes the number of records scanned comparable to $1 + K^d/r$. Including initialization therefore gives the iteration bound (S19). Taylor and numerical errors are $O(K^{-s})$ away from narrow cell-boundary strips; clipping bounds the error on these strips, whose measure is $O(K^{-2s})$. Combining the two regions proves (S18). The final proof in Section D.2 verifies the width and coefficient bounds and that the same parameter choice works for every larger iteration count.

D.1 Shared Taylor-table evaluator

This subsection constructs a shared residual block that reads encoded cell records, evaluates their local Taylor polynomials, and uses a cell gate to retain the contribution from the cell containing the input. We establish its approximation error, state dimension, hidden width, and number of iterations per record.

Fix $k = \lceil s \rceil - 1$ and $\alpha = s - k \in (0, 1]$. Index Taylor terms by ordered coordinate lists

$$\mathcal{W}_k = \prod_{\ell=0}^k \{1, \dots, d\}^\ell, \quad M_{\text{Tay}} = |\mathcal{W}_k| = \sum_{\ell=0}^k d^\ell, \quad V = \sum_{w \in \mathcal{W}_k} |w| = \sum_{\ell=0}^k \ell d^\ell.$$

The empty list indexes the constant term. Put $B_{\text{eval}} = 32(M_{\text{Tay}} + 1) + 1$. The evaluator dimensions are

$$q_0 = 9d + 7M_{\text{Tay}} + 5V + 29, \quad r_0 = (7d + 7M_{\text{Tay}} + 5V + 14)(8d + 62). \quad (\text{S20})$$

For $K = 2^m$, $m \geq 1$, use the integer precisions

$$b = J = (k + 1)m, \quad G = (2(k + 1) + 1)m + 2.$$

These choices give $2^{-b}, 4^{-J} = O_s(K^{-s})$. Define

$$\tau_m = d(5 + 2m) + M_{\text{Tay}}(5 + 2b) + V(3 + 2J) + 2G + 11. \quad (\text{S21})$$

Lemma D.2 (Taylor-table engine). *Assume $M_{\text{par}} \geq \max\{3B_{\text{eval}}, H\}$. There is one residual FFN block with dimensions (S20) and coefficients bounded by $3B_{\text{eval}}$ that has the following property. For every $f_0 \in \mathcal{H}_d^s(H)$ and every ordered list $\mathcal{J}$ of $1 \leq N_* \leq K^d$ distinct fine-grid cells, initialize it with $x \in [0, 1]^d$ and the scalar code of their cell locations and quantized Taylor coefficients defined in (S24). After $N_*\tau_m$ iterations it is in an absorbing terminal state. Every evaluator data coordinate, including its timer, is bounded in magnitude by two; the returned normalized output u satisfies $|u| \leq 1$. Its returned value $g_{\mathcal{J}} = Hu$ obeys*

$$|g_{\mathcal{J}}(x) - f_0(x)| \leq C_{d,s} H K^{-s}$$

inside any listed cell at distance at least 2^{-G} from every face. For a list of all cells, $\|g_{\mathcal{J}} - f_0\|_2^2 \leq C_{d,s} H^2 K^{-2s}$. The same block works for all targets and lists at fixed d, s, K ; the target and list enter only through the initial code. Its affine input function and output function satisfy the stated parameter bound.

D.1.1 Ordered Taylor records and scalar encoding

For each cell we store its location and the coefficients of a local polynomial. We first turn these numbers into binary digits, assemble one record per cell, and then pack the records into a single scalar. The shared block reads this scalar one bit at a time to recover the information needed for polynomial evaluation. The encoding is chosen when constructing the approximating network; during evaluation the network receives only the input x and the encoded records.

Local Taylor approximation. Partition $[0, 1]^d$ into cells of side $h = K^{-1}$. If a_j is a cell's lower corner, put $x_j = a_j + (h/2)\mathbf{1}$. For $w = (i_1, \dots, i_\ell) \in \mathcal{W}_k$, write $\partial_w f = \partial_{i_1} \cdots \partial_{i_\ell} f$ and $(x - x_j)^w = \prod_{t=1}^\ell (x_{i_t} - x_{j,i_t})$, with the usual empty-product convention. The normalized polynomial is

$$P_j(x) = \sum_{w \in \mathcal{W}_k} c_{j,w} (x - x_j)^w, \quad c_{j,w} = \frac{\partial_w f_0(x_j)}{H|w|!}. \quad (\text{S22})$$

This is the ordinary multivariate Taylor polynomial: an order- ℓ Fréchet derivative expands into all d^ℓ ordered coordinate choices, each with factor $1/\ell!$. Merging equal monomials gives the usual multi-index coefficients, but the evaluator does not perform that merge. All $c_{j,w}$ belong to $[-1, 1]$.

Taylor expansion along the segment from x_j to an interior point x in the cell gives

$$|HP_j(x) - f_0(x)| \leq d^k H h^{k+\alpha} = d^k H h^s. \quad (\text{S23})$$

For $k = 0$, this is the Hölder condition. For $k \geq 1$, subtract the order- k derivative at x_j in the integral remainder. Expanding the directional derivative into its d^k coordinate terms bounds its variation by $d^k H \|x - x_j\|_\infty^{k+\alpha}$. The integral weight has mass at most one. This argument includes $\alpha = 1$ and uses no regularity outside the cube.

Quantizing the coefficients. The cell location lies on the grid and can be stored exactly. Its Taylor coefficients need finite binary representations. For any coefficient $c_{j,w} \in [-1, 1]$, define

$$n_b(c_{j,w}) = \min\{2^b - 1, \lfloor 2^{b-1}(c_{j,w} + 1) \rfloor\}, \quad \tilde{c}_{j,w} = 2^{1-b} n_b(c_{j,w}) - 1.$$

The integer $n_b(c_{j,w})$ is between zero and $2^b - 1$, so it fits in b binary digits. Decoding this integer and applying the second formula recovers $\tilde{c}_{j,w}$, with $|\tilde{c}_{j,w} - c_{j,w}| \leq 2^{1-b}$ and $\tilde{c}_{j,w} \in [-1, 1]$.

The contents of one cell record. A record contains the following fields, in the order shown.

Field	Stored values	Number of bits
Location	$Ka_{j,1}, \dots, Ka_{j,d}$	dm
Coefficients	$n_b(c_{j,w})$, in the fixed order of $\mathcal{W}_k$	$M_{\text{Taylor}} b$
Continuation	One if another cell follows; zero for the last cell	1

Each location integer lies in $\{0, \dots, 2^m - 1\}$ and uses exactly m bits; each coefficient integer uses exactly b bits. A fixed-length binary string for one integer is called a word, including zero padding to fill its prescribed length. Within each word, the bits are stored from least to most significant. Thus an ℓ -bit integer $n = \sum_{t=1}^{\ell} \eta_t 2^{t-1}$ is stored in the order $\eta_1, \dots, \eta_\ell$, where $\eta_t \in \{0, 1\}$. For example, the integer one in a two-bit word is stored as $(1, 0)$.

The lower corner determines the cell center x_j and its faces because h is fixed. The polynomial factors are determined by $\mathcal{W}_k$, so they do not need to be stored. Each record therefore has exactly $dm + M_{\text{Taylor}} b + 1$ bits. The common field lengths tell the decoder where each location or coefficient word ends; the final bit tells it whether to begin another cell record or halt.

Packing the records into one scalar. Take the N_* cells in the order specified by $\mathcal{J}$ and join their records end to end. Here N_* counts records, whereas $L = N_*(dm + M_{\text{Taylor}} b + 1)$ counts all their binary digits. Write $\beta_i \in \{0, 1\}$ for the i th digit of this complete sequence. The digits include location bits, coefficient bits, and continuation bits. Store them in the scalar

$$c = \sum_{i=1}^L 2\beta_i 4^{-i}, \quad L = N_*(dm + M_{\text{Taylor}} b + 1), \quad \beta_i \in \{0, 1\}. \quad (\text{S24})$$

Thus the i th base-four digit of c is $2\beta_i$, either zero or two. The scalar c stores the entire record list, rather than an individual Taylor coefficient. Every remaining suffix, when shifted to the start of the code, belongs to $[0, 2/3]$.

Reading bits and recovering the stored values. If the first bit is zero, the code is at most $\sum_{i \geq 2} 2 \cdot 4^{-i} = 1/6$. If it is one, the code is at least $1/2$. This gap lets the ReLU function

$$D(c) = 3 \operatorname{ReLU}(c - 1/6) - 3 \operatorname{ReLU}(c - 1/2)$$

return the first bit exactly. After reading it, the update $c^+ = 4c - 2D(c)$ removes its base-four digit and shifts the remaining digits forward. The same decoder can then read the next bit.

At the start of each word, reset an accumulator a to zero. Read its prescribed number of bits using the simultaneous updates

$$c^+ = 4c - 2D(c), \quad a^+ = (a + D(c))/2,$$

where both right-hand sides use the old code. For the word $\eta_1, \dots, \eta_\ell$ defined above, after ℓ steps the accumulator is $a = \sum_{t=1}^{\ell} \eta_t 2^{t-1} / 2^\ell = n/2^\ell$. Consequently, a location word gives $a = Ka_{j,i}/2^m = a_{j,i}$ directly. A coefficient word gives $a = n_b(c_{j,w})/2^b$, from which $2a - 1 = \tilde{c}_{j,w}$. The evaluator saves the decoded location and uses the coefficients in their prescribed term order. After processing these words, it reads and removes the continuation bit: one starts the next record and zero ends the list.

A one-cell example. Let $d = 1$, $k = 0$, and $m = b = 2$. A cell with lower corner $a_j = 1/4$ has location integer $Ka_j = 1$, stored as $(1, 0)$. Its only Taylor term is constant. If its coefficient is $1/2$, then $n_b(1/2) = 3$, stored as $(1, 1)$, and quantization recovers $1/2$ exactly. For a list containing only this cell, the continuation bit is zero. The complete five-bit record is

$$\left(\underbrace{1, 0}_{\text{location}}, \quad \underbrace{1, 1}_{\text{coefficient}}, \quad \underbrace{0}_{\text{end}} \right), \quad c = 0.20220_{(\text{base } 4)} = \frac{69}{128}.$$

The decoder returns these bits in order. The first word gives $a = 1/4$; after resetting the accumulator, the second gives $a = 3/4$ and hence $2a - 1 = 1/2$. The final zero instructs the evaluator to halt.

The code is one counted input-function bias for the standalone evaluator and one counted lookup coefficient per group for the spatial construction.

D.1.2 Shared square and multiplication recurrences

Lemma D.3 (Bounded square registers). *For $x \in [0, 1]$, initialize $(v_0, w_0, S_0) = (x, 1, x)$ and iterate*

$$\begin{aligned} v_{j+1} &= v_j/2 - \operatorname{ReLU}(v_j - w_j/2), & w_{j+1} &= w_j/4, \\ S_{j+1} &= S_j - v_j/2 + \operatorname{ReLU}(v_j - w_j/2). \end{aligned} \tag{S25}$$

For every $j \geq 0$, $0 < w_j = 4^{-j}$, $0 \leq v_j \leq w_j$, $0 \leq S_j \leq 1$, and

$$w_j(S_j - x^2) = w_j v_j - v_j^2, \quad 0 \leq S_j - x^2 \leq 4^{-j}/4.$$

The same update coefficients apply at every iteration.

Proof. The displayed invariant holds at initialization. Substitution into (S25), separately for $v_j \leq w_j/2$ and $v_j \geq w_j/2$, preserves it and gives $0 \leq v_{j+1} \leq w_{j+1}$. Consequently $S_j - x^2 = v_j(w_j - v_j)/w_j \in [0, w_j/4]$. Also $S_{j+1} = S_j - v_{j+1} \leq S_j \leq x \leq 1$, while the invariant implies $S_j \geq x^2 \geq 0$. This proves all claims. $\square$

The recurrence is a rescaled shared-coefficient form of the dyadic square construction in Yarotsky (2017). For $u, v \in [-1, 1]$, run two square channels for J updates and put

$$\mathbf{M}_J(u, v) = S_J(|u + v|/2) - S_J(|u - v|/2). \quad (\text{S26})$$

They share the register w . Absolute values are sums of two ReLUs, and $uv = ((u + v)/2)^2 - ((u - v)/2)^2$ gives

$$|\mathbf{M}_J(u, v) - uv| \leq 4^{-J}/2, \quad |\mathbf{M}_J(u, v)| \leq 1. \quad (\text{S27})$$

For $w = (i_1, \dots, i_\ell)$, start at $\tilde{c}_{j,w}$ and successively apply $\mathbf{M}_J$ with factors $x_{i_v} - x_{j,i_v}$, $v = 1, \dots, \ell$. Denote the final value by $\tilde{t}_{j,w}(x)$; for the empty list it equals $\tilde{c}_{j,w}$. Every intermediate value stays in $[-1, 1]$, since $x_i - x_{j,i} \in [-1, 1]$ for all inputs in the cube. Hence

$$|\tilde{t}_{j,w}(x) - c_{j,w}(x - x_j)^w| \leq 2^{1-b} + |w|4^{-J}/2.$$

Summing over the ordered term list gives

$$\left| \sum_w \tilde{t}_{j,w}(x) - P_j(x) \right| \leq M_{\text{Tay}} 2^{1-b} + V 4^{-J}/2 \leq C_{d,s} K^{-s}. \quad (\text{S28})$$

Accumulate $\tilde{t}_{j,w}/M_{\text{Tay}}$ in a register p and set $v_j = \Pi_1(M_{\text{Tay}}p)$. Every partial normalized sum has magnitude at most one. Clipping cannot increase the error relative to f_0/H , so

$$|H v_j(x) - f_0(x)| \leq C_{d,s} H K^{-s} \quad (\text{S29})$$

for interior points of cell j . This calculation separates the analytic Taylor remainder from the errors of the actual decoded coefficients and shared multiplication recurrence.

D.1.3 The evaluator as a finite-state program

The evaluator implements a fixed finite-state program. Its purpose is simple: for each encoded cell record, it decodes the cell location and Taylor coefficients, evaluates the corresponding local polynomial, keeps the contribution only if the input lies in that cell, adds the contribution to the output accumulator, and then either advances to the next record or halts. Thus one record is processed through the following five stages:

- (i) decode the cell location;
- (ii) decode and evaluate the Taylor terms;

- (iii) construct the cell-membership gate;
- (iv) add the gated value to the output accumulator;
- (v) read the continuation bit and either repeat or halt.

The point of the phase construction below is that the number of state coordinates needed to implement these stages is fixed: increasing the resolution changes only the number of repeated block applications.

State variables and phases. The state has two parts. Data coordinates store numerical values, whereas phase coordinates specify which elementary instruction is currently active. Write $e_1, \dots, e_P$ for the allocated phase coordinates, including an absorbing halt phase. On every reachable state exactly one phase coordinate equals one and all the others equal zero. The active phase determines both the data update and the next phase; all assignments read the old state and are updated simultaneously.

The vectors $x, a_j \in \mathbb{R}^d$ and the remaining scalar registers give $D_{\text{data}} = 2d + 15$ data coordinates in total.

Register	Role
x	Input vector.
a_j	Lower corner of the current cell.
c	Unread remainder of the encoded record list.
a	Accumulator for the binary word being decoded.
t	Running product for the current Taylor term.
p	Taylor sum scaled by $1/M_{\text{Tay}}$.
d_{face}	Minimum signed distance to the current cell's faces.
ψ	Cell gate controlling the local output contribution.
v	Clipped polynomial value for the current cell.
u	Accumulated output across processed cells.
β	Continuation bit: one to continue, zero to halt.
v_+, v_-	Intermediate states of the two square recurrences.
S_+, S_-	Square approximations whose difference gives a product.
w	Scale shared by the two square recurrences.
τ_{eval}	Timer for a counted repetition.

Initially, x contains the input, c contains the encoded record list, the first phase coordinate equals one, and every other data and phase coordinate is zero; in particular $u = 0$.

Primitive counted routines. All loops used by the evaluator have deterministic lengths known from d, s and the resolution parameters. A single three-phase counter implements a repetition of any prescribed length ℓ without allocating ℓ different phases. For $\ell \geq 1$, an initialization phase sets $\tau_{\text{eval}} = 2^{-(\ell-1)}$; an operation phase performs one update; and a check phase doubles the timer and returns to the operation phase unless the old timer equals one. The branch signal $\text{ReLU}(2\tau_{\text{eval}} - 1)$ is zero at all earlier checks and one at the last check. Hence the routine performs exactly ℓ operation/check pairs and takes $1 + 2\ell$ iterations. A zero-length repetition exits immediately after initialization. A word-reading

routine, denoted conceptually by $\text{READ}(\ell)$, adds one phase that first resets a and then uses the counted decoder for ℓ bits. It therefore uses four phases and $2 + 2\ell$ iterations. Saving the decoded location or using the decoded coefficient is handled by a separate one-step assignment.

A multiplication routine, denoted $\text{MULT}(J)$, initializes the two square channels, performs J counted square updates, and finally assigns $t = S_+ - S_-$. It uses five phases and $3 + 2J$ iterations. These are the only repeated arithmetic routines needed below.

Stage (i): decoding the cell location. Reset the polynomial accumulator to $p = 0$. For each coordinate $i = 1, \dots, d$, run $\text{READ}(m)$ and save the decoded value in $a_{j,i}$. The coordinate list ends with one explicit identity step. Thus the location stage uses $5d + 1$ phase coordinates and $d(3 + 2m) + 1$ iterations after the initial reset of p .

Stage (ii): evaluating the Taylor polynomial. Process the Taylor terms in the fixed order of $\mathcal{W}_k$. For a term indexed by w , run $\text{READ}(b)$, set $t = 2a - 1$, and then apply $\text{MULT}(J)$ successively to the $|w|$ factors $x_i - a_{j,i} - h/2$ specified by w . For the empty list no multiplication is needed. Each factor list ends with one identity step; then add t/M_{Taylor} to p . After the last Taylor term, the term list itself ends with one additional identity step. The running term and every partial polynomial accumulator remain in $[-1, 1]$.

Consequently the Taylor stage uses $7M_{\text{Taylor}} + 5V + 1$ phase coordinates and

$$M_{\text{Taylor}}(5 + 2b) + V(3 + 2J) + 1$$

iterations. The factor-list identity contributes the extra fixed step in the $5 + 2b$ cost for each Taylor term, whereas the final $+1$ is the identity step terminating the whole term list.

Stage (iii): constructing the cell gate. First set $v = \Pi_1(M_{\text{Taylor}}p)$. In a separate phase, initialize $d_{\text{face}} = 1$. Scan the d lower faces and then the d upper faces using

$$d_{\text{face}}^+ = d_{\text{face}} - \text{ReLU}(d_{\text{face}} - \xi), \quad \xi = x_i - a_{j,i} \text{ or } a_{j,i} + h - x_i.$$

Each update replaces the current value by the minimum of that value and the signed distance to one face. The lower and upper faces form one ordered list of $2d$ faces, which ends with one identity step. The final d_{face} is positive inside the cell and nonpositive outside its interior.

Initialize $\psi = \text{ReLU}(d_{\text{face}})$ and perform G counted updates

$$\psi^+ = 2\psi - \text{ReLU}(2\psi - 1).$$

They yield

$$\psi_j(x) = \min\{1, 2^G \text{ReLU}(d_{\text{face},j}(x))\}.$$

Hence $\psi_j(x) = 0$ outside the cell interior and $\psi_j(x) = 1$ whenever x is at least 2^{-G} from every face. This stage uses $2d + 7$ phase coordinates and $2d + 5 + 2G$ iterations.

Stage (iv): accumulating the selected contribution. Update

$$u^+ = u + \text{ReLU}(v + \psi) - \text{ReLU}(v - \psi) - \psi = u + \Pi_\psi(v).$$

The increment is zero when $\psi = 0$ and equals v when $\psi = 1$. Distinct cells have disjoint localization supports, so every partial output sum remains in $[-1, 1]$. This stage uses one phase and one iteration.

Stage (v): advancing or halting. Read the continuation bit $\beta = D(c)$ and remove it simultaneously through $c^+ = 4c - 2D(c)$. A separate marker-check phase reads the saved bit: $\beta = 1$ returns to the start of the record routine, retaining u and the unread suffix code, whereas $\beta = 0$ enters the absorbing halt phase. For valid record lists the marker is exactly zero or one. The halt phase has zero residual, so all later block applications preserve the terminal state. Together, marker read and marker check use two phases and two iterations, and the absorbing halt state uses one additional phase.

The bounds above, Lemma D.3, and the decoder bounds keep every data register within $[-2, 2]$ throughout the program. The next subsection shows that this entire finite-state program is realized by one shared residual ReLU block.

D.1.4 Compiling the finite-state program into one residual ReLU block

The preceding evaluator is a finite-state program whose elementary instructions have a uniform algebraic form. The following lemma isolates the compilation step used to turn such a program into one shared residual ReLU block.

Lemma D.4 (Finite-state program compilation). *Consider a finite-state program with D data coordinates and P one-hot phase coordinates. Suppose that, on every reachable state, the active phase a updates each data coordinate according to*

$$z_i^+ = z_i + A_{ia}(z) + c_{ia,1} \text{ReLU}\{\ell_{ia,1}(z)\} + c_{ia,2} \text{ReLU}\{\ell_{ia,2}(z)\},$$

where A_{ia} and $\ell_{ia,\nu}$ are affine functions and unused hinges are set to zero. Suppose also that every phase transition is a default move, possibly redirected by one branch test whose ReLU output is exactly zero or one on the reachable branch states.

Assume that, for some $B \geq 1$, every ungated affine preactivation has absolute value at most $2B$ on the reachable data box and every affine and hinge output coefficient has absolute value at most B . Then the program is realized exactly on its reachable states by a single residual ReLU block

$$\Phi(y) = y + W_2 \text{ReLU}(W_1 y + b_1) + b_2, \quad b_2 = 0,$$

on $D + P$ state coordinates, with hidden width at most

$$P(4D + 2),$$

and every block coefficient bounded in magnitude by $3B$. If one phase has zero residual and transitions to itself, that phase is an absorbing terminal state of the same block.

Proof. Let e_a denote the one-hot indicator of phase a and set $\Lambda = 2B$. For a feature belonging to phase a , add $\Lambda(e_a - 1)$ to its preactivation. If phase a is active, this addition is zero. If it is inactive, the preactivation is at most $2B - \Lambda = 0$, so the feature vanishes.

For one data coordinate in phase a , represent the signed affine residual $A = A_{ia}(z)$ by

$$\text{ReLU}\{A + \Lambda(e_a - 1)\} - \text{ReLU}\{-A + \Lambda(e_a - 1)\}.$$

This equals A in the active phase and zero otherwise. Gate the at most two hinge features in the same way. Hence four hidden units per data coordinate suffice for every phase, with zero-padding when fewer are needed.

It remains to update the phase coordinates. For phase a , one feature $\text{ReLU}(e_a)$ implements the default move and can also supply a phase-specific constant residual. A second gated feature implements the binary redirect when the branch is taken: its output cancels the default successor and activates the alternative successor. Because the branch signal is exactly zero or one on reachable branch states, these two units preserve the one-hot phase encoding. Thus each phase uses at most $4D + 2$ hidden units, giving total width $P(4D + 2)$.

Concatenating all phase-specific features yields one affine–ReLU–affine residual block. The gating coefficient has magnitude $\Lambda = 2B$; combining it with an original affine coefficient or bias of magnitude at most B gives the bound $3B$. Taking zero residual and a self-transition in the designated halt phase makes that state absorbing. $\square$

Application to the evaluator. Every instruction in the program of D.1.3 satisfies the lemma’s two-hinge condition. The decoder

$$D(c) = 3 \text{ReLU}(c - 1/6) - 3 \text{ReLU}(c - 1/2)$$

uses two hinges; absolute value and clipping use two; a face-minimum update uses one; the gate recurrence uses one; and the gated output increment uses two. For example, in a face-comparison phase a , the single gated hinge

$$d_{\text{face}}^+ = d_{\text{face}} - \text{ReLU}\{d_{\text{face}} - \xi + 2B_{\text{eval}}(e_a - 1)\}$$

takes the minimum with the current face distance when $e_a = 1$ and leaves d_{face} unchanged in every other phase. The branch tests are zero, $2\tau_{\text{eval}} - 1$, or β . On every reachable check state their ReLU outputs are exactly zero or one, so the phase transitions meet the binary-redirect hypothesis.

By the register bounds from D.1.3, Lemma D.3, and the decoder bounds, every evaluator data coordinate stays in $[-2, 2]$. With $B = B_{\text{eval}}$, all ungated affine preactivations are bounded in absolute value by $2B_{\text{eval}}$, and all affine and hinge output coefficients are bounded by B_{eval} . Lemma D.4 therefore gives one shared residual ReLU block with coefficient bound $3B_{\text{eval}}$.

Resource bookkeeping. The phase and iteration counts established in the previous subsection are summarized below. The identity steps terminating the coordinate, factor, term, and face lists are included explicitly.

Program group	Phase coordinates	Iterations per record
Reset polynomial accumulator	1	1
Coordinate words, saves, and final identity	$5d + 1$	$d(3 + 2m) + 1$
Terms, additions, and list identities	$7M_{\text{Tag}} + 5V + 1$	$M_{\text{Tag}}(5 + 2b) + V(3 + 2J) + 1$
Clip, face comparisons, and gate	$2d + 7$	$2d + 5 + 2G$
Output accumulation	1	1
Marker read and check	2	2
Absorbing halt	1	0
Total	$7d + 7M_{\text{Tag}} + 5V + 14$	τ_m

Therefore

$$P = 7d + 7M_{\text{Tag}} + 5V + 14.$$

Adding the $D_{\text{data}} = 2d + 15$ data coordinates gives

$$q_0 = D_{\text{data}} + P = 9d + 7M_{\text{Tag}} + 5V + 29,$$

which is (S20). The hidden width is

$$r_0 = P(4D_{\text{data}} + 2) = (7d + 7M_{\text{Tag}} + 5V + 14)(8d + 62),$$

again exactly as in (S20). Summing the iteration column gives

$$\tau_m = d(5 + 2m) + M_{\text{Tag}}(5 + 2b) + V(3 + 2J) + 2G + 11,$$

which is (S21). Thus the architecture size is fixed once d and s are fixed, while increasing the resolution affects the precision constants and the number of repeated applications, not the number of distinct block parameters.

An all-zero evaluator phase vector leaves its initially bounded data unchanged until the selector activates the first evaluator phase.

D.1.5 Correctness, error, and precision

Proof of Lemma D.2. Induct first over the bits of each word, then over its factor and term lists, and finally over the cell records. The decoder removes exactly the prescribed word and preserves its suffix code. The square invariant bounds every arithmetic operation. The separate counted-loop initialization and checks give their specified lengths; the marker returns to the same record program or enters halt. The bounds established above justify all inactive gates at each step. Thus the compiled block executes the complete program and returns

$$u(x) = \sum_{j \in \mathcal{J}} \Pi_{\psi_j(x)}(v_j(x))$$

at time $N_*\tau_m$. At most one summand is nonzero. In a listed cell whose face distances are at least 2^{-G} , its gate is one and (S29) applies. The halt state is fixed by the block, so the entire terminal state persists under all additional iterations.

For the full grid, write $g_{\text{all}} = g_{\mathcal{J}}$ when $\mathcal{J}$ lists every cell. Let S_G be the union, inside the cube, of the open strips of half-width $\delta = 2^{-G}$ around all grid hyperplanes, including the boundary. A union bound gives

$$\mu(S_G) \leq 2d(K+1)\delta \leq dK^{-2s}.$$

Indeed $G = (2(k+1) + 1)m + 2$ and $s \leq k+1$. Off S_G the containing cell has gate one, giving pointwise error at most $C_{d,s}HK^{-s}$. On S_G , both the returned value and target are bounded by H . Therefore

$$\|g_{\text{all}} - f_0\|_2^2 \leq C_{d,s}H^2K^{-2s}. \quad (\text{S30})$$

The input function uses input copies, the code, and phase constants of magnitude at most one; the output-function coefficient is H . This proves the lemma. $\square$

The code in (S24) is dyadic with denominator 2^{2L} and uses at most $2L = O_{d,s}(N_*m)$ binary fractional bits. The mesh and timer constants require $O_s(m)$ bits. Coefficient magnitudes remain uniformly bounded, while the required precision grows with the resolution.

D.2 Spatial selection and proof of Theorem D.1

The evaluator above scans every record in its input list. We now trade additional width for fewer scanned records. The fine cells are partitioned into coarse groups, a bounded-weight front end identifies the group containing x and loads only that group's encoded Taylor table, and a one-block handoff starts the evaluator. The group codes are target-specific lookup weights and are counted among the fitted coefficients. The final width choice makes the number of groups comparable to $\min\{r, K^d\}$, producing the runtime bound in (S19).

D.2.1 A bounded-weight spatial selector

The purpose of the front end is to use width to avoid scanning all K^d fine-cell records. Choose

$$u_{\text{grp}} \in \{0, \dots, m\}, \quad A = 2^{u_{\text{grp}}}, \quad N_{\text{grp}} = A^d,$$

and partition the fine grid into N_{grp} coarse cubes. Each coarse cube contains $2^{d(m-u_{\text{grp}})}$ fine cells. For $a = (a_1, \dots, a_d) \in \{0, \dots, A-1\}^d$, write

$$j(a) = \sum_{i=1}^d A^{i-1}a_i$$

for its group index and let $\gamma_a \in [0, 2/3]$ be the scalar code of the fine-cell records in that group.

The construction below has one objective: given x , load γ_a for the unique coarse cube containing x , and then emit a single start pulse for the evaluator. The following lemma records this property and the required resources.

Lemma D.5 (Bounded-weight spatial selector). *Set*

$$L_m = G + dm, \quad C_* = 2^{L_m}, \quad \ell = du_{\text{grp}}.$$

There is one residual ReLU front end with

$$q_{\text{front}} = d + 10, \quad r_{\text{front}} = 10dA + 10d + 15N_{\text{grp}} + 60, \quad M_{\text{front}} = 22,$$

such that the following holds. If x is at distance at least C_*^{-1} from every coarse-grid face, then after exactly

$$L_m + du_{\text{grp}} + 2$$

iterations the front end is in an absorbing halt state with

$$c_{\text{front}} = \gamma_a, \quad \eta_{\text{start}} = 1,$$

where a is the coarse-cell index of x . Before that lookup iteration, both payload coordinates c_{front} and η_{start} and their residual increments are zero. Every coefficient is bounded in magnitude by 22.

Proof. The proof has four steps: generate a common scale, decode the coarse-cell coordinates, convert the normalized address to an integer address, and perform a one-step table lookup.

Step 1: generate a common scale with bounded coefficients. The front end uses data registers

$$U_1, \dots, U_d, C, I, \tau_{\text{front}}, c_{\text{front}}, \eta_{\text{start}}$$

and five phase coordinates for warm-up, address, scaling, lookup, and halt. Write e_w and e_a for the warm-up and address phase coordinates, respectively. The phase encoding is scaled: on every reachable state exactly one phase coordinate equals the current scale C and all the others are zero.

Initialize

$$U_i = x_i, \quad C = 1, \quad \tau_{\text{front}} = 2^{-(L_m-1)},$$

with the warm-up phase equal to one and all other front-end coordinates zero. During warm-up use

$$\begin{aligned} U_i^+ &= 2U_i, & C^+ &= 2C, & \tau_{\text{front}}^+ &= 4\tau_{\text{front}}, \\ e_w^+ &= 2e_w - 2F, & e_a^+ &= e_a + 2F, & F &= \text{ReLU}(2\tau_{\text{front}} - C). \end{aligned}$$

Before warm-up update j , $C = 2^j$ and $\tau_{\text{front}}/C = 2^{j-(L_m-1)}$. Thus $F = 0$ before the last update and $F = C$ on the last one. The scaled one-hot invariant is therefore preserved, and after exactly L_m warm-up iterations,

$$U_i = C_* x_i, \quad C = C_*, \quad \tau_{\text{front}} = 2C_*,$$

with the address phase active at scale C_* . Repeated doubling creates the large spatial scale C_* without any large trainable coefficient.

Step 2: decode the coarse-cell coordinates. For one coordinate define

$$H_i = \sum_{j=1}^A \left[\text{ReLU}(U_i - jC/A + 1) - \text{ReLU}(U_i - jC/A) \right].$$

Suppose x_i lies in coarse interval a_i and is at least C_*^{-1} from both of its faces. Since $U_i = C_* x_i$ and $C = C_*$, every summand with $j \leq a_i$ equals one and every summand with $j > a_i$ equals zero; the endpoint term $j = A$ is also zero. Hence

$$H_i = a_i.$$

One address iteration therefore writes

$$I = \sum_{i=1}^d \frac{A^{i-1}}{N_{\text{grp}}} H_i = \frac{j(a)}{N_{\text{grp}}}. \quad (\text{S31})$$

All coefficients A^{i-1}/N_{grp} have magnitude at most one. **Step 3: convert to the integer address.** Because $N_{\text{grp}} = A^d = 2^{du_{\text{grp}}} = 2^\ell$, the desired integer address is

$$j(a) = 2^\ell I.$$

If $\ell \geq 1$, the address phase initializes $\tau_{\text{front}} = 2^{-(\ell-1)}C$ and activates the scaling phase. Perform ℓ updates

$$I^+ = 2I, \quad \tau_{\text{front}}^+ = 2\tau_{\text{front}},$$

and exit when the old timer equals C , using the branch $\text{ReLU}(2\tau_{\text{front}} - C)$. If $\ell = 0$, skip this phase. In either case, lookup starts with

$$I = j(a), \quad \tau_{\text{front}} = 2C_*.$$

Step 4: lookup the encoded group. For an integer j define the triangular hat

$$\chi_j(I) = \text{ReLU}(I - j + 1) - 2\text{ReLU}(I - j) + \text{ReLU}(I - j - 1).$$

At integer arguments,

$$\chi_j(I) = \mathbf{1}\{I = j\}.$$

The lookup iteration therefore sets

$$c_{\text{front}} = \sum_a \gamma_a \chi_{j(a)}(I) = \gamma_a, \quad \eta_{\text{start}} = C/C_* = 1. \quad (\text{S32})$$

Each shift $j(a)$ is implemented as $(j(a)/C_*)C$, whose coefficient has magnitude at most one. Lookup then enters the scaled halt phase, so the front-end residual is zero at all later iterations.

It remains to compile these five phases into one residual ReLU block. A front-end preactivation $A(z)$ is gated by adding $16(e - C)$, where the active phase coordinate satisfies $e = C$. On every valid trajectory all data coordinates are bounded by a fixed multiple of C , and every inactive preactivation is at most $16C$ before gating. The relevant branch outputs are always either zero or C . Hence the scaled one-hot invariant is preserved and all inactive features vanish.

Allocate, for each phase, $2(d + 5)$ signed-linear units, $2dA$ address hinges, $3N_{\text{grp}}$ lookup hinges, and two clock/branch units, with inactive phase-specific output columns set to zero.

Summing over the five phases gives

$$q_{\text{front}} = d + 10, \quad r_{\text{front}} = 10dA + 10d + 15N_{\text{grp}} + 60.$$

All coefficients are bounded by 22. The timeline is

$$L_m \text{ warm-up steps} + 1 \text{ address step} + du_{\text{grp}} \text{ scaling steps} + 1 \text{ lookup step},$$

which proves the claimed finishing time. Before lookup, both payload coordinates and their residual increments are zero by construction. $\square$

D.2.2 One-block handoff and resource bounds

Lemma D.5 produces exactly the two quantities needed to start the evaluator: the selected table code γ_a and a one-time start pulse. We now fuse the selector and evaluator without composing their hidden layers.

Initialize the evaluator input registers to x and all its remaining data registers and phase coordinates to zero. It therefore remains dormant until the selector loads the code and activates its first phase. Let Φ_{ev} and Φ_{front} denote the two compiled residual maps. Let L copy the front-end code increment into the evaluator code coordinate and the start-pulse increment into its first phase coordinate, and be zero elsewhere. Use the single combined update

$$(z, y)^+ = (\Phi_{\text{ev}}(z) + L\{\Phi_{\text{front}}(y) - y\}, \Phi_{\text{front}}(y)). \quad (\text{S33})$$

This is still one residual ReLU block: concatenate the two hidden layers and copy the indicated front-end output rows to the corresponding evaluator outputs. No hidden-layer composition and no multiplication of computed states is used.

The handoff occurs only once. Its three regimes are

front-end stage	$\Phi_{\text{front}}(y) - y$	evaluator state
before lookup	0 on payload rows	dormant
lookup	$(\gamma_a, 1)$ on payload rows	loaded and started
after lookup	0	runs its own recurrence.

Thus the evaluator cannot start early and cannot be loaded twice. Selection takes $L_m + du_{\text{grp}} + 2$ steps by Lemma D.5. The selected group contains

$$2^{d(m-u_{\text{grp}})} = \frac{K^d}{N_{\text{grp}}}$$

fine-cell records, and D.1 requires τ_m iterations per record. Therefore the exact finishing time is

$$S_{m,u_{\text{grp}}} = L_m + du_{\text{grp}} + 2 + 2^{d(m-u_{\text{grp}})}\tau_m. \quad (\text{S34})$$

The combined dimensions and a sufficient coefficient bound are

$$\begin{aligned} q_* &= q_0 + d + 10, \\ r(u_{\text{grp}}) &= r_0 + 10d 2^{u_{\text{grp}}} + 10d + 15 2^{du_{\text{grp}}} + 60, \\ B_* &= 2(3B_{\text{eval}} + 22). \end{aligned} \tag{S35}$$

All parameter slots, including zeros, are included in the dense fitted class. Extra coordinates and hidden units can be zero-padded without changing the output or increasing the coefficient bound.

D.2.3 Finite-resolution error and width selection

Proof of Theorem D.1. More width permits more coarse groups and hence fewer records to scan. The total runtime is the selection time plus the number of records in the selected group multiplied by the evaluation time per record.

Fix the target and $K = 2^m$. Put

$$b_0 = r_0 + 10d + 60, \quad b_1 = 10d + 15, \quad R(u_{\text{grp}}) = b_0 + b_1 2^{du_{\text{grp}}}.$$

For $d \geq 1$, $2^{u_{\text{grp}}} \leq 2^{du_{\text{grp}}}$ and $r(u_{\text{grp}}) \leq R(u_{\text{grp}})$. The following fixed thresholds suffice:

$$q \geq q_* = q_0 + d + 10, \quad r \geq r_* = b_0 + b_1, \quad M_{\text{par}} \geq M_* := \max\{1, H, B_*\}. \tag{S36}$$

Choose the largest $u_{\text{grp}} \in \{0, \dots, m\}$ with $R(u_{\text{grp}}) \leq r$. If $u_{\text{grp}} = m$, each group has one cell. Otherwise $r < R(u_{\text{grp}} + 1) \leq (b_0 + 2^d b_1) 2^{du_{\text{grp}}}$. In either case the group count is comparable to $\min\{r, K^d\}$, with constants depending only on d, s . Since $\tau_m \leq C_{d,s} m$, (S34) gives

$$S_{m, u_{\text{grp}}} \leq C_{d,s} m \left(1 + \frac{K^d}{r} \right) \leq C_{d,s} \left\{ \log(eK) + \frac{K^d}{r} \log(eK) \right\}. \tag{S37}$$

Let S_G be the fine-grid strip set from the evaluator proof. Every coarse face is a fine-grid hyperplane and $C_*^{-1} \leq 2^{-G}$. Hence outside S_G the input has the coarse-face margins needed for exact selection, and the loaded group contains its fine cell. The fused block then returns the correct local value at time $S_{m, u_{\text{grp}}}$ and preserves it at all later iterations. On S_G , the final clipping always bounds the error by $2H$, even for a nonvalid interpolated code. Thus

$$\|g_T - f_0\|_2^2 \leq C_{d,s} H^2 K^{-2s}, \quad T \geq S_{m, u_{\text{grp}}}. \tag{S38}$$

Take $q_0^F = q_*$, $r_0^F = r_*$, and $M_0^F = M_*$. The input function is affine, the final output function is Hu followed by clipping, and (S33) is one shared residual block. All entries are bounded by M_* and are counted in $B_L^F(q, r)$. The group count, encoded tables, and precision constants are chosen using f_0, K, q, r , before the eventual iteration count is specified. Both the front end and evaluator halt on valid inputs; clipping handles the exceptional set at every time. Enlarging the common constant C to include the fixed H^2 proves (S18)–(S19) for all admissible T with one parameter choice. $\square$

The inversion from an iteration budget to a resolution remains in the proof of Lemma B.1

in Section B.2.

E Transformer approximation construction

We implement the shared Taylor evaluator of Appendix D in the post-LN Transformer of Section 3.2. Three additional steps are needed. The input is distributed across tokens, so attention must gather it and locate its cell group. Layer normalization changes the state scale, so we encode the evaluator’s values relative to a common reference coordinate. Finally, the output function is affine, so we correct the computed value before reading it from the normalized state.

Theorem E.1 (Finite-resolution construction for looped post-LN Transformers). *Fix $d \geq 1$, $s > 0$ and $H > 0$, the token partition of Section 3.2, and $\varepsilon_{\text{LN}} > 0$. Let μ be uniform probability measure on $[0, 1]^d$. There exist a constant $C > 0$ and thresholds $q_0^{\text{Tr}}, r_0^{\text{Tr}}, M_0^{\text{Tr}} > 0$, depending only on these fixed quantities, with the following property. For every $M_{\text{par}} \geq M_0^{\text{Tr}}$, integers $q \geq q_0^{\text{Tr}}$, $r \geq r_0^{\text{Tr}}$, dyadic resolution $K = 2^m$ with $m \geq 1$, and $f_0 \in \mathcal{H}_d^s(H)$, there is one admissible input-function, update-block, and output-function parameter choice for the post-LN Transformer whose looped candidates $g_T \in \mathcal{F}_{L, \text{Tr}}(q, r, T)$ satisfy*

$$\|g_T - f_0\|_{L^2(\mu)}^2 \leq CK^{-2s} \quad (\text{S39})$$

for every integer

$$T \geq C\{\log^2(eK) + (K^d/r)\log(eK)\}. \quad (\text{S40})$$

The parameter choice may depend on f_0, K, q, r , but is the same for all such T . Every scalar coefficient belongs to $[-M_{\text{par}}, M_{\text{par}}]$, and all endpoint and block coordinates are counted in $B_L^{\text{Tr}}(q, r)$, which is independent of T .

Proof roadmap. The proof implements the Taylor evaluator from Appendix D inside a post-LN Transformer in three stages. Section E.1 uses paired homogeneous coordinates to represent the evaluator’s registers: layer normalization changes their common scale while preserving the encoded native computation. Section E.2 uses attention to aggregate the input and find its coarse-cell address, then loads the corresponding Taylor table and runs the shared evaluator. The available hidden width permits $N_{\text{grp}} \asymp \min\{r, K^d\}$ groups, so table evaluation takes $O((1 + K^d/r)\log(eK))$ iterations. Section E.3 corrects the remaining normalization scale so that an affine output function recovers the target value. Routing and terminal correction together cost $O(\log^2(eK))$, while Taylor-table evaluation costs $O((1 + K^d/r)\log(eK))$. Outside small routing and cell-boundary exceptional sets, Taylor evaluation and terminal correction have $O(K^{-s})$ error; clipping controls the exceptional sets. Section E.4 combines these ingredients, checks the parameter bounds, and proves (S39)–(S40) for every T beyond the finishing time.

E.1 Homogeneous simulation through layer normalization

We distinguish the values used by the evaluator from their scaled representation in a token. For a *native state* $z \in \mathbb{R}^p$, set $q_{\text{rep}} = 2(p + 1)$. For $q \geq q_{\text{rep}}$, let $L_q = \lfloor q/q_{\text{rep}} \rfloor$ and define $\mathcal{I}_q(z)$ by repeating $(z, 1, -z, -1)$ exactly L_q times and padding with zeros to dimension

q . For $\alpha > 0$, we call $\alpha\mathcal{I}_q(z)$ its *physical token state*. Three reference quantities have different roles. The fixed +1 entry of the paired encoding has physical value α ; it supplies homogeneous affine biases. The native register C belongs to the spatial selector and may grow during initialization. The two reference *tokens*, whose native key coordinates are +1 and -1, generate the attention comparison signal. Neither C nor a reference token is the homogeneous reference coordinate.

E.1.1 Exact simulation of a native residual update

Let θ collect all attention, FFN, and layer-normalization parameters in the block

$$Y = \text{LN}_1\{S + \text{Attn}(S)\}, \quad \Phi^{\text{postLN}}(S) = \text{LN}_2\{Y + \text{FFN}(Y)\}. \quad (\text{S41})$$

Both normalizations have fixed stabilization $\varepsilon_{\text{LN}} > 0$ and trainable gains and biases. The block has

$$P_\theta^{\text{postLN}}(q, r) = 4q^2 + (2q + 1)r + 9q \quad (\text{S42})$$

dense parameters. The input function with local inputs and the output function are counted in (S47) below. Throughout the construction the parameter bound M_{par} is fixed above a threshold depending only on the fixed model quantities, including $\sqrt{1 + \varepsilon_{\text{LN}}}$.

Lemma E.2 (Homogeneous state representation). *Let a residual function on $\mathbb{R}^p$ have the form*

$$\Phi^{\text{res}}(z) = z + V \text{ReLU}(Uz + b). \quad (\text{S43})$$

For $q \geq q_{\text{rep}}$, the paired encoding defined above satisfies

$$\lambda_q = \frac{2L_q}{q}, \quad \frac{1}{q_{\text{rep}}} \leq \lambda_q \leq \frac{2}{q_{\text{rep}}}.$$

There is a post-LN block with zero attention output and the same FFN hidden width as Φ^{res} such that

$$S = \alpha\mathcal{I}_q(z), \quad \alpha > 0 \quad \implies \quad \Phi^{\text{postLN}}(S) = \alpha'\mathcal{I}_q(\Phi^{\text{res}}(z)), \quad \alpha' > 0. \quad (\text{S44})$$

The representation does not increase the magnitudes of the copied FFN coefficients. If the entries of U, V, b have magnitude at most M , the block is admissible whenever $M_{\text{par}} \geq \max\{M, \sqrt{1 + \varepsilon_{\text{LN}}}\}$.

Proof. Set both normalization gains to $\sqrt{1 + \varepsilon_{\text{LN}}}$ and their biases to zero. The coordinates of $\mathcal{I}_q(z)$ have mean zero, so the first normalization produces $\tilde{\alpha}\mathcal{I}_q(z)$ for some $\tilde{\alpha} > 0$. Each hidden unit reads Uz from the first copy and represents its bias b as a coefficient on that copy's reference coordinate. Its preactivation is therefore $\tilde{\alpha}(Uz + b)$. Positive homogeneity of ReLU gives the desired residual increment on every positive state copy and its negative on every negative copy. The reference and padding coordinates receive zero increments. The intermediate state is exactly $\tilde{\alpha}\mathcal{I}_q(\Phi^{\text{res}}(z))$, which again has mean zero. The second normalization changes only its positive common scale. All actual FFN biases are zero, and copying the output columns does not change their coefficient magnitudes. $\square$

Ratios to a reference coordinate identify the represented state in the proof. The network output will instead be obtained by the affine function constructed below. Repeating the paired representation keeps λ_q between fixed positive constants as q increases; all repeated matrix entries are included in the dense parameter count. Throughout the homogeneous construction, every constant in an active preactivation is represented as a multiple of the current reference coordinate; consequently, each clipping and threshold operation is unchanged by the positive common scale.

E.1.2 Scale recurrence and homogeneous instructions

For a represented token $\alpha\mathcal{I}_q(z)$, set $d_\alpha = \alpha^{-2}$ and $\rho_{\text{LN}} = \varepsilon_{\text{LN}}/(1 + \varepsilon_{\text{LN}})$. Since the coordinate mean is zero and the mean square is $\alpha^2\lambda_q(\|z\|_2^2 + 1)$, one normalization gives exactly

$$d'_\alpha = \frac{\lambda_q(\|z\|_2^2 + 1) + \varepsilon_{\text{LN}}d_\alpha}{1 + \varepsilon_{\text{LN}}}. \quad (\text{S45})$$

When attention is active, the first normalization uses the native state *after* its attention increment. The second uses the state after the row-wise FFN. The projections below write paired increments and leave the homogeneous reference unchanged, so both normalized states retain the same representation. Thus the FFN instructions operate on native values even when physical token scales differ.

The native instructions below use simultaneous assignments: every right-hand side and transition test reads the old state. For a phase e whose active value is $C = 1$, a signed affine residual $A(z)$ uses

$$\text{ReLU}\{A(z) + \Lambda(e - C)\} - \text{ReLU}\{-A(z) + \Lambda(e - C)\},$$

and a hinge $\text{ReLU}(B(z))$ uses $\text{ReLU}\{B(z) + \Lambda(e - C)\}$. Active instructions are exact; inactive ones vanish when $|A|, |B| < \Lambda C$. Every numerical constant in these forms multiplies the homogeneous reference coordinate, not the selector register C . The selector's own phases instead have active value C , as in Section D.2. These gates use only one ReLU hidden layer.

E.2 Attention routing and table evaluation

The homogeneous representation lets us describe the computation in native values. We first gather the input in a controller token, then use attention comparisons to determine its coarse-cell address and load the corresponding Taylor code. The attention and FFN matrices remain fixed throughout. In the routing formulas below, input coordinates are numbered from 0 to $d - 1$: x_i denotes the $(i + 1)$ st coordinate of the original input vector. The local and aggregate input registers use the same convention; coordinate i corresponds to register U_{i+1} of the selector.

E.2.1 Token protocol, native registers, and parameter counts

Use the fixed partition $I_1, \dots, I_M$ of the input coordinates, with M data tokens, one controller token, and two reference tokens. Thus

$$N = M + 3. \quad (\text{S46})$$

The data-token input function at position j reads only x_{I_j} , and the other three input functions are constant. Counting all affine endpoints, projections, biases, and normalization gains gives

$$\begin{aligned} P_{\text{end}} &= q(d + 2N) + 1, \\ P_{\text{block}} &= 4q^2 + (2q + 1)r + 9q, \\ B_L^{\text{Tr}} &= P_{\text{end}} + P_{\text{block}}, \quad B_U^{\text{Tr}} = P_{\text{end}} + TP_{\text{block}}. \end{aligned} \quad (\text{S47})$$

These are the full dense counts, including entries assigned special values by the construction.

Use the q_0 native coordinates of the evaluator in Section D.1. The attention-addressed front end below uses $d + 7$ further coordinates; reserve three zero coordinates so their total allocation is $q_* = q_0 + d + 10$, as in Appendix D. This front end receives its address from attention, loads the evaluator directly, and clears its four clocks after lookup. Its realization and count are specified below. Reserve the 15 data and 18 phase coordinates for the terminal correction in Section E.3. The additional attention and routing coordinates are listed below. Here $x^{\text{loc}}, x^{\text{agg}} \in \mathbb{R}^d$; each indexed routing coordinate is allocated for $i = 0, \dots, d - 1$.

Coordinate	Role
k	Key coordinate; the two reference tokens use $+1$ and -1 .
a_{att}	Attention comparison signal read by the FFN.
e_{boot}	Phase coordinate for input initialization.
x^{loc}	Local input entries, with zeros outside the token partition.
x^{agg}	Input entries gathered by attention.
u_i	Centered binary remainder for input coordinate i .
w_i	Saturating comparator used to determine the next bit.
b_i	Binary prefix accumulated for coordinate i .
p_i	Weight of the next binary digit.
$\tau_{\text{cmp},i}$	Timer for comparisons within one bit decision.
$\tau_{\text{level},i}$	Timer for the bits extracted from coordinate i .
$e_{C,i}, e_{M,i}, e_{H,i}$	Phase coordinates for comparison, commit, and handoff.

The total native dimension and encoding threshold are

$$p_{\text{nat}} = q_* + 11d + 36, \quad q_{\text{hom}} = 2(p_{\text{nat}} + 1). \quad (\text{S48})$$

Apply Lemma E.2 with $p = p_{\text{nat}}$ at any $q \geq q_{\text{hom}}$, so $1/q_{\text{hom}} \leq \lambda_q \leq 2/q_{\text{hom}}$. The constant q_{hom} is independent of q, r, K . All tokens start with physical scale one and native $C = 1$. Data tokens contain their local input coordinates; the reference tokens have $k = 1$ and $k = -1$. The controller contains $e_{\text{boot}} = 1$, the front-end timer prescribed in Appendix D, and the terminal-correction code prescribed below. All other native coordinates are zero. These initializations obey the local-input protocol and are affine in each token's allowed

inputs.

E.2.2 Fixed attention projections and input aggregation

For token j with physical state $\alpha_j \mathcal{I}_q(z_j)$, choose projections whose query, key, and nonzero value channels are

$$\begin{aligned} Q_j &= \alpha_j \left(\sum_i u_{j,i} \right) \mathbf{1}_q, & K_j &= \frac{\alpha_j k_j}{\sqrt{q}} \mathbf{1}_q, \\ V_j &= \alpha_j (k_j, Nx_{j,0}^{\text{loc}}, \dots, Nx_{j,d-1}^{\text{loc}}, 0, \dots, 0). \end{aligned} \quad (\text{S49})$$

Each projection reads the first positive copy. The output projection writes value channel zero into a_{att} and the next d channels into x^{agg} , with opposite signs in negative copies. All biases are zero; every projection entry has magnitude at most $\max\{N, 1\}$. In particular, $Q_j K_l^\top / \sqrt{q} = \alpha_j \alpha_l (\sum_i u_{j,i}) k_l$, including the prescribed head scaling.

At boot all queries vanish. Uniform attention therefore gathers the complete input x into every token's aggregate registers, while the reference values cancel in the signal channel. The first normalization changes only the common scale of each row. The controller's boot instruction copies the aggregate into the Taylor input and selector registers U_i . At every FFN application, in every token, unconditional signed-linear updates clear $x^{\text{loc}}, x^{\text{agg}}, a_{\text{att}}$. All assignments read the old state, so these clears do not erase the input before it is copied or the signal before the comparator uses it. Subsequently, local-input values remain zero, and only the two reference tokens supply nonzero values.

E.2.3 Uniform comparison signal

The reference rows differ only in the sign of k , so they retain a common physical scale η_t . After boot, their native states consist of $C = 1$ and $k = \pm 1$; data rows retain only $C = 1$. Before the first normalization, the reference native squared norm including the homogeneous reference is $3 + \|x\|^2$, and after clearing it is three. Since $q_{\text{hom}} \geq 2(d+3)$, the scale recurrence gives

$$3/q_{\text{hom}} \leq \eta_t^{-2} \leq 1, \quad 1 \leq \eta_t \leq \sqrt{q_{\text{hom}}/3}. \quad (\text{S50})$$

The controller has scale $0 < \alpha_t \leq \sqrt{q_{\text{hom}}}$, since its homogeneous reference remains fixed and (S45) preserves $\alpha_t^{-2} \geq 1/q_{\text{hom}}$.

Only one controller remainder u_i is active at a time. Writing it as u , the two reference logits are $\pm \alpha_t \eta_t u$ and the other logits are zero. The signal in native coordinates, unchanged by the first normalization, is therefore

$$a_t(u) = \frac{\eta_t}{\alpha_t} F_N(\alpha_t \eta_t u), \quad F_N(t) = \frac{2 \sinh t}{N - 2 + 2 \cosh t}. \quad (\text{S51})$$

For $t \geq 0$, $F_N(t) \geq (2/N) \tanh t$, and $F'_N(t)$ is the variance of a softmax-distributed variable in $\{-1, 0, 1\}$, hence is at most one. Put $C_{\text{att}} = q_{\text{hom}}/\sqrt{3}$. For $|u| \leq 1$, $|\alpha_t \eta_t u| \leq C_{\text{att}}$.

Monotonicity of $\tanh(t)/t$ and (S50) consequently give

$$\begin{aligned} \text{sign } a_t(u) &= \text{sign } u, & c_{\text{sig}}|u| &\leq |a_t(u)| \leq (q_{\text{hom}}/3)|u|, \\ c_{\text{sig}} &= \frac{2}{N} \frac{\tanh C_{\text{att}}}{C_{\text{att}}} > 0. \end{aligned} \quad (\text{S52})$$

These constants do not depend on the iteration, grid resolution, or padded width. No lower bound on the controller scale is needed for this comparison.

E.2.4 Complete binary-routing program

For $K = 2^m$, choose $A = 2^h$ with $0 \leq h \leq m$ and let $N_{\text{grp}} = A^d$. The desired coarse address and normalized prefixes are

$$j_i(x) = \min\{A - 1, \lfloor Ax_i \rfloor\}, \quad I(x) = \sum_{i=0}^{d-1} A^i j_i(x), \quad b_i = j_i(x)/A. \quad (\text{S53})$$

For $h \geq 1$, choose a comparison length L as specified below and write $\zeta_k = 2^{-(k-1)}$. Initializing coordinate i means setting

$$(u_i, w_i, \tau_{\text{cmp},i}, \tau_{\text{level},i}, b_i, p_i) = (2x_i - 1, 0, \zeta_L, \zeta_h, 0, 1) \quad (\text{S54})$$

and activating $e_{C,i}$. Unused coordinates already have $w_i = b_i = 0$, so only u_i, p_i and the two timers require nonzero assignments. The boot instruction performs this initialization for $i = 0$ and clears e_{boot} . In that instruction x_0 is read from the old aggregate.

In comparison phase C_i , the FFN uses the current native attention signal:

$$w_i^+ = \Pi_1(2w_i + a_{\text{att}}), \quad \tau_{\text{cmp},i}^+ = 2\tau_{\text{cmp},i}. \quad (\text{S55})$$

It stays in C_i unless the old timer equals one, in which case it moves to commit phase M_i . Thus exactly L comparisons precede every commit. The commit assignments are

$$\begin{aligned} u_i^+ &= \Pi_1(2u_i - w_i), & b_i^+ &= b_i + \text{ReLU}((p_i + w_i - 1)/2), & p_i^+ &= p_i/2, \\ w_i^+ &= 0, & \tau_{\text{cmp},i}^+ &= \zeta_L, & \tau_{\text{level},i}^+ &= 2\tau_{\text{level},i}. \end{aligned} \quad (\text{S56})$$

It returns to C_i unless the old level timer equals one, in which case it moves to handoff phase H_i . This performs exactly h commits. At H_i , set $u_i, w_i, \tau_{\text{cmp},i}, \tau_{\text{level},i}, p_i$ to zero and remove $e_{H,i}$. If $i < d - 1$, retain b_i , initialize coordinate $i + 1$ by (S54), and activate $e_{C,i+1}$. For $i = d - 1$, instead assign the old prefixes to the selector index,

$$I \leftarrow \sum_{i=0}^{d-1} \frac{A^{i+1}}{N_{\text{grp}}} b_i = I(x)/N_{\text{grp}}, \quad (\text{S57})$$

clear every b_i , and activate the selector warm-up phase $e_w = 1$. All coefficients in this address assignment are at most one. These simultaneous assignments preserve the prefixes until they have been read.

For $h = 0$, boot copies the input and passes directly to e_w with index zero; no routing timer

is used. In either case, at handoff all routing registers in the controller are zero. Its input, front-end timer, and terminal-correction code remain intact. The query coordinates are then zero, the local values are zero, and the equal-scale reference values cancel. Attention is therefore exactly zero in every subsequent iteration without changing its matrices.

E.2.5 Correctness and exceptional inputs

For a fixed remainder with $|u_i| \geq \delta$, every comparison signal has the same sign and magnitude at least $c_{\text{sig}}\delta$. Starting from $w_i = 0$, induction in (S55) gives

$$|w_i^{(j)}| \geq \min\{1, c_{\text{sig}}\delta(2^j - 1)\}.$$

Once it reaches one it stays there until the commit. Thus L comparisons return $w_i = \text{sign } u_i$ when $c_{\text{sig}}\delta(2^L - 1) \geq 1$. For this exact sign, the remainder update is the centered binary shift, and the prefix increment is $p_i/2$ for sign $+1$ and zero for sign -1 . Hence the initialized program returns the prefixes in (S53) after h bits.

Exclude inputs for which one of the dh ideal centered binary remainders has magnitude below δ , together with the measure-zero endpoint faces $x_i = 1$. Under uniform measure, each remainder event has measure δ , so the exceptional set satisfies $\mu(\mathcal{B}_{\text{route}}) \leq dh\delta$. Choose

$$\delta = K^{-2s}/(dh), \quad L = \left\lceil \log_2 \left(1 + \frac{1}{c_{\text{sig}}\delta} \right) \right\rceil. \quad (\text{S58})$$

Then $\mu(\mathcal{B}_{\text{route}}) \leq K^{-2s}$ and $L = O_{d,s,N}(m + \log(h+1))$. Off this set, induction over commits justifies the use of the ideal remainders and makes the selected address exact. On the exceptional set the phases and timers still run for their prescribed lengths: $|u_i|, |w_i| \leq 1$, $p_i = 2^{-j}$ after j commits, and every prefix increment lies in $[0, p_i/2]$. Thus $b_i \in [0, 1]$ and both timers are at most two throughout routing. The program takes exactly $1 + d\{h(L+1) + 1\}$ blocks, including boot and handoffs.

E.2.6 ReLU features and routing resource count

Use the homogeneous gates from Section E.1 with $\Lambda = 64(q_{\text{hom}} + 1)$. During routing $C = 1$ and $|a_{\text{att}}| \leq q_{\text{hom}}/3$. At later stages the routing coordinates are zero and the selector coordinates are $O(C)$. Thus every inactive affine or hinge feature is suppressed. Reference and data tokens have zero phases, so their native states remain fixed after unconditional clearing. Every feature reads the same old state.

A signed affine residual requires two units. A clipped assignment $y^+ = \Pi_1(v)$ has residual $-1 - y + \text{ReLU}(v+1) - \text{ReLU}(v-1)$ and requires four units. A counter test uses

$$\text{ReLU}\{2\tau_{\text{cmp},i} - 1 + \Lambda(e_{C,i} - C)\} \quad \text{or} \quad \text{ReLU}\{2\tau_{\text{level},i} - 1 + \Lambda(e_{M,i} - C)\}.$$

These are exact zero-one indicators at the active checks. The first alone moves C_i to M_i when needed. At a commit, one $\text{ReLU}(e_{M,i})$ unit makes the default move to C_i , and the second test redirects it to H_i . An unconditional handoff uses one $\text{ReLU}(e_{H,i})$ unit.

Comparison uses four units for its clipped update, two for its timer, and one for the

conditional phase move, at most eight in total. Commit uses four for the remainder, one for the prefix hinge, eight for bit-weight, comparator, and timer updates, and two for the phase move, at most sixteen. Handoff clears five data registers with ten units and uses one phase unit. The table below counts the hidden units needed to implement routing within the shared block; unused units are zero-padded:

Update group	Hidden units
Boot input copies and first-coordinate initialization	$4d + 9$
Comparison, commit, and handoff clears	$(8 + 16 + 11)d$
Initialization of each following coordinate	$8(d - 1)$
Final index assignment and prefix clearing	$2 + 2d$
Unconditional local-input, aggregate, and signal clearing	$4d + 2$
Total additional routing allocation	$53d + 5$

Boot counts two units for each of the $2d$ input copies, two for each of u_0, p_0 and its two timers, and one phase transition. The next-coordinate initializations each use eight units because their comparator and prefix are already zero. Final index assignment and prefix clearing are gated by H_{d-1} and share its already counted phase transition.

E.2.7 Handoff to the Taylor evaluator

Routing has supplied the group address. The front end now loads that group's code and starts the evaluator from Appendix D. The front end uses the following $d + 7$ coordinates; it loads the code directly into the evaluator and activates its first phase.

Coordinate	Role
$U_1, \dots, U_d$	Input copies scaled together with C .
C	Common scale for the selector data and active phase.
I	Group address, rescaled to an integer before lookup.
τ_{front}	Timer for warm-up and address scaling.
e_w	Warm-up phase: repeated doubling of the state scale.
e_a	Phase that initializes the address-scaling timer.
e_s	Phase that scales the address to its integer value.
e_l	Lookup phase that loads the code and starts evaluation.

Initially $C = 1$, $U_i = x_i$, and $\tau_{\text{front}} = 2^{-(L_m-1)}$, with $L_m = G + dm$ from Appendix D. Routing has already written the normalized address into I . Warm-up uses the same bounded-weight doubling updates as Section D.2, leaving I unchanged, and produces $C = C_* = 2^{L_m}$. The next phase only initializes the scaling timer and advances its clock; it does not evaluate (S31) or reset I . For $dh \geq 1$ it sets $\tau_{\text{front}} = 2^{-(dh-1)}C$ and enters e_s ; for $h = 0$ it goes directly to lookup. The scaling phase performs dh updates $I^+ = 2I$, $\tau_{\text{front}}^+ = 2\tau_{\text{front}}$ and exits at the old condition $\tau_{\text{front}} = C$. Thus the lookup address is the integer $I(x)$ on valid routing inputs.

For $j = 0, \dots, N_{\text{grp}} - 1$, let γ_j be the base-four code of the Taylor records in the coarse group with integer address j , using the encoding in (S24). The lookup hinges are

$$H_j(I, C) = \text{ReLU}(I - jC/C_* + 1) - 2\text{ReLU}(I - jC/C_*) + \text{ReLU}(I - jC/C_* - 1).$$

In that phase, add $\sum_{j=0}^{N_{\text{grp}}-1} \gamma_j H_j(I, C)$ directly to the initially zero evaluator code. Remove e_1 and initialize the evaluator's first phase to one using $C_*^{-1} \text{ReLU}(e_1)$. Simultaneously rescale

$$(C, U_i, I, \tau_{\text{front}}) \leftarrow C_*^{-1}(C, U_i, I, \tau_{\text{front}}). \quad (\text{S59})$$

Every right-hand side uses the old state, so rescaling does not change the selected code or first-phase activation. It uses $2(d+3)$ additional signed-affine units, with residual coefficients $-(1 - C_*^{-1})$. Afterwards $C = 1$, $|U_i|, |I| \leq 1$, $|\tau_{\text{front}}| \leq 2$, and all four front-end clocks are zero. The three reserved coordinates stay zero. The evaluator's first instruction resets its polynomial accumulator; its separate timer τ_{eval} is initialized only when the first counted word routine is reached. This is the program in Section D.1, including its explicit identity steps. Outside $\mathcal{B}_{\text{route}}$, it evaluates K^d/N_{grp} valid records in $K^d \tau_m/N_{\text{grp}}$ iterations. At $h = 0$ the one full-grid code is loaded in the same way.

This direct front end uses $2d + 6$ warm-up units, three timer/transition units, five address-scaling units, and $3N_{\text{grp}} + 1$ lookup/handoff units. Their sum is $3N_{\text{grp}} + 2d + 15$. The rescaling units are counted with the terminal-correction allocation $C_{\text{add}} = 2q_* + 2d + 184$ below. Together with the evaluator width r_0 from (S20) and the $53d + 5$ routing units, the sufficient hidden width is

$$r_{\text{impl}} = r_0 + C_{\text{add}} + 3N_{\text{grp}} + 55d + 20. \quad (\text{S60})$$

This count corresponds to the direct attention-addressed construction above. The same allocation covers $h = 0$ with inactive units zero-padded. Front-end gates use $\Lambda(e - C)$ and evaluator gates use their own $\Lambda_{\text{eval}}(e - 1)$ before homogenization. After handoff, the front end's auxiliary registers do not enter evaluator features. These bounds therefore suppress inactive instructions in all subsequent phases.

E.3 Terminal correction and affine output function

Taylor evaluation has produced $u \in [-1, 1]$, whose rescaled value Hu approximates the target. It remains to read this value from the physical token: an affine output function cannot divide by the token's changing reference coordinate. We therefore transform u within the shared block, clear the auxiliary coordinates, and let normalization converge to a scale at which an affine readout recovers Hu to the required accuracy.

E.3.1 Approximation of the inverse normalization map

The required native transformation is

$$g_{\text{inv}}(u) = \frac{u}{\sqrt{1 - u^2/2}}, \quad \frac{\sqrt{2}g_{\text{inv}}(u)}{\sqrt{g_{\text{inv}}(u)^2 + 2}} = u. \quad (\text{S61})$$

For $|u| \leq 1$, $|g_{\text{inv}}(u)| \leq \sqrt{2}$ and

$$g_{\text{inv}}(u) = u \sum_{j=0}^{\infty} c_j (u^2/2)^j, \quad c_j = \binom{2j}{j} 4^{-j} \in [0, 1].$$

The integers J, b, ℓ below control the precision of the terminal correction and are distinct from the Taylor-word parameters of Appendix D. Compute $t = S_\ell(|u|)/2 \in [0, 1/2]$ using Lemma D.3. Round $c_j/4$ down to a b -digit dyadic value a_j , and encode $a_J, \dots, a_0$ with continuation digits in one base-four scalar. This code is a counted input-function bias of magnitude at most $2/3$ and uses $O((J+1)b)$ binary digits. Starting at $p_{J+1} = 0$, evaluate

$$p_j = a_j + \mathbf{M}_\ell(t, p_{j+1}), \quad v = 4\mathbf{M}_\ell(u, p_0). \quad (\text{S62})$$

Since $a_j \leq 1/4$, $t \leq 1/2$, and the multiplication error is at most $4^{-\ell}/2 \leq 1/8$, induction gives $|p_j| \leq 1/4 + |p_{j+1}|/2 + 1/8 \leq 7/8 < 1$ whenever $|p_{j+1}| \leq 1$. All multiplication inputs therefore remain admissible.

Put $t_0 = u^2/2$ and let P_j be the exact truncated Horner values with coefficients $c_j/4$ and argument t_0 . Then $0 \leq P_j \leq 1/2$ and $|t - t_0| \leq 4^{-\ell}/8$. Thus

$$|p_j - P_j| \leq \frac{1}{2}|p_{j+1} - P_{j+1}| + 2^{-b} + \frac{9}{16}4^{-\ell}.$$

Summing this contracting recursion, allowing for the final multiplication, and bounding the series tail gives

$$|v - g_{\text{inv}}(u)| \leq 8 \cdot 2^{-b} + 7 \cdot 4^{-\ell} + 2^{-J}. \quad (\text{S63})$$

For $K = 2^m$, choose

$$b = \lceil sm \rceil + 6, \quad \ell = \lceil sm/2 \rceil + 4, \quad J = \lceil sm \rceil + 6.$$

The error is $O_s(K^{-s})$ and is less than $1/8$, so $|v| < 2$. The phase implementation below uses 15 data and 18 phase registers and, including selector rescaling and the Taylor-to-output handoff, at most

$$C_{\text{add}} = 2q_* + 2d + 184 \quad (\text{S64})$$

additional hidden units. Its runtime is at most

$$(J+1)(2b+2\ell+8) + 4\ell + 12 = O_s(\log^2(eK)). \quad (\text{S65})$$

All assignments and phase tests below are simultaneous and read the old state; unlisted coordinates are unchanged. The timer τ_{read} is separate from $\tau_{\text{eval}}, \tau_{\text{front}}$ and both routing timers.

E.3.2 Coordinates and initialization

The table below describes the 15 additional data coordinates and the 18 phase coordinates used for terminal correction. Symbols reused from Appendix D, including c, a, t, p, v , denote separate coordinates local to this subsection. The Taylor output u remains in its original coordinate and is unchanged until the terminal transition.

Coordinate	Role
c	Unread remainder of the encoded correction coefficients.
a	Accumulator for the coefficient being decoded.
t	Approximation to $u^2/2$, used in the Horner recursion.
p	Current value of the Horner recursion.
v_s, S_s, w_s	Square-recurrence state, approximation, and scale for $ u $.
v_+, v_-	Intermediate states of the two multiplication channels.
S_+, S_-	Square approximations whose difference gives a product.
w	Scale shared by the multiplication channels.
τ_{read}	Timer for coefficient decoding and square recurrences.
β	Continuation bit for the correction coefficient list.
v	Corrected value approximating $g_{\text{inv}}(u)$.
$e_1, \dots, e_{18}$	Phase coordinates for the correction program below.

Initially, c contains the coefficient code and the other 14 data coordinates are zero. Add 18 phase coordinates $e_1, \dots, e_{18}$, all initially zero. One hidden unit transfers the Taylor terminal phase into e_1 . Let

$$D(c) = 3 \text{ReLU}(c - 1/6) - 3 \text{ReLU}(c - 1/2), \quad \zeta_k = 2^{-(k-1)}.$$

The code contains $a_J, \dots, a_0$, each with b digits read from least to most significant. A continuation digit follows every word; all are one except the last, which is zero.

E.3.3 Square evaluation and coefficient decoding

The table below specifies how the shared block computes t and decodes one coefficient, listing each phase's updates and transition. The notation $h_s = \text{ReLU}(v_s - w_s/2)$ denotes an expression in the current state, not an additional coordinate.

Phase	Assignments	Next phase
1	$v_s = S_s = u , w_s = 1, \tau_{\text{read}} = \zeta_\ell$	2
2	$v_s^+ = v_s/2 - h_s, S_s^+ = S_s - v_s/2 + h_s$ $w_s^+ = w_s/4$	3
3	$\tau_{\text{read}}^+ = 2\tau_{\text{read}}$	4 if $\tau_{\text{read}} = 1$; otherwise 2
4	$t = S_s/2, p = a = 0, \tau_{\text{read}} = \zeta_b$	5
5	$c^+ = 4c - 2D(c), a^+ = (a + D(c))/2$	6
6	$\tau_{\text{read}}^+ = 2\tau_{\text{read}}$	7 if $\tau_{\text{read}} = 1$; otherwise 5

Absolute values are represented by $|x| = \text{ReLU}(x) + \text{ReLU}(-x)$. At the checks of a length- k iteration, the old counter values are $\zeta_k, \dots, 1/2, 1$. Thus precisely k iteration/check pairs are performed. The final check writes two into the counter, which is reset by the following initialization.

E.3.4 Multiplication and Horner updates

Each multiplication uses four phases. For inputs a_*, b_* that remain fixed until the final assignment, put $w_+ = (a_* + b_*)/2, w_- = (a_* - b_*)/2$ and $h_\pm = \text{ReLU}(v_\pm - w/2)$. The table below gives the four-phase multiplication routine used in both the Horner update and final

output.

Phase	Assignments	Next phase
i	$v_{\pm} = S_{\pm} = w_{\pm} $, $w = 1$, $\tau_{\text{read}} = \zeta_{\ell}$	$i + 1$
$i + 1$	$v_{\pm}^+ = v_{\pm}/2 - h_{\pm}$, $S_{\pm}^+ = S_{\pm} - v_{\pm}/2 + h_{\pm}$ $w^+ = w/4$	$i + 2$
$i + 2$	$\tau_{\text{read}}^+ = 2\tau_{\text{read}}$	$i + 3$ if $\tau_{\text{read}} = 1$; otherwise $i + 1$
$i + 3$	$y = \lambda(S_+ - S_-) + a_{\text{add}}$	specified below

The two uses of this routine have the following inputs, outputs, and transitions:

i	(a_*, b_*)	y	λ	a_{add}	Next phase
7	(t, p)	p	1	a	11
14	(u, p)	v	4	0	18

The two sets of phases use the same data coordinates. They implement $p \leftarrow a_j + \mathbf{M}_{\ell}(t, p)$ and $v \leftarrow 4\mathbf{M}_{\ell}(u, p)$, respectively. The coefficient four occurs only in the final assignment and does not change the multiplication input domain.

E.3.5 Continuation and terminal transition

The remaining phases either start the next coefficient or finish the output calculation and clear the auxiliary coordinates:

Phase	Assignments	Next phase
11	$c^+ = 4c - 2D(c)$, $\beta = D(c)$	12
12	none	13 if $\beta = 1$; otherwise 14
13	$a = 0$, $\tau_{\text{read}} = \zeta_b$	5
18	all old coordinates except C set to zero; all new data coordinates except v set to zero	none

At phase 18, $C = 1$ and the other 17 new phase coordinates vanish. All extra routing coordinates of the controller have already been cleared; the terminal instructions need only clear the q_* original coordinates and the terminal-correction registers listed in the table. Removing e_{18} leaves only $C = 1$ and v . All residual updates subsequently vanish, and the normalization convergence established below applies.

E.3.6 ReLU realization and resource bounds

For a phase coordinate e and an assignment

$$z_i \longleftarrow F(z) + \sum_{\nu} a_{\nu} \text{ReLU}(G_{\nu}(z)),$$

where F, G_{ν} are affine, use the preactivations

$$\pm(F(z) - z_i) + \Lambda(e - C), \quad G_{\nu}(z) + \Lambda(e - C).$$

The first pair represents the signed affine residual; each additional hinge uses one unit. At the active phase, $e = C = 1$ and the update is exact. At every other phase, the $O(C)$ bounds on the relevant coordinates make these units inactive for the same sufficiently large $\Lambda = 64(q_{\text{hom}} + 1)$. Every feature reads the same old state, so all phases are represented in one hidden layer.

An unconditional transition uses one $\text{ReLU}(e)$ unit with output coefficients -1 on the old phase and 1 on the next phase. A conditional transition adds one feature to this default move:

$$\text{ReLU}(2\tau_{\text{read}} - 1 + \Lambda(e - C)) \quad \text{or} \quad \text{ReLU}(\beta + \Lambda(e - C)).$$

On the admissible counter and continuation values, these are exact zero-one indicators. The table below counts the hidden units for these phases to establish (S64), using two units per signed affine residual and counting repeated hinges separately.

Phases	Units per phase	Total
Square (1–4)	13, 9, 4, 9	35
Coefficient decoding (5–6)	9, 4	13
Horner multiplication (7–10)	21, 15, 4, 3	43
Continuation (11–13)	9, 2, 5	16
Final multiplication (14–17)	21, 15, 4, 3	43
Terminal transition (18)	$2(q_* - 1) + 2 \cdot 14 + 1$	$2q_* + 27$

For example, phase 1 uses four units for each absolute-value assignment, two for each constant assignment and one for the phase transition, totaling $4 + 4 + 2 + 2 + 1 = 13$. The first five rows sum to 150. Adding the $2(d + 3)$ units for (S59) and one terminal-to-output transition gives $2q_* + 2d + 184$ units in total and proves (S64). The 15 data and 18 phase coordinates are precisely those reserved in (S48); no further native coordinates are needed.

Each coefficient requires at most $2b + 2\ell + 5$ iterations. Initialization, final multiplication and the two terminal transitions require at most another $4\ell + 6$. This proves the slightly looser bound (S65). Neither the coordinate count nor the number of phases depends on b, ℓ, J .

E.3.7 Normalization convergence and affine output function

The correction has produced $v \approx g_{\text{inv}}(u)$. We now show that the remaining normalization steps make it readable by an affine output. After the terminal transition, the controller’s only nonzero native coordinates are $C = 1$ and v ; its homogeneous reference remains one. All native updates and attention increments vanish. In (S45), $\|z\|_2^2 + 1 = v^2 + 2$, so, writing $d_{\alpha,0}$ for the inverse-square scale after this transition,

$$d_* = \lambda_q(v^2 + 2), \quad d_{\alpha,j} = d_* + \rho_{\text{LN}}^{2j}(d_{\alpha,0} - d_*). \quad (\text{S66})$$

On valid inputs, routing data are uniformly bounded, selector data grow at most as $CK^{2(k+1)+d+1}$, and evaluator and terminal-correction data are bounded. The contracting scale recurrence therefore gives $1/q_{\text{hom}} \leq d_{\alpha,0} \leq CK^{C_{d,s}}$, uniformly over the padded widths. Since $|v| < 2$ and $\lambda_q \geq 1/q_{\text{hom}}$, at most $C_{d,s,\varepsilon_{\text{LN}}} \log(eK)$ additional blocks ensure $|d_{\alpha,j}/d_* - 1| \leq CK^{-s}$ and also at most $1/2$.

Choose the affine output function to read the first positive copy of the controller's v coordinate with weight

$$w_q = H\sqrt{2\lambda_q}. \quad (\text{S67})$$

At the limiting scale its value is $H\sqrt{2}v/\sqrt{v^2+2}$. The map $v \mapsto \sqrt{2}v/\sqrt{v^2+2}$ is 1-Lipschitz, so (S61) and (S63) bound its error relative to Hu by CHK^{-s} . The finite-scale error has the same order by (S66). Further iterations contract the scale discrepancy, so this bound persists. The output-function weight is uniformly bounded and included in the dense endpoint count.

E.4 Proof of Theorem E.1

Proof. Fix $f_0 \in \mathcal{H}_d^s(H)$ and $K = 2^m$. Take $q_0^{\text{Tr}} = q_{\text{hom}}$ and $r_0^{\text{Tr}} = r_0 + C_{\text{add}} + 55d + 23$. For each $r \geq r_0^{\text{Tr}}$, choose the largest $0 \leq h \leq m$ whose width in (S60) is at most r . Since this width is a fixed additive constant plus $3 \cdot 2^{dh}$, bounded consecutive ratios give $N_{\text{grp}} = 2^{dh} \asymp_{d,s} \min\{r, K^d\}$. The constructions above fit every $q \geq q_{\text{hom}}$ by repeated paired encoding and every larger hidden width by zero padding.

All native instructions are affine residuals and ReLU hinges, homogenized as in Lemma E.2. They coexist in one FFN hidden layer. The initialization, routing, evaluation, and terminal-correction phases are disjoint; their inactive features vanish under the bounds verified above. It suffices to take

$$M_0^{\text{Tr}} = \max\{3B_{\text{eval}}, 64(q_{\text{hom}} + 1) + 2, N, \sqrt{1 + \varepsilon_{\text{LN}}}, 2H\}.$$

This bounds every coefficient, including the encoded tables, homogeneous bias columns, normalization gains, and affine output function. The same fixed block performs every phase, and all parameter slots are counted in $B_L^{\text{Tr}}(q, r)$ from (S47).

For $h \geq 1$, routing takes $1 + d\{h(L+1) + 1\} = O_{d,s,N}(m^2)$ blocks; for $h = 0$ it takes only boot. Selector initialization and address scaling cost $O_{d,s}(m)$. The K^d/N_{grp} Taylor records cost $O_{d,s}((K^d/N_{\text{grp}})m)$ blocks by Lemma D.2. The terminal correction costs $O_s(m^2)$ and normalization settling costs $O_{d,s,\varepsilon_{\text{LN}}}(m)$. Consequently, a common finishing time satisfies

$$T_{\text{req}} \leq C \left\{ \log^2(eK) + \frac{K^d}{r} \log(eK) \right\}. \quad (\text{S68})$$

The first term also covers the saturation case of one record per group.

Let $\mathcal{B}$ be the union of the routing exceptional set and the fine-cell localization strips of Appendix D. Then $\mu(\mathcal{B}) \leq CK^{-2s}$. On its complement, routing selects the correct integer address, the selector loads a valid table, and the Taylor output satisfies $|Hu - f_0(x)| \leq CHK^{-s}$. The terminal correction adds at most CHK^{-s} for every $T \geq T_{\text{req}}$, and clipping cannot increase this error. On $\mathcal{B}$, the final clipped error is always at most $2H$. For the resulting candidate g_T ,

$$\|g_T - f_0\|_{L^2(\mu)}^2 \leq CH^2K^{-2s} + 4H^2\mu(\mathcal{B}) \leq CH^2K^{-2s}.$$

The table codes, phase timers, block coefficients, and affine endpoints were fixed using only f_0, K, q, r . After evaluation and terminal correction, further iterations only contract the

remaining scale discrepancy on valid inputs; clipping remains available elsewhere. Enlarging C to absorb the fixed H^2 proves (S39)–(S40) for every admissible T with this same parameter choice. The selection of K from a prescribed iteration budget is made only in Section B.2, in the proof of Lemma B.1. $\square$

F Numerical illustrations and experimental details

The numerical studies illustrate the parameter-budget comparison with trained residual FFNs. We call the fitted model with one residual update block applied once ($T = 1$) *Single*; it includes the input and output functions and uses the same block family as the other methods. *Loop* repeatedly applies one fitted block, while *Untied* composes separately parameterized blocks. *Single* is the common $T = 1$ special case of *Loop* and *Untied*. Comparing *Loop* with *Single* illustrates the effect of looping with shared parameters; comparing *Untied* with *Single* illustrates the effect of greater depth with separately parameterized blocks. Each comparison uses a common cap B on all trainable input-function, block, output-function, and bias coefficients; admissible architectures can use fewer than B coefficients. Computation may differ across the three methods under the same cap. The Transformer results are theoretical.

Sections F.1–F.3 present known-function regression, energy-based generative models, and real-data prediction. Section F.4 collects the experimental protocols.

Regression is evaluated by mean squared error (MSE), classification by predictive log loss, and density estimation by squared Hellinger distance. For bounded Gaussian regression, Proposition 3.2 relates integrated squared error to Hellinger risk. Expected predictive log loss equals a method-independent conditional entropy plus the covariate-averaged Kullback–Leibler divergence between the true and fitted conditional probabilities. This divergence bounds the corresponding squared Hellinger distance from above. The multiclass tasks extend the empirical comparison beyond the theoretical binary-response model. These experiments describe finite-sample behavior under their fitting procedures; the theorems concern worst-case risk of full-class sieve MLEs.

F.1 Regression model simulation

We use localized smooth bumps, a smooth recurrence, and scaled Friedman1 in dimensions two, three, and five, respectively. Inputs are uniform on $[0, 1]^d$, and $Y = f_0(X) + \varepsilon$ with independent, mean-zero Gaussian noise of standard deviation 0.02 for bumps and recurrence and 0.05 for Friedman1. Training uses squared-error loss and Adam with validation-based tuning; test MSE is measured against the known f_0 . At the prespecified cap $B = 128$, *Loop* has the smallest mean MSE for all three families (Table S1). The largest relative reductions occur for the recurrence: 46.5% against *Single* and 70.4% against *Untied*. Table S5 reports the paired differences; uncertainty is greater for the recurrence comparison with *Untied* and both Friedman1 comparisons than for smooth bumps.

Effects of the parameter budget and sample size. Figure S1 varies the parameter cap and sample size. The upper row uses 4096 training and 1024 validation observations, with $B \in \{64, 128, 256, 512, 1024\}$ and $T = 16$ for *Loop*. *Loop* has the smallest mean MSE at every displayed budget for bumps and recurrence; for Friedman1, *Single* performs better at

Simulation	Single	Loop	Untied
Smooth bumps	$2.9193 (0.063) \times 10^{-3}$	$2.6412 (0.11) \times 10^{-3}$	$3.0873 (0.087) \times 10^{-3}$
Smooth recurrence	$2.6431 (0.18) \times 10^{-4}$	$1.4133 (0.26) \times 10^{-4}$	$4.7724 (2.6) \times 10^{-4}$
Scaled Friedman1	$1.9482 (0.72) \times 10^{-3}$	$1.6124 (0.73) \times 10^{-3}$	$3.2363 (0.67) \times 10^{-3}$

Table S1: MSE against known f_0 at $B = 128$: means over eight paired replications, with standard errors (SEs) in parentheses. Boldface marks the smallest mean in each row.

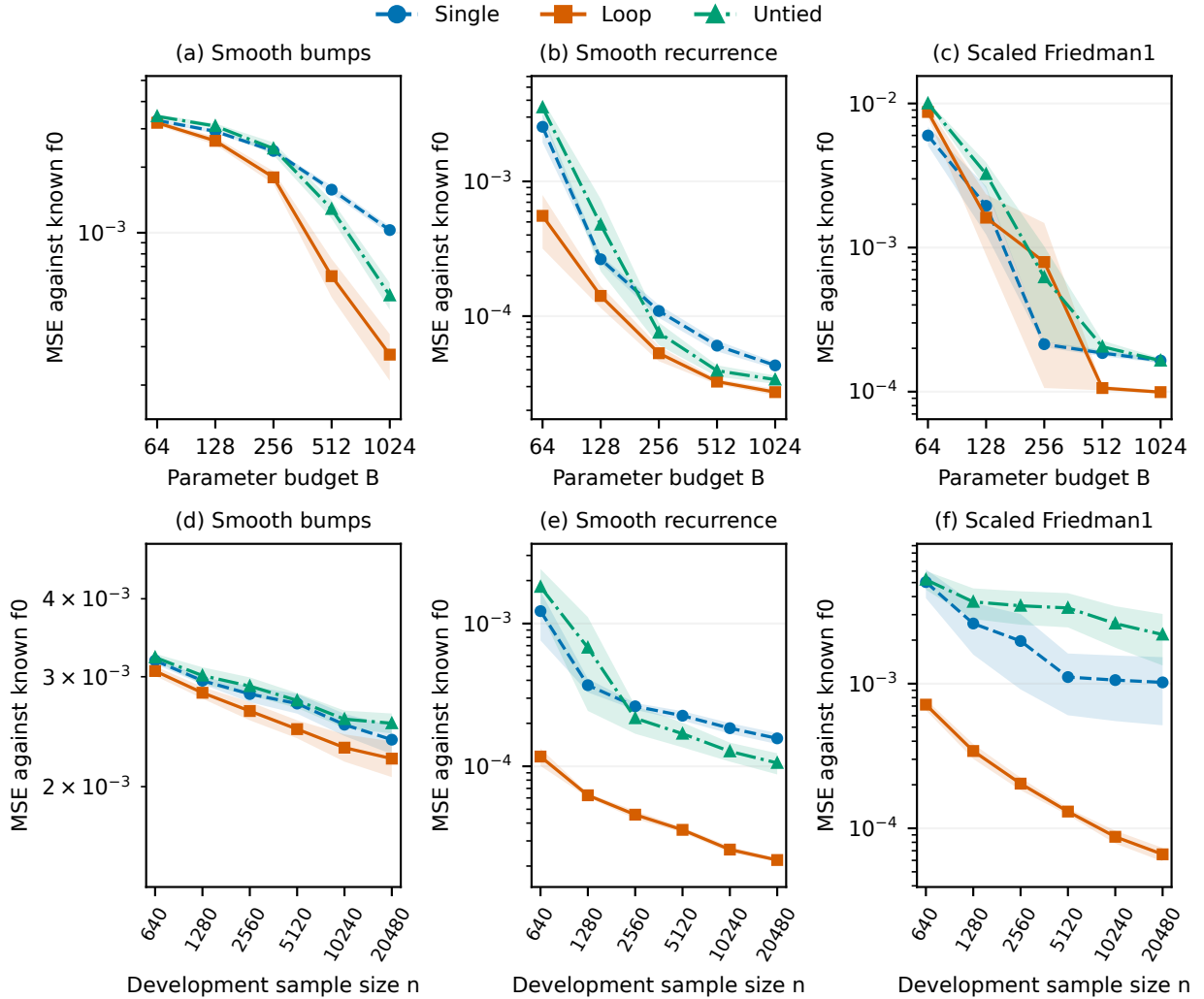


Figure S1: Effects of parameter cap B (top) and development sample size n (bottom). Columns: smooth bumps, smooth recurrence, and scaled Friedman1. Points show mean f_0 MSE over eight paired replications; shading indicates one SE. Both axes are logarithmic.

$B = 64, 256$, and Untied also performs better at $B = 256$. The lower row fixes $B = 128$ and the parameterized modules while varying the development sample size from 640 to 20480, split 80/20 between training and validation. Mean MSE decreases for all three methods, with Loop attaining the smallest mean at every displayed sample size. Each row uses its own frozen configuration and continuation protocol; larger budgets or samples can entail

more training updates. The sample-size curves keep the iteration count fixed, whereas Theorem 4.4 increases T_n with n . Sections F.4.2–F.4.4 give the configurations and update limits.

Loop iterations with continued training. Figure S2 increases Loop’s iteration count from one to 64 at $B = 128$, with fixed state and hidden widths and continued training. Mean MSE falls by 36.9%, 80.6%, and 41.6% for bumps, recurrence, and Friedman1, respectively. The recurrence gains mainly from the first increase and then fluctuates around a lower plateau; the other two curves decrease throughout. These curves reflect joint increases in iteration count and cumulative training, with a fixed parameter count.

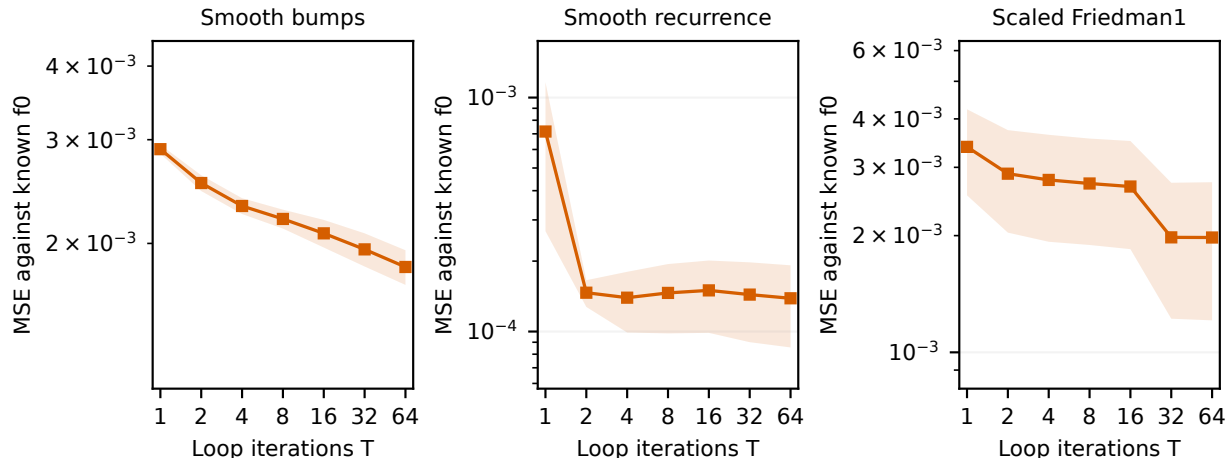


Figure S2: Loop iterations with continued training at $B = 128$. Curves show mean f_0 MSE with one-SE shading over eight replications, using the same seeds at every T . Both axes are logarithmic.

F.2 Energy-based generative-model simulation

We next evaluate density estimation with a fixed, smooth four-mode target distribution in the energy-based family, constructed independently of the fitted architectures. Using three development seeds, we screen parameter caps $B \in \{96, 128, 160, 192\}$ and training sample sizes $n_{\text{tr}} \in \{1024, 2048, 4096\}$. Within each setting and method, validation negative log likelihood (NLL) selects the architecture, optimizer setting, and checkpoint. A setting enters confirmation only if Loop has the smallest mean development squared Hellinger error and wins at least two of the three paired seeds. Before any confirmation sample is generated, this rule freezes the single strongest eligible setting, $B = 160$ and $n_{\text{tr}} = 2048$. Confirmation evaluates the selected setting on fresh samples.

Loop’s mean squared Hellinger error is 0.00892, compared with 0.01511 for Single and 0.01339 for Untied, a reduction of 33.4% relative to the second-lowest mean at the selected operating point. The test NLL means have the same ordering. In Figure S3, the top row averages the eight fitted densities for each method. The bottom row pools 4096 exact rejection draws from each fitted model, so every method panel contains 32768 generated observations. Analytic-density panels share one color scale, and sample panels use one fixed histogram, smoothing bandwidth, and color scale.

Table S2: Energy-based simulation at the development-selected setting $B = 160$, $n_{\text{tr}} = 2048$: mean (SE) over eight paired confirmation replications. Squared Hellinger errors use midpoint quadrature. Boldface marks the smallest mean in each row.

Metric	Single	Loop	Untied
Hellinger ²	0.01511 (0.0020)	0.008919 (0.0014)	0.01339 (0.0017)
Test NLL	−0.03623 (0.0040)	−0.04884 (0.0027)	−0.03996 (0.0033)

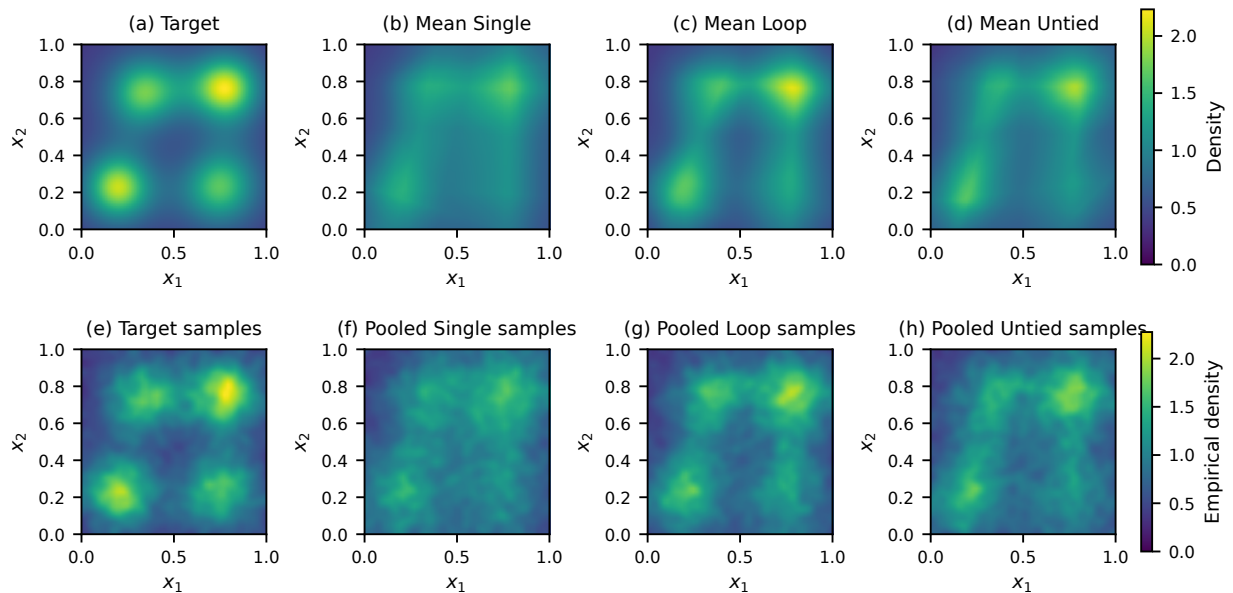


Figure S3: Four-mode density estimation at $B = 160$, $n_{\text{tr}} = 2048$. Top: target and mean fitted densities. Bottom: smoothed empirical densities from 32768 target draws or equally pooled exact rejection samples from the eight fitted models per method.

The target specification, integration grids, and fitting settings are given in Section F.4.5.

F.3 Real-data experiments

We next examine how parameter sharing affects prediction on real data. We compare Single, Loop, and Untied residual FFNs on nine regression and four multiclass classification data sets, using the same parameter cap for the three methods within each task. We evaluate regression by test MSE on the original response scale and classification by predictive log loss.

Table S3 summarizes the results. Loop has the smallest displayed mean on seven of the nine regression tasks and three of the four classification tasks. The clearest mean margins occur for SGEMM and RT-IoT. Several other orderings are small relative to the reported method-specific SEs: for example, the Wall Robot means for Loop and Single are 0.5673 and 0.5680, with SEs 0.029 and 0.036.

Table S7 summarizes regression comparisons using the paired replications. The SGEMM

Table S3: Real-data test performance: mean (SE) over eight paired replications. Boldface marks the smallest mean in each row.

Source	Metric	B	Single	Loop	Untied
<i>Regression</i>					
Airfoil	MSE	64	9.5361 (0.17)	8.9303 (0.23)	9.7895 (0.27)
Power plant	MSE	32	17.937 (0.10)	18.146 (0.36)	17.916 (0.095)
Fish toxicity	MSE	32	0.94739 (0.024)	0.94596 (0.015)	0.95003 (0.026)
White wine	MSE	64	0.59428 (0.0066)	0.59577 (0.0063)	0.59469 (0.0066)
Concrete	MSE	128	36.730 (0.81)	35.633 (0.85)	36.944 (0.42)
California housing	MSE	64	0.36494 (0.0017)	0.36079 (0.0024)	0.36365 (0.0012)
Abalone	MSE	64	4.4706 (0.016)	4.4297 (0.019)	4.4652 (0.0098)
SGEMM	MSE	64	$9.2061 (0.83) \times 10^3$	$6.2200 (0.15) \times 10^3$	$8.4769 (0.72) \times 10^3$
SARCOS	MSE	512	14.765 (0.22)	14.005 (0.19)	14.224 (0.21)
<i>Multiclass classification</i>					
Wall Robot	log loss	1024	0.5680 (0.036)	0.5673 (0.029)	0.6283 (0.044)
Maternal Risk	log loss	512	0.7294 (0.015)	0.7201 (0.013)	0.7280 (0.013)
RT-IoT	log loss	4096	0.04230 (0.0016)	0.03782 (0.0012)	0.04095 (0.0011)
Room Occupancy	log loss	1024	0.04824 (0.023)	0.04968 (0.027)	0.05580 (0.027)

differences are large relative to their paired SEs; several other contrasts at the selected budgets have substantial uncertainty.

Section F.4.6 gives the data splits, preprocessing, and fitting protocols.

F.4 Experimental protocols

This section collects the simulation functions, architectures, fitting procedures, and data sources for the results in Sections F.1–F.3.

F.4.1 Simulation functions, data streams, and error metric

For each family, inputs are independent and uniform on $[0, 1]^d$. Training and validation responses are $Y = f_0(X) + \varepsilon$, with independent Gaussian noise. The three functions are as follows.

Smooth bumps Let $k = 3$, $a_0 = 0.75$, and let $c_j \in \{-1, 1\}$ be independent equiprobable signs for the nine cells $j \in \{0, 1, 2\}^2$. For $x \in [0, 1]^2$, put $j_\ell = \min\{\lfloor kx_\ell \rfloor, k - 1\}$ and $t_\ell = 4(kx_\ell - j_\ell - 1/2)$. Define

$$\phi(t) = \begin{cases} \exp\left(\sum_{\ell=1}^2 \left[1 - \frac{1}{1 - t_\ell^2}\right]\right), & |t_1| < 1, |t_2| < 1, \\ 0, & \text{otherwise,} \end{cases} \quad f_0(x) = \frac{a_0}{k} c_j \phi(t).$$

The compact bumps are smooth, with supremum bound 0.25 and Lipschitz bound $0.75(32/e)\sqrt{2}$. The noise standard deviation (SD) is 0.02. The cell count k is unrelated to the model’s dense parameter count.

Smooth recurrence For $x \in [0, 1]^3$, set $u_0 = x_3 - 1/2$ and iterate

$$u_{j+1} = 0.25u_j + 0.1 \tanh\{2u_j + 3(x_1 - 1/2)\} + 0.08 \sin\{2\pi(x_2 + u_j)\}, \quad j = 0, \dots, 11.$$

The target is $f_0(x) = u_{12}$ and the noise SD is 0.02. The state remains in $[-1/2, 1/2]$. A conservative Lipschitz bound is

$$1 + \frac{\sqrt{0.3^2 + (0.16\pi)^2}}{1 - (0.45 + 0.16\pi)}.$$

Scaled Friedman1 For $x \in [0, 1]^5$, the target is

$$f_0(x) = 0.5 \sin(\pi x_1 x_2) + (x_3 - 1/2)^2 + 0.5x_4 + 0.25x_5.$$

This is Friedman1 divided by 20. Its supremum is bounded by 1.5, its Lipschitz constant by $\sqrt{2(\pi/2)^2 + 1 + 0.25 + 0.0625}$, and the noise SD is 0.05. Each family’s displayed supremum-plus-Lipschitz bound is below 32, consistent with the $s = 1$ smoothness setting.

For B and T , each seed spawns separate random streams for function coefficients, training inputs, validation inputs, test inputs, and the corresponding noises. A family and seed share nested input prefixes across resource values. The n study retains its own frozen random-stream implementation, likewise using nested fitting prefixes and a fixed independent test sample within each seed. Each model is evaluated on $N_{\text{test}} = 32768$ independent uniform test inputs using

$$\widehat{\text{MSE}}_{f_0} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \{\hat{f}(X_i^{\text{test}}) - f_0(X_i^{\text{test}})\}^2.$$

Thus synthetic test scores exclude response noise. All means are arithmetic across eight paired replications, one per seed. The reported SE is the sample SD, computed with denominator seven, divided by $\sqrt{8}$. Plot shading shows the mean plus or minus one SE, and table entries use mean (SE). Target centering and scaling use only the current training responses; test predictions are transformed back to the original target scale. Synthetic inputs remain on their original uniform scale.

F.4.2 Architectures and dense coefficient accounting

The input function is affine from $\mathbb{R}^d$ to $\mathbb{R}^q$. Regression and density-estimation models use an affine scalar output function. A C -class model uses an affine output function from $\mathbb{R}^q$ to $\mathbb{R}^C$, followed by the softmax map to class probabilities. Every residual block has the form

$$\Phi_\theta(z) = z + W_2 \text{ReLU}(W_1 z + b_1) + b_2, \quad W_1 \in \mathbb{R}^{r \times q}, \quad W_2 \in \mathbb{R}^{q \times r}.$$

Single executes one block once. Loop executes one learned block T times. Untied executes T independently parameterized blocks of common dimensions. The networks contain no attention or normalization layers. For a scalar output, the actual dense coefficient counts are

$$P_{\text{Single}} = P_{\text{Loop}} = q(d + 2) + 1 + (2q + 1)r + q,$$

$$P_{\text{Untied}} = q(d + 2) + 1 + T\{(2q + 1)r + q\}.$$

For a C -class output, the corresponding counts are

$$\begin{aligned} P_{\text{Single}}^{(C)} &= P_{\text{Loop}}^{(C)} = q(d + 1) + C(q + 1) + (2q + 1)r + q, \\ P_{\text{Untied}}^{(C)} &= q(d + 1) + C(q + 1) + T\{(2q + 1)r + q\}. \end{aligned}$$

Counts include every trainable dense matrix entry and bias, even when initialized to zero. A cap B is an upper bound on these counts, while iteration count and total optimization cost are tracked separately. Table S4 lists the frozen simulation configurations. The smooth-bump modules differ between the B , T , and n studies; the two rows of Figure S1 therefore retain distinct protocols.

(a) Simulation protocols

Study	Parameter cap	Training / validation observations	Loop iterations	Maximum cumulative updates
B	64, 128, 256, 512, 1024	4096/1024	16	36000
T	128	4096/1024	1, 2, 4, 8, 16, 32, 64	60000
n	128	$n_{\text{tr}} = 2^9, \dots, 2^{14};$ $n_{\text{val}} = n_{\text{tr}}/4$	16	48000

(b) Module dimensions and actual dense parameter counts

Study	Simulation	Single (q, r, T, P)	Loop (q, r, T, P)	Untied (q, r, T, P)
B	Smooth bumps	(4, 11, 1, 120)	(4, 11, 16, 120)	(6, 2, 3, 121)
B	Smooth recurrence	(3, 15, 1, 124)	(2, 23, 16, 128)	(2, 5, 4, 119)
B	Scaled Friedman1	(4, 10, 1, 123)	(5, 7, 16, 118)	(4, 3, 3, 122)
T	Smooth bumps	—	(3, 16, T , 128)	—
T	Smooth recurrence	—	(2, 23, T , 128)	—
T	Scaled Friedman1	—	(5, 7, T , 118)	—
n	Smooth bumps	(3, 16, 1, 128)	(2, 23, 16, 126)	(4, 3, 3, 110)
n	Smooth recurrence	(3, 15, 1, 124)	(2, 23, 16, 128)	(2, 5, 4, 119)
n	Scaled Friedman1	(4, 10, 1, 123)	(5, 7, 16, 118)	(4, 3, 3, 122)

Table S4: Frozen simulation configurations. Here q is the state dimension, r the residual hidden width, T the iteration count, and P the actual dense coefficient count. Panel (b) gives the B study at its cap-128 anchor and the fixed modules for the T and n studies; in the T -study rows, $T \in \mathcal{T} = \{1, 2, 4, 8, 16, 32, 64\}$. Maximum cumulative updates refer to one selected checkpoint ancestry, including initialization stages. Per-fit caps and continuation procedures are specified below.

F.4.3 Validation selection and common optimization settings

The B and T studies use diagnostic seeds 7700–7701 and confirmation seeds 7800–7807. For smooth bumps, each learner and curve receives three candidate state dimensions, obtained by adding 0, 1, 2 to its development dimension. One dimension is chosen by validation loss averaged over both diagnostic seeds and all values of that curve. The recurrence and Friedman1 retain their development dimensions. These choices are frozen before confirmation, and test evaluation begins at the confirmation stage. The cap-128 table rule is also recorded before confirmation test scoring; its models are exactly those used at $B = 128$ in the upper row of Figure S1. The n study retains its separate diagnostic at seeds 7501–7502 and its confirmation seeds 7600–7607.

At the first stage, each learner evaluates two initializations and four hyperparameter combinations: learning rate 0.001 or 0.003 and weight decay 0 or 0.001. At every subsequent stage, the four settings start from the preceding validation-selected checkpoint. Selection uses the current validation MSE and includes the step-zero checkpoint. Each fit receives its full update allowance; the selected checkpoint can occur earlier. Adam is reset for every fit, with cosine learning-rate decay to 5% of its starting value, batch size 128, and validation evaluation every 200 updates. Gradient norm is capped at five; coefficients are clamped to $[-128, 128]$ after each update. Predictions are clipped to $[-8, 8]$ on the training-target standardized scale. Computation uses deterministic 32-bit floating-point operations on a central processing unit (CPU).

For B and T , the outgoing residual matrix is initialized with multiplier $0.05/T$ and its bias is zero. The outgoing matrix and bias use Adam learning rate equal to the base rate divided by the architecture’s actual iteration count; other layers use the base rate. This rule applies to all three learners. The independently frozen n protocol uses multiplier 0.05 and the base rate uniformly across parameter groups.

F.4.4 Continuation paths and cumulative update budgets

Parameter-cap path The displayed caps are 64, 128, 256, 512, 1024, with 4096 training and 1024 validation observations and 16 Loop iterations. Each path first trains at cap 64 using 512 training and 128 validation observations, then uses the full fitting sample at cap 64 before proceeding to larger caps. State dimensions are fixed. Hidden width is the largest positive integer satisfying the cap; baseline depths follow the development allocation, reduced only when needed for feasibility at the initial cap. Increasing width pads the old weights and sets new outgoing coefficients to zero; additional untied blocks are initialized as identities. This embedding preserves the previous function. When training-target normalization changes, the output function is remapped to preserve the unclipped predictions on the original response scale. The clipped predictions are also preserved wherever clipping is inactive under both normalizations. Each fit has at most 6000 updates. The longest selected checkpoint ancestry has at most 18000 updates at the cap-128 table point and 36000 at cap 1024.

Iteration path The iteration grid is 1, 2, 4, 8, 16, 32, 64 at cap 128. Loop’s fixed (q, r) values are (3, 16), (2, 23), and (5, 7) for the three families. Each path begins at $T = 1$ with 512 training and 128 validation observations, continues at $T = 1$ with 4096 training and

Scenario	Loop minus Single	Loop minus Untied
Smooth bumps	$-2.78 (0.98) \times 10^{-4}$	$-4.46 (0.61) \times 10^{-4}$
Smooth recurrence	$-1.23 (0.15) \times 10^{-4}$	$-3.36 (2.8) \times 10^{-4}$
Scaled Friedman1	$-3.36 (5.1) \times 10^{-4}$	$-1.62 (1.1) \times 10^{-3}$

Table S5: Paired MSE differences at the prespecified parameter cap 128 for the simulations in Table S1. Entries are mean (SE) of the eight within-seed differences, with seeds paired across methods. Negative values favor Loop.

1024 validation observations, then increases iterations. On changing T , the outgoing residual coefficients are multiplied by $T_{\text{old}}/T_{\text{new}}$. For nonlinear blocks this provides an approximate continuation of the preceding function. Fits at $T \leq 16$ receive 6000 updates each; those at 32 and 64 receive 12000 each. The longest selected path at $T = 64$ is bounded by 60000 cumulative updates. The corresponding Single and Untied fits use their fixed architecture depths but receive the same stage counts, hyperparameter counts, and per-stage update caps. Figure S2 reports Loop’s performance as iteration count and cumulative training budget increase together.

Sample-size path At fixed cap 128 and the modules in Table S4, the training sizes are 512, 1024, 2048, 4096, 8192, 16384. Validation size is one quarter of training size, so the plotted total sample sizes are 640, 1280, 2560, 5120, 10240, 20480. All three learners use the same nested fitting samples within each seed. At larger samples, the output function is remapped for the new training-target normalization, preserving unclipped original-scale predictions before further fitting. Per-fit caps are 6000 updates for training sizes at most 4096 and 12000 for 8192 and 16384. The longest selected path therefore has at most 48000 updates. This experiment uses its independently frozen modules and optimization rule.

Maximum cumulative updates sum the upper bounds along one selected checkpoint ancestry, including initialization stages. They exclude rejected hyperparameter and initialization candidates. Actual selected paths can be shorter because validation may select an earlier checkpoint. Each individual fit is capped at 12000 updates. Total computation over all candidate fits is a separate resource recorded in the logs.

F.4.5 Energy-based generative-model simulation protocol

The experiment has support $[0, 1]^2$. The target log-density index, equal to the negative energy under the convention of (8), is a sum of four broad Gaussian radial basis terms centered at $(0.20, 0.23)$, $(0.74, 0.24)$, $(0.34, 0.73)$, and $(0.78, 0.76)$, with scales between 0.16 and 0.18, plus a shallow central depression. Its index lies in $[0.0634, 1.814]$ on a 512^2 reference grid, inside the bound $[-3, 3]$. Training updates compute the log normalizer on a fixed 32×32 midpoint grid; validation NLL uses a 128×128 midpoint grid. The development screen crosses $B \in \{96, 128, 160, 192\}$ with $n_{\text{tr}} \in \{1024, 2048, 4096\}$ using seeds 9000–9002, while holding the target fixed. Candidate state dimensions are $q \in \{4, 8, 12\}$, Loop iterations are $T \in \{2, 4, 8\}$, and Untied depths are $T \in \{2, 4\}$. Learning rates are $\{0.001, 0.003\}$ and weight decays are $\{0, 0.001\}$ for every method. Development uses 300 warm and 2500 refinement updates.

The development screen retains settings where Loop has the smallest mean development squared Hellinger error and at least two of three paired wins, then ranks eligible settings by relative advantage. The top-ranked setting, $B = 160, n_{\text{tr}} = 2048$, is frozen before confirmation. Confirmation seeds 9100–9107 use 2048 training draws, 2048 validation draws, 65536 test draws, 800 warm updates, and 8000 refinement updates. Development keeps three structures per method. Within confirmation, validation NLL selects structure, learning rate, weight decay, and checkpoint before test samples or Hellinger scores are computed. Test squared Hellinger error is approximated on a 256×256 midpoint grid. The scenario and model-selection rules are fixed before confirmation. Sample-panel pooling and smoothing affect only visualization.

F.4.6 Real-data protocols

Table S6 records the dimensions and parameter caps for exactly the real-data results displayed in Table S3. The dense coefficient formulas in Section F.4.2 determine which architectures satisfy each cap B , using the scalar output function for regression and the C -logit output function for classification.

Table S6: Dimensions and parameter caps for the real-data experiments in Table S3. A dash in the class column denotes a regression task.

Data set	Observations	d	Classes	B
Airfoil	1,503	5	–	64
Power plant	9,568	4	–	32
Fish toxicity	908	6	–	32
White wine	4,898	11	–	64
Concrete	1,030	8	–	128
California housing	20,640	8	–	64
Abalone	4,177	10	–	64
SGEMM	241,600	14	–	64
SARCOS	48,933	21	–	512
Wall Robot	5,456	24	4	1,024
Maternal Risk	1,014	6	3	512
RT-IoT	30,000	93	12	4,096
Room Occupancy	10,129	19	4	1,024

Each regression task uses a test set fixed across methods and replications. SARCOS retains its 4,449 official test observations, with matching input groups removed from the training pool; the other tasks hold out 20% of input groups. For each seed, the remaining groups are split 75/25 for training and validation. Imputation and standardization are fitted on the training split only. Source-specific caps were selected during development by maximizing the smaller relative validation improvement of Loop over Single and Untied. Caps and candidate shortlists were fixed before confirmation. The development candidate state dimensions are $q \in \{1, 2, 3, 4, 6, 8, 14\}$; Loop uses $T \in \{2, 4, 8, 16\}$ and Untied uses depths $\{1, 2, 3, 4, 8, 16\}$. For every candidate state dimension and iteration count, the residual hidden width r is the largest positive integer whose full dense coefficient count does not exceed the row’s cap. Confirmation uses three shortlisted architectures per method and data set, except for Fish toxicity (Single: two), Concrete (Single and Loop: two each), and SARCOS (Untied: eight).

For every shortlisted architecture, the learning rate is selected from $\{0.001, 0.003\}$ and weight decay from $\{0, 0.001\}$. Training uses a batch size of 128, 1,000 warm updates, and at most 6,000 refinement updates, with validation every 100 updates. Adam uses cosine learning-rate decay to 5% of its initial value, gradient-norm clipping at 5, a dense-coefficient clamp to $[-128, 128]$, and a prediction clip of 8 on the standardized response scale. Within each frozen shortlist, validation MSE selects the architecture, optimizer setting, and checkpoint. Table S3 reports test MSE on the original response scale as mean (SE) over eight paired confirmation replications, corresponding to seeds 8200–8207.

Data set	Loop minus Single	Loop minus Untied
Airfoil	−0.606 (0.32)	−0.859 (0.32)
Power plant	0.209 (0.3)	0.231 (0.31)
Fish toxicity	$-1.43 (18) \times 10^{-3}$	$-4.07 (21) \times 10^{-3}$
White wine	$1.49 (3.6) \times 10^{-3}$	$1.08 (3.3) \times 10^{-3}$
Concrete	−1.1 (0.69)	−1.31 (1.1)
California housing	$-4.15 (2.7) \times 10^{-3}$	$-2.86 (2.8) \times 10^{-3}$
Abalone	−0.0409 (0.012)	−0.0355 (0.016)
SGEMM	$-2.99 (0.76) \times 10^3$	$-2.26 (0.63) \times 10^3$
SARCOS	−0.761 (0.11)	−0.219 (0.15)

Table S7: Paired regression MSE differences for the validation-selected budgets in Table S3. Entries are mean (SE) of the eight within-seed differences. Negative values favor Loop.

For each classification task, a class-stratified split holds out approximately 20% of input groups for testing, fixed across methods and replications. For each seed, the remaining groups are split 75/25 for training and validation; preprocessing uses training data only. Exact duplicate feature rows remain in one split group. Wall Robot uses all 5,456 observations and 24 sensor variables, with contiguous blocks of 60 readings grouped together. Room Occupancy uses all 10,129 rows; the date and time variables are converted to a day ordinal and daily sine and cosine terms, and contiguous blocks of 120 rows define groups. Maternal Risk uses all 1,014 rows and groups duplicate feature rows. RT-IoT uses a prespecified stratified sample of 30,000 rows (sampling seed 20261101) and also groups exact duplicates.

For Wall Robot, development seeds 9700–9703 and confirmation seeds 9800–9807 use candidate state dimensions $q \in \{2, 4, 6, 8, 12, 16, 24\}$, Loop iterations $T \in \{2, 4, 8\}$, and Untied depths $\{2, 3, 4\}$. Development uses 300 warm and 2,200 refinement updates; confirmation uses 600 warm and 5,000 refinement updates. For Room Occupancy, Maternal Risk, and RT-IoT, development seeds 9900–9903 and confirmation seeds 10000–10007 use $q \in \{2, 4, 8, 12, 16\}$, the same Loop and Untied depth grids, and the same maximal feasible hidden-width rule, with 300 warm plus 1,800 refinement updates in development, and 600 warm plus 4,500 refinement updates in confirmation. Both protocols use batch size 256, validation every 100 updates, learning rates $\{0.001, 0.003\}$, and weight decays $\{0, 0.001\}$. Within each method, validation log loss selects the candidate structure, optimizer setting, and checkpoint. Table S3 reports held-out predictive log loss. Each table entry is mean (SE) across eight paired confirmation replications, one per seed.

F.4.7 Measured-source links

- Airfoil: <https://archive.ics.uci.edu/dataset/291>.
- Power plant: <https://archive.ics.uci.edu/dataset/294/combined+cycle+power+plant>.
- Fish toxicity: <https://archive.ics.uci.edu/dataset/504>.
- White wine: <https://archive.ics.uci.edu/dataset/186>.
- Concrete: <https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength>.
- California housing: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html.
- Abalone: <https://archive.ics.uci.edu/dataset/1/abalone>.
- SGEMM: <https://archive.ics.uci.edu/dataset/440/sgemm+gpu+kernel+performance>.
- SARCOS: <https://gaussianprocess.org/gpml/data/>.
- Wall Following Robot Navigation: <https://archive.ics.uci.edu/dataset/194/wall+following+robot+navigation+data>.
- Room Occupancy Estimation: <https://archive.ics.uci.edu/dataset/864/room+occupancy+estimation>.
- Maternal Health Risk: <https://archive.ics.uci.edu/dataset/863/maternal+health+risk>.
- RT-IoT2022: <https://archive.ics.uci.edu/dataset/942/rt+iot2022>.